

A Foundation for Actor Computation

Gul Agha
University of Illinois
agha@cs.uiuc.edu

Ian A. Mason
Stanford University
iam@cs.stanford.edu

Scott Smith
Johns Hopkins University
scott@cs.jhu.edu

Carolyn Talcott
Stanford University
clt@sail.stanford.edu

Abstract

We present an actor language which is an extension of a simple functional language, and provide a precise operational semantics for this extension. Actor configurations are open distributed systems, meaning we explicitly take into account the interface with external components in the specification of an actor system. We define and study various notions of equivalence on actor expressions and configurations.

1. Introduction

The modern computing environment is becoming increasingly distributed. The semantics of components of open distributed systems is in its infancy. The main features of an open distributed system are that new components can be added, existing components can be replaced, and interconnections can be changed, largely without disturbing the functioning of the system. Components have no control over the components with which they might be connected. The behavior of a component is locally determined by its initial state and the history of its interactions with the environment through its interface. The internal state of a component must only be accessible through operations provided by the interface.

The actor model of computation has a built-in notion of local component and interface, and thus it is a natural model to use as a basis for a theory of open distributed computation. The actor model was originally proposed by Hewitt [13]. Actors are self-contained, concurrently interacting entities of a computing system. They communicate via message passing which is asynchronous and fair. Actors can be dynamically created and the topology of actor systems can change dynamically. The actor model is a primitive model of computation, but nonetheless easily expresses a wide range of computation paradigms. It directly supports encapsulation and sharing, and provides a natural extension of both functional programming and object style data abstraction to concurrent open systems [1, 2].

1.1. Overview

In this paper we present a study of a particular actor language. Our actor language is an extension of the call-by-value lambda calculus that includes (in addition to arithmetic primitives and structure constructors, recognizers, and destructors) primitives for creating and manipulating actors. Our approach is motivated by a desire to bridge the gap between theory and practice. The semantic theory we develop is intended to be useful for justifying program transformations for real languages, and to formalize intuitive arguments and properties used by programmers.

In our model we make explicit the notion of open system component through the notion of an *actor configuration*. An actor configuration is a collection of individually named concurrently executing actors, plus two collections of actor names that explicitly define the interface to the environment. Following the tradition of [22, 23, 18, 10, 11] we develop the semantics in two stages. The first stage consists in giving an operational semantics for actor configurations. In the second stage various notions of equivalence are investigated, both of expressions and of configurations. The operational semantics extends that of the embedded functional language in such a way that the equational theory of the functional language is preserved, giving rise to a rich set of equational reasoning principles. There are also numerous equational laws that relate to actor computations, for instance allowing two adjacent message send operations to be permuted.

The operational semantics of actor configurations is defined by a transition relation on configurations. An important aspect of the actor model is the fairness requirement: message delivery is guaranteed, and individual actor computations are guaranteed to progress. We make the fairness requirement explicit in our semantics by requiring infinite sequences of transitions on actor configurations to be fair.

We study two notions of equivalence: observational and interaction. Two expressions/configurations are said to be observationally equivalent if they give rise to the same observations, suitably defined, inside all observing contexts. This notion is closely related to testing equivalence [8]. Observational equivalence provides a semantic basis for developing sound transformation rules for expressions. Although fairness makes some aspects of reasoning more complicated, it simplifies others. Many intuitively correct equations fail in the absence of fairness. We prove that in the presence of fairness, the three standard notions of observational equivalence collapse to two. Two configurations are said to be interaction equivalent if they behave the same considered as black box devices. Interaction equivalence implies observational equivalence. The converse is an open problem.

As a first step towards an algebra of operations on actor configurations, we define a composition operator on configurations. Composition is associative, commutative, has a unit, and is and interaction equivalence is a congruence with respect to composition. This allows large complex configurations to be studied in parts and composed to form the full system. Unlike most notions of modularity and composability, which are static, the notions we define are fundamentally dynamic ones that allow for the interface between components to evolve over time.

Outline

The remainder of this paper is organized as follows. In the remainder of this section we describe the actor model of computation in more detail. §2 gives the syntax and operational semantics of our actor language. In §3 we study the notion of observational equivalence for actor expressions. In §4 we state a variety of basic equational laws along with an intuitive explanation of how these laws are established. In §5 we consider actor configurations. We establish a composability result, define notions of equivalence, relate these to expression equivalence. In §6 we give some simple examples of equivalences between configurations and show how these may be established. In §7 we develop methods for establishing expression equivalence, and use these methods to prove the laws of §4. §8 summarizes the highlights of this paper, and discusses related and future work.

1.2. The Actor Model of Computation

In this subsection we discuss the principles underlying the actor model of computation, compare it to other models, and illustrate programming in our actor language. See [1] for more discussion of the actor model, and for many examples of programming using them.

1.2.1. The Actor Philosophy

The early work on Actors [12, 13, 14, 3, 4, 5, 16, 17] proposed some fundamental principles:

- Every object is an actor; this includes messages and numbers.
- No actor can be operated on, looked at, taken apart or modified in anyway except by sending a message to that actor requesting it perform the operation itself.
- The only thing that happens in an actor system is an event, which consists of an actor receiving a message.

- An actor can send many messages at the same time but can only receive them one at a time. In other words the arrival order is linear; while the structure of events is only partially ordered by the notion of one event preceding another.
- Actors have a set of acquaintances, other actors it knows about and can send messages to. This set can increase in time since the actor may create new acquaintances, and it may also hear about them in messages it receives. These are the only ways the set of acquaintances can increase.

The key features of actor based computation, that have evolved from these guiding principles are:

- Message passing is fundamental.
- Mail delivery is guaranteed, i.e. fairness.
- There is arrival-order non-determinism in message delivery.
- Actors are history sensitive shared objects.
- Resource allocation and interconnection topology are dynamic.

1.2.2. Related Models of Concurrency

We compare the actor model with three related models of concurrency: CSP [15], the π -calculus [20], and Concurrent ML [25, 6]. All of these models have synchronous communication as primitive instead of asynchronous communication. Synchronous communication can be implemented with asynchronous primitives and vice-versa, and forms of both are required in a realistic setting. A more important general distinction is that none of these models incorporate fairness. Without fairness, specifications fail to compose — a process may behave correctly in isolation, but may fail to do so in the presence of other processes (even if it is not interacting with them).

The work on process algebras and the π -calculus has focused on understanding elementary communications between processes, abstracting away other programming language issues. The CSP model assumes a fixed interconnection topology of processes, supports only static storage allocation, and disallows recursive procedures. The π -calculus is to concurrent programming as the pure λ -calculus is to functional programming in the sense that it provides a bare minimum of primitive notions: functions and function application in the case of the λ -calculus; and channels and communication in the case of the π -calculus. Like actors, channels are dynamically created and channel names can be communicated. Unlike actors, channels do not have state, and the π -calculus provides no form of object identity.

Recently there have been two efforts [24, 6] to combine the work on ML with the work on process algebras to obtain a concurrent version of ML. As in process algebras communication channels and concurrency are the fundamental primitive for concurrent versions of ML. Processes can be created dynamically. One process can communicate with another only if they happen to share a communication channel upon which they agree to communicate. In general any number of different processes can send or receive on a given channel, thus they suffer some of the same lack of integrity as π -calculus. In CML [24] this problem may be overcome with the use of ML data abstraction mechanisms.

1.2.3. Actor Primitives

Our actor language is an extension of the call-by-value lambda calculus that includes primitives for creating and manipulating actors. An actor's behavior is described by a lambda-abstraction which embodies the code to be executed when a message is received. The actor primitives are: **send**; **become**; **newadr**; and **initbeh**.

send is for sending messages; **send**(a, v) creates a new message with receiver a and contents v and puts the message into the message delivery system.

become is for changing behavior; **become**(b) creates an anonymous actor to carry out the rest of the current computation, alters the behavior of the actor executing the **become** to be b , and frees that actor to accept another message. This provides additional parallelism. The anonymous actor may send messages or create new actors in the process of completing its computation, but will never receive any messages as its address can never be known.

newadr and **initbeh** are for actor creation. **newadr**() creates a new (uninitialized) actor and returns its address. **initbeh**(a, b) initializes the behavior of a newly created actor with address a to be b . An uninitialized actor can only be initialized by the actor which created it. Without this restriction composability of actor configurations is problematic, as it would permit an external actor to initialize an internally created actor. The allocation of a new address and initialization of the actor's behavior have been separated in order to allow an actor to learn its own address upon initialization. An alternative would be to have a **letactor** construct similar the Scheme **letrec** construct.

1.2.4. Trivial Examples

A simple actor behavior b that expects its message to be an actor address, sends the message 5 to that address, and becomes the same behavior, may be expressed as follows.

$$b = \mathbf{rec}(\lambda y. \lambda x. \mathbf{seq}(\mathbf{send}(x, 5), \mathbf{become}(y)))$$

where **rec** is a definable call-by-value fixed-point combinator (cf. [18]). An equivalent expression of this behavior is:

$$b = \mathbf{rec}(\lambda y. \lambda x. \mathbf{seq}(\mathbf{become}(y), \mathbf{send}(x, 5)))$$

since the order of executing the **become** and the **send** cannot be observed. An expression that would create an actor with this behavior and send it some other actor address a is

$$e = \mathbf{let}\{x := \mathbf{newadr}()\}\mathbf{seq}(\mathbf{initbeh}(x, b), \mathbf{send}(x, a)).$$

The behavior of a sink, an actor that ignores its messages and becomes this same behavior, is defined by

$$\mathbf{sink} = \mathbf{rec}(\lambda b. \lambda m. \mathbf{become}(b)).$$

1.2.5. Actor Cells

It is easy to represent objects with local state in our language. As an example of this we describe an actor akin to a ML reference. The actor responds to two sorts of messages. The first is a **get** message, whose second component is an address to which the response (i.e the current contents of the cell) should be sent. The second message is a **set** message, the second component of this message should be the desired new contents of the cell-actor. We use abstract syntax to hide the gory details of the structure of messages. The desired behavior is:

```

Bcell = rec( $\lambda b.\lambda c.\lambda m.$ 
    if(get?( $m$ ),
        seq(become( $b(c)$ ), send(cust( $m$ ),  $c$ ))
    if(set?( $m$ ),
        become( $b(\mathbf{contents}(m))$ ),
        become( $b(c)$ ))))

```

So evaluating

```

let{ $a := \mathbf{newadr}()$ }seq(initbeh( $a, B_{\text{cell}}(0)$ ),  $e$ )  where
 $e = \mathbf{seq}(\mathbf{send}(a, \langle \mathbf{set}, 3 \rangle), \mathbf{send}(a, \langle \mathbf{set}, 4 \rangle), \mathbf{send}(a, \langle \mathbf{get}, b \rangle))$ 

```

will result in the actor b receiving a message containing either 0, 3 or 4, depending on the arrival order. Similarly accumulators, counters and gensym actors are easily constructed, and used.

1.2.6. Join Continuations

Simple forms of recursion are often amenable to concurrent execution. A typical example is tree recursion. Consider the problem of determining the product of the leaves of a tree (whose leaves are numbers). The problem can be recursively subdivided into the problem of computing the result for the two subtrees, and multiplying the results. The product is then returned.

```

treeprod = rec( $\lambda f.\lambda tree.$ 
    if(isnat( $tree$ ),
         $tree$ ,
         $f(\mathbf{left}(tree)) * f(\mathbf{right}(tree))$ ))

```

In the above code, a $tree$ is passed to **treeprod** which tests to see if the $tree$ is a number (i.e. a leaf). If so it returns the tree, otherwise it subdivides the problem into two recursive calls. **left** and **right** are functions which pick off the left and right branches of the tree. It is clear that the arguments to $*$ may be evaluated concurrently. It is also clear that if a zero is encountered then the computation can terminate. In this example we only deal with the former optimization. The latter optimization, made using continuations, is treated in detail in [31, 27].

Such concurrency can be implemented by using a *join continuation* which synchronizes the evaluation of the different arguments. For example, the **treeprod** program given above can be expressed in terms of actor primitives as:

```

Btreeprod = rec(λb.λself.λm.
    seq(become(b),
        if(isnat(tree(m)),
            send(cust(m), tree(m)),
            let{newcust := newadr()}
                seq(initbeh(newcust, Bjoincont(cust(m), 0, nil))
                    send(self, <left(tree(m)), newcust>)
                    send(self, <right(tree(m)), newcust>))))))

Bjoincont = rec(λb.λcust.λnargs.λfirstnum.λnum
    if(eq(nargs, 0),
        become(b(cust, 1, num)),
        seq(become(sink),
            send(cust, firstnum * num))))

```

When an actor with the **treeprod** behavior, B_{treeprod} , receives a list represented as a tree it creates a customer, called a *join continuation*, and sends two messages to tree-product to evaluate the two halves of the tree. The join continuation (with behavior B_{joincont}) receives two numbers representing the computation of the products of each of the two subtrees. The join continuation then proceeds to multiply the two numbers and send the result to the original requester. Because multiplication is commutative, we need not be concerned about matching the responses to the order of the parameters. If we were dealing with an operator which was not commutative, we would need to tag the message corresponding to each argument and this tag would be returned with the response from the corresponding subcomputation. The replacement behavior of the join continuation would then depend on the order in which the evaluation of arguments was completed.

An advantage of explicit join continuations is that they provide considerable flexibility – they can be used to control the evaluation order, to do partial computations, and to do dynamic error handling. For example, if the number 0 is encountered, the join continuation can immediately return a 0 – in some cases without waiting for the results of evaluating the other subtree.

The above program may not be optimal in other respects. For example **sends** may be quite expensive. Consequently it may be prudent to check that the subcomputation is worth dispatching. This observation together with the fact, proved in §4, that **sends** of

values commute leads to the possibly faster version:

```

Btreeproduct = rec( $\lambda b. \lambda self. \lambda m.$ 
    seq(become( $b$ ),
        if(isnat(tree( $m$ )),
            send(cust( $m$ ), tree( $m$ )),
            let{newcust := newadr()}
                seq(initbeh(newcust, Bjoincont(cust( $m$ ), 0, nil))
                    let{l := left(tree( $m$ )), r := right(tree( $m$ ))}
                        if(or(isnat( $l$ ), isnat( $r$ )),
                            send(cust( $m$ ), treeprod( $l$ ) * treeprod( $r$ )),
                            seq(send(self, <math>l, newcust>math>)
                                send(self, <math>r, newcust>math>))))))

```

1.3. Notation.

We use the usual notation for set membership and function application. Let Y, Y_0, Y_1 be sets. We specify meta-variable conventions in the form: let y range over Y , which should be read as: the meta-variable y and decorated variants such as $y', y_0, \dots$, range over the set Y . Y^n is the set of sequences of elements of Y of length n . Y^* is the set of finite sequences of elements of Y . $\bar{y} = [y_1, \dots, y_n]$ is the sequence of length $\text{Len}(\bar{y}) = n$ with i th element y_i . (Thus $[]$ is the empty sequence.) $u * v$ denotes the concatenation of the sequences u and v . If u is a non-empty sequence, then $\text{Last}(u)$ is the last element of u . $\mathbf{P}_\omega[Y]$ is the set of finite subsets of Y . $\mathbf{M}_\omega[Y]$ is the set of (finite) multi-sets with elements in Y . $Y_0 \xrightarrow{f} Y_1$ is the set of finite maps from Y_0 to Y_1 . $[Y_0 \rightarrow Y_1]$ is the set of total functions, f , with domain Y_0 and range contained in Y_1 . We write $\text{Dom}(f)$ for the domain of a function and $\text{Rng}(f)$ for its range. For any function f : $f\{y := y'\}$ is the function f' such that $\text{Dom}(f') = \text{Dom}(f) \cup \{y\}$, $f'(y) = y'$, and $f'(z) = f(z)$ for $z \neq y, z \in \text{Dom}(f)$; and $f \upharpoonright Y$ is the restriction of f to the set Y .

2. A Simple Lambda Based Actor Language

In this section we give the syntax and operational semantics of our actor language.

2.1. Syntax

We take as given countable sets $\mathbb{X}$ (variables) and At (atoms). $\mathbb{F}_n$ is the set of primitive operations of arity n and $\mathbb{F} = \bigcup_{n \in \mathbb{N}} \mathbb{F}_n$. We assume At contains $\mathbf{t}, \mathbf{nil}$ for booleans, as well as constants for natural numbers, $\mathbb{N}$. $\mathbb{F}$ contains arithmetic operations, recognizers **isatom** for atoms, and **isnat** for numbers (arities 1, 1), branching **br** (arity 3), pairing **ispr, pr**, **1st, 2nd** (arities 1, 2, 1, 1), and actor primitives **newadr**, **initbeh**, **send**, and **become** (arities 0, 2, 2, 1). The sets of expressions, $\mathbb{E}$, value expressions (or just values), $\mathbb{V}$, and contexts (expressions with holes), $\mathbb{C}$, are defined inductively as follows.

Definition ($\mathbb{E} \quad \mathbb{V} \quad \mathbb{C}$):

$$\begin{aligned}\mathbb{V} &= \text{At} \cup \mathbb{X} \cup \lambda \mathbb{X}. \mathbb{E} \cup \mathbf{pr}(\mathbb{V}, \mathbb{V}) \\ \mathbb{E} &= \text{At} \cup \mathbb{X} \cup \lambda \mathbb{X}. \mathbb{E} \cup \mathbf{app}(\mathbb{E}, \mathbb{E}) \cup \mathbb{F}_n(\mathbb{E}^n) \\ \mathbb{C} &= \text{At} \cup \mathbb{X} \cup \lambda \mathbb{X}. \mathbb{C} \cup \mathbf{app}(\mathbb{C}, \mathbb{C}) \cup \mathbb{F}_n(\mathbb{C}^n) \cup \{\bullet\}\end{aligned}$$

We let x, y, z range over $\mathbb{X}$, v range over $\mathbb{V}$, e range over $\mathbb{E}$, and C range over $\mathbb{C}$. Since we are working with a syntactic reduction semantics, there is no distinction between a value expression and the value it denotes. Hence we use the terms *value* and *value expression* interchangeably. $\lambda x.e$ binds the variable x in the expression e . We write $\text{FV}(e)$ for the set of free variables of e . We write $e[x := e']$ to denote the expression obtained from e by replacing all free occurrences of x by e' , avoiding the capture of free variables in e' . Contexts are expressions with holes. We use $\bullet$ to denote a hole. $C[e]$ denotes the result of replacing any holes in C by e . Free variables of e may become bound in this process. **let**, **if**, and **seq** are the usual syntactic sugar, **seq** being a sequencing primitive.

$$\begin{aligned}\mathbf{let}\{x := e_0\}e_1 &\text{ abbreviates } \mathbf{app}(\lambda x.e_1, e_0) \\ \mathbf{seq}(e_0, e_1) &\text{ abbreviates } \mathbf{app}(\mathbf{app}(\lambda z.\lambda x.x, e_0), e_1) \\ \mathbf{if}(e_0, e_1, e_2) &\text{ abbreviates } \mathbf{app}(\mathbf{br}(e_0, \lambda z.e_1, \lambda z.e_2), \mathbf{nil}) \quad \text{for } z \text{ fresh}\end{aligned}$$

2.2. Reduction Semantics for Actor Configurations

We give the semantics of actor expressions by defining a transition relation on actor configurations. Actor configurations have some (but not necessarily all) of the addresses of internal actors known to the outside world. These known actors are called *receptionists* because they serve as the receiving interface with the environment. An open configuration may also know addresses of some actors in the outside world. These actors are called *external actors*. The sets of receptionists and external actors are the interface of an actor configuration to its environment. They specify what actors are visible and what actor connections must be provided for the configuration to function. Both the set of receptionists and the set of external actors may grow as the configuration evolves. In addition, an actor configuration contains an actor map and a multi-set of pending messages. An actor map is a finite map from actor addresses to actor states. Each actor state is one of

- (? _{a}) uninitialized, having been newly created by an actor named a ;
- (b) ready to accept a message, where b is its behavior, a lambda abstraction; or
- [e] busy executing e , here e represents the actor's current (local) processing state.

A message m contains the address of the actor to whom it is sent and the message contents, $\langle a \Leftarrow v \rangle$. The contents v can be any value constructed from atoms and actor addresses using constructors. Lambda abstractions and constructions containing lambda abstractions are not allowed to be communicated in messages. There are three reasons for this restriction. Firstly, allowing lambda abstractions to be communicated in values violates the actor principle that only an actor can change its own behavior, because a **become** in a lambda message may change the receiving actor behavior. Secondly, if lambda abstractions are communicated to external actors, there is no reasonable way to control what actor addresses are actually exported. This has unpleasant consequences in reasoning

about equivalence, amongst other things. This restriction is not a serious limitation since the address of an actor whose behavior is the desired lambda abstraction can be passed in a message. Thirdly, if lambda abstractions can be communicated in messages then syntactic extensions to the language that involve transformations such as CPS can not be done on a per actor basis, since this would require transformation of code that might arrive in a message.

We classify actor configuration transitions as internal or external to the configuration. The internal transitions are:

rcv receipt of a message by an actor not currently busy computing; and
exec an actor executing a step of its current computation.

The internal transitions involve a single active actor, which we call the *focus actor* for the transition. **exec** transitions may be purely local (a λ -transition), or messages may be sent, or a new actor may be created, or a newly created actor may be initialized. **rcv** transitions consume a message and the focus actor becomes busy.

In addition to the internal transitions of a configuration, there are i/o transitions that correspond to interactions with external agents:

in arrival of a message to a receptionist from the outside; and
out passing a message out to an external actor.

2.2.1. Actor Configurations

We assume that we are given a countable set Ad of actor addresses. To simplify notation, we identify Ad with $\mathbb{X}$. This pun is useful for two reasons: it allows us to use expressions to describe actor states and message contents; and it allows us to avoid problems of choice of names for newly created actors by appealing to an extended form of alpha conversion. (See [18, 10] for use of this pun to represent reference cells.)

Definition (cV As $\mathbb{M}$): The set of communicable values, cV , the set of actor states, As , and the set of messages, $\mathbb{M}$, are defined as follows.

$$\text{cV} = \text{At} \cup \mathbb{X} \cup \text{pr}(\text{cV}, \text{cV}) \quad \text{As} = (?_{\mathbb{X}}) \cup (\text{V}) \cup [\text{E}] \quad \mathbb{M} = \langle \text{V} \Leftarrow \text{V} \rangle$$

We let cv range over cV and m range over $\mathbb{M}$.

Definition (Actor Configurations): An actor configuration with actor map, α , multi-set of messages, μ , receptionists, ρ , and external actors, χ , is written

$$\langle\langle \alpha \mid \mu \rangle\rangle_{\chi}^{\rho}$$

where $\rho, \chi \in \mathbf{P}_{\omega}[\mathbb{X}]$, $\alpha \in \mathbb{X} \xrightarrow{f} \text{As}$, and $\mu \in \mathbf{M}_{\omega}[\mathbb{M}]$. Further, it is required that, letting $A = \text{Dom}(\alpha)$, the following constraints are satisfied:

- (0) $\rho \subseteq A$ and $A \cap \chi = \emptyset$ (receptionists are internal and external actors are external),
- (1) if $\alpha(a) = (?_{a'})$, then $a' \in A$ (uninitialized actors created are locally),
- (2) if $a \in A$, then $\text{FV}(\alpha(a)) \subseteq A \cup \chi$, and if $\langle v_0 \Leftarrow v_1 \rangle \in \mu$, then $\text{FV}(v_i) \subseteq A \cup \chi$ for $i < 2$ (χ set is complete).

We let $\mathbb{K}$ denote the set of actor configurations and let κ range over $\mathbb{K}$. The *receptionists* ρ are names of actors within the configuration that external components may freely interact with; all other actors in the (actor) configuration are local and thus inaccessible from the outside. The *external* actors χ are names of actors that are outside this configuration but to which messages may be directed. A configuration in which both the receptionist and external actor sets are empty is said to be *closed*. For closed configurations we may omit explicit mention of the empty ρ and χ sets. The actor map portion of a configuration is presented as a list of actor states each subscripted by the actor address which is mapped to this state. If $\alpha'(a) = (b)$, and α is α' with a omitted from its domain, we write α' as $(\alpha, (b)_a)$ to focus attention on a . Similarly for other states subscripted with addresses.

The set of possible computations of an actor configuration is defined in terms of the labelled transition relation $\mapsto$ on configurations. Although this is, on the surface an interleaving semantics, it is easy to modify our transition system to obtain a truly concurrent semantics either by forming a labelled transition system with independence [26], or by using concurrent rewriting [19].

2.2.2. Decomposition and Reduction

To describe the internal transitions other than message receipt, a non-value expression is decomposed uniquely into a reduction context filled with a redex. Reduction contexts are expressions with a unique hole that play the role of continuations in abstract machine models of sequential computation. In order to distinguish holes used for different purposes, we use the sign $\square$ for the hole occurring in a reduction context, and call such holes redex holes (although they may in fact be filled with non redex expressions). We have defined the decomposition to correspond to a left-most, outer-most, call-by-value evaluation order, thus preserving the semantics of the embedded functional language.

Definition ($\mathbb{E}_{\text{rdx}}$ $\mathbb{R}$): The set of redexes, $\mathbb{E}_{\text{rdx}}$, and the set of reduction contexts, $\mathbb{R}$, are defined by

$$\begin{aligned}\mathbb{E}_{\text{rdx}} &= \mathbf{app}(\mathbb{V}, \mathbb{V}) \cup (\mathbb{F}_n(\mathbb{V}^n) - \mathbf{pr}(\mathbb{V}, \mathbb{V})) \\ \mathbb{R} &= \{\square\} \cup \mathbf{app}(\mathbb{R}, \mathbb{E}) \cup \mathbf{app}(\mathbb{V}, \mathbb{R}) \cup \mathbb{F}_{n+m+1}(\mathbb{V}^n, \mathbb{R}, \mathbb{E}^m)\end{aligned}$$

We let R range over $\mathbb{R}$.

An expression e is either a value or it can be decomposed uniquely into a reduction context filled with a redex. Thus, local actor computation is deterministic.

Lemma (Unique decomposition):

- (0) $e \in \mathbb{V}$, or
- (1) $(\exists! R, r)(e = R[r])$

Proof: An easy induction on the structure of e . $\square$

Redexes can be split into two classes, purely functional and actor redexes. The actor redexes are: **newadr**(), **initbeh**(v_0, v_1), **become**(v), and **send**(v_0, v_1). Reduction rules for the purely functional case are given by a relation $\mapsto^A$ on *expressions*.

Definition ($\xrightarrow{\lambda}$):

$$(\text{beta-v}) \quad R[\mathbf{app}(\lambda x.e, v)] \xrightarrow{\lambda} R[e[x := v]]$$

$$(\text{delta}) \quad R[\delta(v_1, \dots, v_n)] \xrightarrow{\lambda} R[v']$$

where δ is an n -ary algebraic operation, $v_1, \dots, v_n \in \mathbb{A}t^n$, and $\delta(v_1, \dots, v_n) = v'$.

$$(\text{br}) \quad R[\mathbf{br}(v_1, v_2, v)] \xrightarrow{\lambda} \begin{cases} R[v_1] & \text{if } v \in \mathbb{V} - \{\mathbf{nil}\} \\ R[v_2] & \text{if } v = \mathbf{nil} \end{cases}$$

$$(\text{ispr}) \quad R[\mathbf{ispr}(v)] \xrightarrow{\lambda} \begin{cases} R[\mathbf{t}] & \text{if } v \in \mathbf{pr}(\mathbb{V}, \mathbb{V}) \\ R[\mathbf{nil}] & \text{if } v \in \mathbb{V} - \mathbf{pr}(\mathbb{V}, \mathbb{V}) \end{cases}$$

$$(\text{fst}) \quad R[\mathbf{1}^{\text{st}}(\mathbf{pr}(v_0, v_1))] \xrightarrow{\lambda} R[v_0]$$

$$(\text{snd}) \quad R[\mathbf{2}^{\text{nd}}(\mathbf{pr}(v_0, v_1))] \xrightarrow{\lambda} R[v_1]$$

$$(\text{eq}) \quad R[\mathbf{eq}(v_0, v_1)] \xrightarrow{\lambda} \begin{cases} R[\mathbf{t}] & \text{if } v_0 = v_1 \in \mathbb{A}t \\ R[\mathbf{nil}] & \text{if } v_0, v_1 \in \mathbb{A}t \text{ and } v_0 \neq v_1 \end{cases}$$

The single-step transition relation $\mapsto$ on actor configurations is generated by the following rules. Each rule is given a label l consisting of a tag indicating the primitive instruction, and additional parameters. In all cases other than **i/o** transitions (with tags **in**, **out**) the first parameter names the *focus* actor of the transition.

Definition ($\mapsto$):

$$\langle \mathbf{fun} : a \rangle \quad e \xrightarrow{\lambda} e' \Rightarrow \langle\langle \alpha, [e]_a \mid \mu \rangle\rangle_x^\rho \mapsto \langle\langle \alpha, [e']_a \mid \mu \rangle\rangle_x^\rho$$

$$\langle \mathbf{new} : a, a' \rangle \quad \langle\langle \alpha, [R[\mathbf{newadr}()]]_a \mid \mu \rangle\rangle_x^\rho \mapsto \langle\langle \alpha, [R[a']]_a, (?_a)_{a'} \mid \mu \rangle\rangle_x^\rho \quad a' \text{ fresh}$$

$$\langle \mathbf{init} : a, a' \rangle \quad \langle\langle \alpha, [R[\mathbf{initbeh}(a', v)]]_a, (?_a)_{a'} \mid \mu \rangle\rangle_x^\rho \mapsto \langle\langle \alpha, [R[\mathbf{nil}]]_a, (v)_{a'} \mid \mu \rangle\rangle_x^\rho$$

$$\langle \mathbf{bec} : a, a' \rangle \quad \langle\langle \alpha, [R[\mathbf{become}(v)]]_a \mid \mu \rangle\rangle_x^\rho \mapsto \langle\langle \alpha, [R[\mathbf{nil}]]_{a'}, (v)_a \mid \mu \rangle\rangle_x^\rho \quad a' \text{ fresh}$$

$$\langle \mathbf{send} : a, m \rangle \quad \langle\langle \alpha, [R[\mathbf{send}(v_0, v_1)]]_a \mid \mu \rangle\rangle_x^\rho \mapsto \langle\langle \alpha, [R[\mathbf{nil}]]_a \mid \mu, m \rangle\rangle_x^\rho \quad m = \langle v_0 \Leftarrow v_1 \rangle$$

$$\langle \mathbf{rcv} : a, cv \rangle \quad \langle\langle \alpha, (v)_a \mid \langle a \Leftarrow cv \rangle, \mu \rangle\rangle_x^\rho \mapsto \langle\langle \alpha, [\mathbf{app}(v, cv)]_a \mid \mu \rangle\rangle_x^\rho$$

$$\langle \mathbf{out} : m \rangle \quad \langle\langle \alpha \mid \mu, m \rangle\rangle_x^\rho \mapsto \langle\langle \alpha \mid \mu \rangle\rangle_x^{\rho'}$$

if $m = \langle a \Leftarrow cv \rangle$, $a \in \chi$, and $\rho' = \rho \cup (\mathbf{FV}(cv) \cap \mathbf{Dom}(\alpha))$

$$\langle \mathbf{in} : m \rangle \quad \langle\langle \alpha \mid \mu \rangle\rangle_x^\rho \mapsto \langle\langle \alpha \mid \mu, m \rangle\rangle_{\chi \cup (\mathbf{FV}(cv) - \mathbf{Dom}(\alpha))}^\rho$$

if $m = \langle a \Leftarrow cv \rangle$, $a \in \rho$ and $\mathbf{FV}(cv) \cap \mathbf{Dom}(\alpha) \subseteq \rho$

We write $\kappa_0 \xrightarrow{l} \kappa_1$ if $\kappa_0 \mapsto \kappa_1$ according to the rule labelled by l . We call this triple a labelled transition. We say l is *enabled in configuration* κ if there is some κ' such that

$\kappa \xrightarrow{l} \kappa'$. The transitions are labelled to allow us to reason about sequences of transitions in terms of the rules applied, and to allow for alternative representation of computations, including: sequences of configurations; sequences of labeled transitions; and sequences of labels. Note that we have chosen the labels to include sufficient information that κ' is uniquely determined by κ and l .

i/o transitions are transitions with tags **in** or **out** and **rcv** transitions are transitions with tag **rcv**. The remaining transitions are called **exec** transitions. The **exec** transitions correspond to the execution of functional or actor redexes.

Subtilties We allow ill-formed messages to be created, but such messages can never be delivered. The last three rules assure this by restricting the form of the message: the target must be an actor and the contents must be a communicable value. In the case of input, the actor is further restricted to be a receptionist. We could easily prevent the formation of ill-formed messages, and actor states that have non-behaviors.

A clone produced to carry on after a become is not allowed to initialize an actor created by its cloner. This is a technical simplification, with some additional bookkeeping we could keep track of cloners and allow clones to initialize. Alternatively, this technical detail would disappear if we used the **letactor** construct for actor creation.

These choices affect the details of expression equivalence, but not the basic properties. Such choices will become more important if we want to model an implemented language and consider matters such as signaling of exceptions.

2.2.3. Computation Sequences and Paths

Definition (Computation trees): If κ is a configuration, then we define the *computation tree* for κ , $\mathcal{T}(\kappa)$, to be the set of all finite sequences of labeled transitions of the form $[\kappa_i \xrightarrow{l_i} \kappa_{i+1} \mid i < n]$ for some $n \in \mathbb{N}$. We call such sequences *computation sequences* and let ν range over them.

Lemma (Anonymity): If $\kappa \xrightarrow{\langle \text{bec} : a, a' \rangle} \kappa'$ and ν is any computation sequence in the computation tree, $\mathcal{T}(\kappa)$, then ν contains no transitions with label of the form $\langle \text{send} : a', v \rangle$.

Definition (Computation paths): The sequences of a computation tree are partially ordered by the initial segment relation. A *computation path* from κ is a maximal linearly ordered set of computation sequences in the computation tree, $\mathcal{T}(\kappa)$. Note that a path can also be regarded as a (possibly infinite) sequence of labelled transitions. We use $\mathcal{T}^\infty(\kappa)$ to denote the set of all paths from κ , and let π range over $\mathcal{T}^\infty(\mathbb{K})$. When thinking a path as a possibly infinite sequence we write $[\kappa_i \xrightarrow{l_i} \kappa_{i+1} \mid i < \infty]$ where $\infty \in \mathbb{N} \cup \{\omega\}$ is the length of the sequence.

Since the result of a transition is uniquely determined by the starting configuration and the transition label, computation sequences and paths can be equally represented by their initial configuration and the sequence of transitions labels. The sequence of configurations can be computed by induction on the index of occurrence.

Definition (Cfig): Let κ be a configuration and let $L = [l_i \mid i < \infty]$ be a sequence of labels corresponding to a computation from κ . The i th configuration of the computation from κ determined by L , $\text{Cfig}(\kappa, L, i)$, is defined by induction on i as follows.

(0) $\text{Cfig}(\kappa, L, 0) = \kappa$

- (1) $\text{Cfig}(\kappa, L * [l_i], i + 1) = \kappa'$ where $\text{Cfig}(\kappa, L, i) \xrightarrow{l_i} \kappa'$.

Thus, the path π determined by κ, L is the sequence

$$[\text{Cfig}(\kappa, L, i) \xrightarrow{l_i} \text{Cfig}(\kappa, L, i + 1) \mid i < \infty]$$

This notation has the advantage that when an initial starting configuration is fixed, either implicitly or explicitly, computation sequences in the computation tree can be identified with sequences of labels. When the sequence L is finite we let $\text{Cfig}(\kappa, L)$ denote the final configuration: $\text{Cfig}(\kappa, L) = \text{Cfig}(\kappa, L, \text{Len}(L))$.

Definition (multi-step transition): Let $L = [l_j \mid j < n]$ be a finite sequence of transition labels (possibly empty). L is a *multi-step transition* $\kappa \xrightarrow{L} \kappa'$ just if $\text{Cfig}(\kappa, L) = \text{Cfig}(\kappa, L, \text{Len}(L)) = \kappa'$. Or in other words, we can find a $[\kappa_j \mid j \leq n]$ such that $\kappa = \kappa_0$, $\kappa' = \kappa_n$, and $[\kappa_j \xrightarrow{l_j} \kappa_{j+1} \mid j < n]$.

2.2.4. Fairness

We do not consider all paths admissible. We rule out those computations that are unfair, i.e. those in which there is some transition that should eventually happen and does not.

Definition (Fair paths): A path $\pi = [\nu_i \xrightarrow{l_i} \nu_{i+1} \mid i < \infty]$ in the computation tree $\mathcal{T}(\kappa)$ is fair if each enabled transition eventually happens or becomes permanently disabled. That is, if l is enabled in κ_i and is not of the form $\langle \text{in} : m \rangle$, then $\kappa_j \xrightarrow{l} \kappa_{j+1}$ for some $j \geq i$, or l has the form $\langle \text{rcv} : a, cv \rangle$ and for some $j \geq i$ a is busy and never again becomes ready to accept a message. For a configuration κ we define $\mathcal{F}(\kappa)$ to be the subset of $\mathcal{T}^\infty(\kappa)$ that are fair.

Note that finite computation paths are fair, since by maximality all of the enabled transitions must have happened.

3. Equivalence of Expressions

In this section we study equivalence of expressions of our actor language. Our notion of equivalence is a combination of the now classic *operational equivalence* of [23] and *testing equivalence* of [8]. For the deterministic functional languages of the sort Plotkin studied, this equivalence is defined as follows. Two program expressions are said to be equivalent if they behave the same when placed in any observing context. An observing context is some complete program with a hole, such that all of the free variables in the expressions being observed are captured when the expressions are placed in the hole. The notion of “behave the same” is (for deterministic functional languages) typically equi-termination, i.e. either both converge or both diverge, but is usually insensitive to minor variations.

3.1. Events

The first step is to find proper notions of “observing context” and “behave the same” in an actor setting. The analogue of an observing context is an observing actor configuration: a configuration that contains an actor state with a hole. Since termination is not relevant for actor configurations, we instead introduce an observer primitive, **event** and observe whether or not in a given computation, **event** is executed. This approach is similar to that used in defining testing equivalence for CCS [8]. Since the language is nondeterministic, three different observations may be made instead of two: either **event** occurs for all possible executions, it occurs in some executions but not others, or it never occurs.

Definition (event): Formally, the language of observing contexts is obtained by introducing a new 0-ary primitive operator, **event**. We extend the reduction relation $\mapsto$ by adding the following rule.

$$\langle \mathbf{e} : a \rangle \quad \llbracket \alpha, [R[\mathbf{event}()]]_a \mid \mu \rrbracket_x^\rho \mapsto \llbracket \alpha, [R[\mathbf{nil}]]_a \mid \mu \rrbracket_x^\rho$$

Definition ($\mathbb{O}$): For an expression e , the observing configurations are configurations over the extended language of the form $\llbracket \alpha, [C]_a \mid \mu \rrbracket$, such that filling the hole in C with e results in a closed configuration. We use $\mathbb{O}$ to denote the set of observing configurations, and let O range over $\mathbb{O}$.

In our definition of observing configuration, the holes appear in the current state of an single executing actor. It is not hard to see that allowing holes in any actor state does not change the resulting notion of equivalence. A generalization of this fact, (**ocx**) is proved in §7.

We observe **event** transitions in the fair paths. We say that a path succeeds, **s**, if an **event** transition occurs in it, otherwise it fails, **f**. $obs(\pi)$ is the **s/f** observation of a single complete computation π , and $Obs(\kappa)$ is the set of observations possible for all paths of a closed actor configuration.

Definition (observations): Let κ be a configuration of the extended language, and let $\pi = [\kappa_i \xrightarrow{l_i} \kappa_{i+1} \mid i < \bowtie]$ be a fair path, i.e. $\pi \in \mathcal{F}(\kappa)$. Define

$$obs(\pi) = \begin{cases} \mathbf{s} & \text{if } (\exists i < \bowtie, a)(l_i = \langle \mathbf{e} : a \rangle) \\ \mathbf{f} & \text{otherwise} \end{cases}$$

$$Obs(\kappa) = \begin{cases} \mathbf{s} & \text{if } (\forall \pi \in \mathcal{F}(\kappa))(obs(\pi) = \mathbf{s}) \\ \mathbf{sf} & \text{if } (\exists \pi \in \mathcal{F}(\kappa))(obs(\pi) = \mathbf{s}) \text{ and } (\exists \pi \in \mathcal{F}(\kappa))(obs(\pi) = \mathbf{f}) \\ \mathbf{f} & \text{if } (\forall \pi \in \mathcal{F}(\kappa))(obs(\pi) = \mathbf{f}) \end{cases}$$

3.2. Three Equivalences

The natural notion of observational equivalence is that equal observations are made in all closing configuration contexts. However, it may be possible in some cases to use a weaker equivalence. An **sf** observation may be considered as good as an **s** observation, and a new equivalence arises if these observations are equated. Similarly, an **sf** observation may be as bad as an **f** observation. We may define the following three equivalences.

Definition ($\cong_{1,2,3}$):

(1) (testing or convex or Plotkin or Egli-Milner)

$$e_0 \cong_1 e_1 \quad \text{iff} \quad \text{Obs}(O[e_0]) = \text{Obs}(O[e_1]) \text{ for all observing contexts } O \in \mathbb{O}$$

(2) (must or upper or Smyth)

$$e_0 \cong_2 e_1 \quad \text{iff} \quad \text{Obs}(O[e_0]) = \mathbf{s} \Leftrightarrow \text{Obs}(O[e_1]) = \mathbf{s} \text{ for all observing contexts } O \in \mathbb{O}$$

(3) (may or lower or Hoare)

$$e_0 \cong_3 e_1 \quad \text{iff} \quad \text{Obs}(O[e_0]) = \mathbf{f} \Leftrightarrow \text{Obs}(O[e_1]) = \mathbf{f} \text{ for all observing contexts } O \in \mathbb{O}$$

By construction each of these equivalence relations is a congruence.

Theorem (congruence):

$$e_0 \cong_j e_1 \Rightarrow C[e_0] \cong_j C[e_1] \quad \text{for } j \in \{1, 2, 3\}$$

3.3. Partial Collapse

Note that may-equivalence ($\cong_3$) is determined by computation trees. This is because all **events** are observed after some finite amount of time. This means the definition is independent of whether or not fairness is required. Since fairness sometimes makes proving equivalences more difficult, it is useful that may-equivalence can always be proved ignoring the fairness assumption. The other two equivalences are sensitive to choice of paths admitted as computations. In particular when fairness is required, as in our model, $\cong_2$ is in fact the same as $\cong_1$. In models without the fairness requirement, they are distinct. In either case, $\cong_3$ is distinct from $\cong_1$ and $\cong_2$.

Theorem (partial collapse):

$$(1 = 2) \quad e_0 \cong_2 e_1 \text{ iff } e_0 \cong_1 e_1$$

$$(1 \Rightarrow 3) \quad e_0 \cong_1 e_1 \text{ implies } e_0 \cong_3 e_1$$

$$(3 \not\Rightarrow 1) \quad e_0 \cong_3 e_1 \text{ does not imply } e_0 \cong_1 e_1$$

To demonstrate $(1 = 2)$ we consider a fixed pair of expressions e_0, e_1 and categorize their closing configuration contexts O according to what observations are made by $O[e_0]$ and $O[e_1]$.

$$O \in o : o' \Leftrightarrow \text{Obs}(O[e_0]) = o \wedge \text{Obs}(O[e_1]) = o'$$

for $o, o' \in \{\mathbf{s}, \mathbf{sf}, \mathbf{f}\}$. This partitions the observing configuration contexts of e_0 and e_1 into nine sets labeled $o : o'$ for $o, o' \in \{\mathbf{s}, \mathbf{sf}, \mathbf{f}\}$. We can characterize the various possibilities for equivalence in terms of which sets must be empty as follows.

Lemma (sets):

- $e_0 \cong_1 e_1$ iff at most the sets labeled $\mathbf{s} : \mathbf{s}$, $\mathbf{sf} : \mathbf{sf}$, and $\mathbf{f} : \mathbf{f}$ are non-empty.
- $e_0 \cong_2 e_1$ iff the sets labeled $\mathbf{s} : \mathbf{sf}$, $\mathbf{sf} : \mathbf{s}$, $\mathbf{s} : \mathbf{f}$, $\mathbf{f} : \mathbf{s}$ are all empty.
- $e_0 \cong_3 e_1$ iff the sets labeled $\mathbf{s} : \mathbf{f}$, $\mathbf{f} : \mathbf{s}$, $\mathbf{sf} : \mathbf{f}$, $\mathbf{f} : \mathbf{sf}$ are all empty.

The key to collapsing $\cong_2$ into $\cong_1$ is the observation that if $\text{Obs}(O[e_0]) = \mathbf{f}$ and $\text{Obs}(O[e_1]) = \mathbf{sf}$ it is always possible to construct a O^* such that $\text{Obs}(O^*[e_0]) = \mathbf{s}$, and $\text{Obs}(O^*[e_1]) = \mathbf{sf}$.

Lemma (f.sf): For some e_0, e_1 , if the set labeled $\mathbf{f} : \mathbf{sf}$ is non-empty then the set labeled $\mathbf{s} : \mathbf{sf}$ is non-empty. Symmetrically, if the set labeled $\mathbf{sf} : \mathbf{f}$ is non-empty then the set labeled $\mathbf{sf} : \mathbf{s}$ is non-empty.

Proof (f.sf): Let $O \in \mathbf{f} : \mathbf{sf}$. Form O' by replacing all occurrences of `event()` in O by `send(a, nil)` for some fresh variable a . Let O^* be obtained by adding to O' a message $\langle a \Leftarrow \mathbf{t} \rangle$ and an actor a where a has the following behavior: If a receives the message $\mathbf{t}$, it executes `event()` and becomes a sink, and if a receives the message `nil`, it just becomes a sink. Recall that a sink is an actor that ignores its message and becomes a sink. We claim $O^* \in \mathbf{s} : \mathbf{sf}$. If $O[e_0]$ never executes `event()`, then in any fair complete computation, the $\mathbf{t}$ message will be received by a , so $O^*[e_0]$ will always execute `event()`. If $O[e_1]$ executes `event()` in some computation, then in the corresponding computations for $O^*[e_1]$, sometimes `nil` will be received by a before $\mathbf{t}$ is received and sometimes it won't, hence $O^*[e_1]$ will execute `event()` in some computations, but not in all. $\square$

Proof (partial collapse):

$1 = 2$ Assume $e_0 \cong_2 e_1$. Then by (**sets**) the sets labeled $\mathbf{s} : \mathbf{sf}$, $\mathbf{sf} : \mathbf{s}$, $\mathbf{s} : \mathbf{f}$, $\mathbf{f} : \mathbf{s}$ are all empty. By (**f.sf**) $\mathbf{f} : \mathbf{sf}$ and $\mathbf{sf} : \mathbf{f}$ must also be empty. Hence by (**sets**), $e_0 \cong_1 e_1$. $\square_1 = 2$

$1 \Rightarrow 3$ By (**sets**) $\square_1 \Rightarrow 3$

$3 \not\Rightarrow 1$ We construct expressions e_0, e_1 such that $e_0 \cong_3 e_1$, but $\neg(e_0 \cong_2 e_1)$. Let e_0 create an actor that sends a message (say `nil`) to an external actor a and becomes a sink, and let e_1 create an actor that may or may not send a message `nil` to a depending on a coin flip (there are numerous methods of constructing coin flipping actors), and also then becomes a sink. Let O be an observing configuration context that with an actor a that executes `event` just if `nil` is received. Then $Obs(O[e_0]) = \mathbf{s}$ but $Obs(O[e_1]) = \mathbf{sf}$, so $\neg(e_0 \cong_2 e_1)$. To show that $e_0 \cong_3 e_1$, show for arbitrary O that some path in the computation of $O[e_0]$ contains an event iff some path in the computation of $O[e_1]$ contains an event. This is easy, because when e_1 's coin flip indicates `nil` is sent, the computation proceeds identically to e_0 's computation. $\square_3 \not\Rightarrow 1$

$\square$

Hereafter, $\cong$ (observational equivalence) will be used as shorthand for either $\cong_1$ or $\cong_2$.

The fairness requirement is critical in the proof of $(1 = 2)$. For example in CCS, where fairness is not assumed, no such collapse of $\cong_2$ to $\cong_1$ occurs. So, although fairness complicates some aspects of the theory, it simplifies others. If we omitted the fairness requirement we could make more $\cong$ -distinctions between actors. For example, let a_0 be a sink. Let a_1 be an actor that also ignores its messages and becomes the same behavior, but it continues executing an infinite loop. The infinite looping actor could prevent the rest of the configuration's computation from progressing. In the presence of fairness no such starvation can occur, so the two are equivalent. Thus fairness allows modular reasoning about liveness properties: one can reason about the behavior of individual actors (and configurations) without worrying about whether composition with another would cause that actor to starve out.

4. Laws of Expression Equivalence

With a notion of equivalence on actor expressions defined, a library of useful equivalences can be established. The first part of this section contains a collection of purely functional laws that continue to hold in the actor setting. The second part contains a collection of laws for manipulating expressions that involve actor primitives. These laws are proved in §7.

4.1. Functional Laws

Since our reduction rules preserve the evaluation semantics of the embedded functional language, many of the equational laws for this language (cf. [29]) continue to hold in the full actor language. A first simple observation is two communicable values are observationally equivalent iff they are the same value expression.

Lemma (cv):

$$cv_0 \cong cv_1 \Leftrightarrow cv_0 = cv_1$$

Proof : The if direction is trivial. The only-if direction, is proved by exhibiting an observing context that distinguishes expressions that are not equal. Clearly both must be atoms, or variables, or pairs, otherwise they can be distinguished using **eq** and **ispr**. For example,

$$O = \langle\langle [\text{if}(\text{eq}(cv_0, \bullet), \text{event}, \text{nil})]_a \mid \rangle\rangle$$

distinguishes the atom cv_0 from any non-atom (and any other atom). Similarly,

$$O = \langle\langle [\text{let}\{x := 0\}\text{let}\{y := 1\}\text{if}(\text{eq}(x, \bullet), \text{event}, \text{nil})]_a \mid \rangle\rangle$$

distinguishes the variables x, y . Similarly, if both are pairs, we can construct contexts to distinguish differences in the components. $\square$

Beta-v conversion and other laws of the lambda-c calculus [21], and the laws for conditional and pairing continue to hold in the actor setting.

Theorem (functional laws):

- (beta-v) $\text{app}(\lambda x.e, v) \cong e\{x := v\}$
- (ift) $\text{if}(v, e_1, e_2) \cong e_1 \quad \text{if } v \in (\text{At} - \{\text{nil}\}) \cup \mathbb{L} \cup \text{pr}(\mathbb{V}, \mathbb{V})$
- (ifn) $\text{if}(\text{nil}, e_1, e_2) \cong e_2$
- (ifelim) $\text{if}(v, e, e) \cong e$
- (iflam) $\lambda x.\text{if}(v, e_1, e_2) \cong \text{if}(v, \lambda x.e_1, \lambda x.e_2) \quad x \notin \text{FV}(v)$
- (isprt) $\text{ispr}(\text{pr}(v_0, v_1)) \cong \mathbf{t}$,
- (isprn) $\text{ispr}(v) \cong \text{nil} \quad v \in \text{At} \cup \mathbb{L}$
- (fst) $1^{\text{st}}(\text{pr}(v_0, v_1)) \cong v_0$
- (snd) $2^{\text{nd}}(\text{pr}(v_0, v_1)) \cong v_1$

Each of these laws (except for (**iflam**)) is a consequence of the following operational law.

Theorem (red-exp):

$$e_0 \xrightarrow{\lambda} e_1 \Rightarrow e_0 \cong e_1$$

The theorem (**rcx**) is a special case of a theorem proved in [28].

Theorem (rcx): If R is a reduction context, then

$$(\text{letx}) \quad \text{let}\{x := e\}R[x] \cong R[e]$$

$$(\text{let.dist}) \quad R[\text{let}\{x := e\}e_0] \cong \text{let}\{x := e\}R[e_0]$$

$$(\text{if.dist}) \quad R[\text{if}(e, e_1, e_2)] \cong \text{if}(e, R[e_1], R[e_2])$$

In fact these laws can be derived from (**beta-v**), the **if** laws and the following special instances.

$$(\text{app}) \quad e_0(e_1) \cong (\lambda f.f(e_1))(e_0)$$

$$(\text{cmps}) \quad f(g(e)) \cong (\lambda x.f(g(x)))(e)$$

$$(\text{id}) \quad \text{app}(\lambda x.x, e) \cong e$$

Some useful corollaries of (**rcx**) are the following.

Corollary (uni-rcx):

$$(\text{let.arg}) \quad v(\text{let}\{x := e_0\}e_1) \cong \text{let}\{x := e_0\}v(e_1)$$

$$(\text{if.if}) \quad \text{if}(\text{if}(e_0, e_1, e_2), e_a, e_b) \cong \text{if}(e_0, \text{if}(e_1, e_a, e_b), \text{if}(e_2, e_a, e_b))$$

The above laws are really about equivalence of reduction contexts. They are instances of the following operational law.

Theorem (red-rcx):

$$R_0[x] \xrightarrow{\lambda} e' \wedge R_1[x] \xrightarrow{\lambda} e' \Rightarrow R_0[e] \cong R_1[e]$$

where x is a fresh variable.

We also note that any expressions that hang (reduce in a finite number of lambda steps to a stuck state) or have infinite lambda computations are observationally equivalent. If the reductions could involve non-lambda steps the result clearly fails since they could have different effects such as message sends that other actors in the configuration could observe. We let **stuck** $\in$ *Hang* be a prototypical stuck expression, for example **app**(0, 0), and let **bot** $\in$ *Infin* be a prototypical expression with infinite computation, for example **app**($\lambda x.\text{app}(x, x), \lambda x.\text{app}(x, x)$).

To make these ideas more precise we define *Hang* and *Infin* as follows.

Definition (*Hang*): Let *Hang* be the set of non-value expressions such that (every closed instance) lambda reduces (i.e. $\xrightarrow{\lambda}$ in possibly 0 steps) to a stuck state – an expression e' that decomposes as $R[r]$ where r is a functional redex (i.e any non-actor redex) that does not reduce.

Definition (*Infin*): Let *Infin* be the set of (non-value) expressions e such that (every closed instance) has an infinite lambda reduction sequence. Thus $e \in$ *Infin* just if we can find e_j for $j \in \mathbb{N}$ such that $e_0 = e$ and $e_j \xrightarrow{\lambda} e_{j+1}$.

Theorem (hang-infin): If $e_0, e_1 \in \text{Hang} \cup \text{Infin}$, then $e_0 \cong e_1$.

4.2. Basic Laws for Actor Primitives

Now we consider the equational properties of the actor primitives, **send**, **become**, **newadr**, and **initbeh**. As is the case for a language with state operations, $\text{seq}(e, e) \cong e$ fails to hold because the computation e could have effects such as message sends. A stronger analogy exists between the actor primitives and the reference primitives $\{\text{mk}, \text{get}, \text{set}\}$: **newadr** is an allocation primitive analogous to **mk**; **initbeh** and **become** update or alter state; while the effect of **send** depends on the state in a way analogous to **get**. There are limits to this analogy, for example **send** does not return anything of interest. In fact **send**, **become**, and **initbeh** all return **nil** as values. Thus

$$(\text{triv}) \quad \vartheta(\bar{x}) \cong \text{seq}(\vartheta(\bar{x}), \text{nil}) \quad \text{for } \vartheta \in \{\text{send}, \text{become}, \text{initbeh}\}$$

That **newadr** is an allocation primitive analogous to **mk** manifests itself in its properties. However, because we have not allowed clones to initialize newly spawned actors, one characteristic property of allocation fails to hold. Namely,

$$(\text{non-delay}) \quad \text{let}\{y := e_0\}\text{let}\{x := \text{newadr}()\}e_1 \not\cong \text{let}\{x := \text{newadr}()\}\text{let}\{y := e_0\}e_1$$

where x is not free in e_0 .

Once allocated, an actor behavior is updated by either **become** or **initbeh**. In analogy with **set** both **become** and **initbeh** satisfy certain, slightly different, cancellation laws:

$$(\text{can-b}) \quad \text{seq}(\text{become}(v_0), \text{become}(v_1)) \cong \text{seq}(\text{become}(v_0), \text{nil}) \cong \text{become}(v_0)$$

$$(\text{can-i}) \quad \begin{aligned} \text{seq}(\text{initbeh}(v, v_0), \text{initbeh}(v, v_1)) &\cong \text{seq}(\text{initbeh}(v, v_0), \text{stuck}) \\ &\cong \text{seq}(\text{initbeh}(v, v_0), \text{bot}) \end{aligned}$$

Note the difference between these two principles. In the case of **become** the second call is equivalent to **nil**, while in the case of **initbeh** it is **stuck** (which is equivalent to diverging).

4.2.1. Commuting Operations

How the effects of the actor primitives interact with one another is of paramount importance. We have seen some aspects of this interaction above. We now study the interactions more systematically.

Definition (commutes): We say two operations ϑ_0 and ϑ_1 commute if

$$\text{let}\{x_0 := \vartheta_0(\bar{y})\}\text{let}\{x_1 := \vartheta_1(\bar{z})\}e \cong \text{let}\{x_1 := \vartheta_1(\bar{z})\}\text{let}\{x_0 := \vartheta_0(\bar{y})\}e$$

for all $e \in \mathbb{E}$, $x_0 \notin \bar{z}$, $x_1 \notin \bar{y}$ and x_0 distinct from x_1 . Similarly we say two expressions e_0 and e_1 commute iff

$$\text{let}\{x_0 := e_0\}\text{let}\{x_1 := e_1\}e \cong \text{let}\{x_1 := e_1\}\text{let}\{x_0 := e_0\}e$$

if the obvious hygiene conditions hold.

newadr commutes with every operation except **become**. For example the expressions

$$e_0 = \text{let}\{y := \text{newadr}()\}\text{let}\{z := \text{become}(b)\}\text{initbeh}(y, b')$$

$$e_1 = \text{let}\{z := \text{become}(b)\}\text{let}\{y := \text{newadr}()\}\text{initbeh}(y, b')$$

are not equivalent, since the first will always fail to execute the initialization and the second will always succeed. A distinguishing context is

$$\langle\langle (\lambda x.\mathbf{event}())_{a_0}, [\mathbf{seq}(\bullet, \mathbf{send}(a_0, 0))]_a \mid \emptyset \rangle\rangle$$

If we allowed clones to initialize, then **newadr** would also commute with **become**. On the other hand, by **(can-b)** and **(can-1)** neither **become** nor **initbeh** commute with themselves, since this amounts to equivalence of two becomes (or initializations) with different behaviors. The remaining operation **send**, like **newadr**, does commute with itself:

$$(\text{com-ss}) \quad \mathbf{seq}(\mathbf{send}(v_0, v_1), \mathbf{send}(v_2, v_3)) \cong \mathbf{seq}(\mathbf{send}(v_2, v_3), \mathbf{send}(v_0, v_1))$$

send also commutes with **become**

$$(\text{com-sb}) \quad \mathbf{seq}(\mathbf{send}(a_0, v_0), \mathbf{become}(v_1)) \cong \mathbf{seq}(\mathbf{become}(v_1), \mathbf{send}(a_0, v_0))$$

The question of whether or not two distinct operations commute is simplified by the observation, captured in **(partial)**, that a computation may have observable effects even if it diverges. This is in contrast to the sequential case, where an effect of a subcomputation is only observable if the computation completes.

Lemma (partial): If ϑ is not a total operation, then ϑ does not commute with **send**, **become** or **initbeh**.

Proof (partial): If $\vartheta(\bar{y})$ diverges, then

$$\mathbf{let}\{x := \vartheta(\bar{y})\}\mathbf{let}\{x_1 := \mathbf{send}(a, v)\}e$$

will not execute the **send**, whereas

$$\mathbf{let}\{x_1 := \mathbf{send}(a, v)\}\mathbf{let}\{x := \vartheta(\bar{y})\}e$$

will execute the **send**. Consequently the two expressions are easily distinguished. Similarly with the two operations **become** and **initbeh**. $\square$

Since **initbeh** is partial, it does not commute with either **send** or **become**. For example

$$e_0 = \mathbf{seq}(\mathbf{initbeh}(a_0, b_0), \mathbf{send}(a_1, 0))$$

$$e_1 = \mathbf{seq}(\mathbf{send}(a_1, 0), \mathbf{initbeh}(a_0, b_0))$$

are distinguished by

$$O_0 = \langle\langle (\lambda x.\mathbf{event}())_{a_1}, (?_{a_1})_{a_0}, [\bullet]_a \mid \mu \rangle\rangle$$

for $a \neq a_1$, or by

$$O_1 = \langle\langle (\lambda x.\mathbf{event}())_{a_1}, (b)_{a_0}, [\bullet]_a \mid \mu \rangle\rangle$$

Also,

$$\begin{aligned} e_0 &= \mathbf{seq}(\mathbf{initbeh}(a_0, b_0), \mathbf{become}(\lambda x. \mathbf{send}(a_1, 0))) \\ e_1 &= \mathbf{seq}(\mathbf{become}(\lambda x. \mathbf{send}(a_1, 0)), \mathbf{initbeh}(a_0, b_0)) \end{aligned}$$

are distinguished by O_0, O_1 if μ contains $\langle a \Leftarrow 0 \rangle$.

(partial) emphasizes that the valid equations for actor primitives is sensitive to the details of when we check for ill-formed redexes. For example if we restricted the **send** redex to avoid ill-formed messages (**com-ss, com-sb**) would no longer hold.

We summarize these results in the following:

Lemma (commutes):

	n	s	i	b
n	+	+	+	-
s	+	+	-	+
i	+	-	-	-
b	-	+	-	-

- (n) **newadr** commutes with **send**, **newadr**, and **initbeh**, but not with **become**.
- (s) **send** commutes with **send**, **become**, and **newadr**, but not with **initbeh**.
- (i) **initbeh** commutes with **newadr**, but not with **send**, **become**, and **initbeh**.
- (b) **become** commutes with **send**, but not with **initbeh**, **newadr**, or **become**.

Note that the remaining operations in $\mathbb{F}$ (i.e. the arithmetic operations, branching **br**, and the pairing **ispr, pr, 1st, 2nd**) are all context insensitive, and thus those that are total commute with all other operations. In the case **if** this is perhaps worth pointing out:

Lemma (commutes-if): If ϑ commutes with e_0 and e_1 , then it also commutes with **if**(z, e_0, e_1)

Proof: This follows from (**if-lam, if-dist**). $\square$

Using these basic principles we can prove more complex properties, the following theorem being the most obvious.

Theorem (commutes): Suppose that e_0 and e_1 are built up from $\mathbb{V}$ using only the operations, **if** and **let**. Furthermore suppose every operation occurring in e_0 commutes with every operation occurring in e_1 . Then

$$\mathbf{let}\{x_0 := e_0\}\mathbf{let}\{x_1 := e_1\}e \cong \mathbf{let}\{x_1 := e_1\}\mathbf{let}\{x_0 := e_0\}e$$

provided x_j is not free in e_{1-j} for $j < 2$,

Proof (commutes): The proof is by induction on the complexity of e_0 . We sketch the induction step. We may assume, without loss of generality, that e_0 decomposes into $R[\vartheta(\bar{y})]$ with R being non-trivial. Then

$$\begin{aligned}
& \mathbf{let}\{x_0 := e_0\}\mathbf{let}\{x_1 := e_1\}e \cong \\
& \cong \mathbf{let}\{x_0 := R[\vartheta(\bar{y})]\}\mathbf{let}\{x_1 := e_1\}e \\
& \quad \text{by hypothesis} \\
& \cong \mathbf{let}\{x_0 := \mathbf{let}\{z := \vartheta(\bar{y})\}R[z]\}\mathbf{let}\{x_1 := e_1\}e \\
& \quad \text{by (cong) and (letx)} \\
& \cong \mathbf{let}\{z := \vartheta(\bar{y})\}\mathbf{let}\{x_0 := R[z]\}\mathbf{let}\{x_1 := e_1\}e \\
& \quad \text{by (let.dis)} \\
& \cong \mathbf{let}\{z := \vartheta(\bar{y})\}\mathbf{let}\{x_1 := e_1\}\mathbf{let}\{x_0 := R[z]\}e \\
& \quad \text{by the induction hypothesis and (cong)} \\
& \cong \mathbf{let}\{x_1 := e_1\}\mathbf{let}\{z := \vartheta(\bar{y})\}\mathbf{let}\{x_0 := R[z]\}e \\
& \quad \text{by the induction hypothesis and (cong)} \\
& \cong \mathbf{let}\{x_1 := e_1\}\mathbf{let}\{x_0 := \mathbf{let}\{z := \vartheta(\bar{y})\}R[z]\}e \\
& \quad \text{by (let.dis)} \\
& \cong \mathbf{let}\{x_1 := e_1\}\mathbf{let}\{x_0 := e_0\}e \\
& \quad \text{by (cong) and (letx)}
\end{aligned}$$

□

Note that the theorem fails in the case when the expressions contain **app** and λ due to the possibility of divergence.

To prove other properties involving the actor primitives we need to consider the whole context in which the effect occurs, which is at least one other actor and in general is an actor configuration.

5. Manipulation of Actor Configurations

In this section we establish some fundamental results concerning actor configurations. We show how configurations may be composed or decomposed into larger or smaller configurations. We define two notions of equivalence between configurations: observational and interaction equivalence. We also develop some basic tools for establishing these equivalences.

For simplicity we have treated the collection of messages in a configuration as a multi-set. In this section we make the identity of different messages with the same target and contents explicit, by assuming that each message in the computation is uniquely identified as being there initially, or as being generated by a particular send or input transition. We write m rather than $\langle a \Leftarrow cv \rangle$ and assume that m contains this identifying information.

5.1. Composition of Actor Configurations

Actor configurations can be composed to form new actor configurations. Furthermore, the computations of the composed configuration can be obtained by merging computations of the component configurations. This composition operation is commutative, associative, and has the empty configuration as unit. This is made precise by the following definitions and lemmas.

Definition (Composable): Two configurations $\kappa_i = \langle\langle \alpha_i \mid \mu_i \rangle\rangle_{\chi_i}^{\rho_i}$, $i < 2$ are composable if $\text{Dom}(\alpha_0) \cap \text{Dom}(\alpha_1) = \emptyset$, $\chi_0 \cap \text{Dom}(\alpha_1) \subseteq \rho_1$, and $\chi_1 \cap \text{Dom}(\alpha_0) \subseteq \rho_0$.

Definition (Composition, decomposition): The composition $\kappa_0 \parallel \kappa_1$ of composable configurations $\kappa_i = \langle\langle \alpha_i \mid \mu_i \rangle\rangle_{\chi_i}^{\rho_i}$, $i < 2$ is defined by

$$\kappa_0 \parallel \kappa_1 = \langle\langle \alpha_0 \cup \alpha_1 \mid \mu_0 \cup \mu_1 \rangle\rangle_{(\chi_0 \cup \chi_1) - (\rho_0 \cup \rho_1)}^{\rho_0 \cup \rho_1}$$

(κ_0, κ_1) is a decomposition of κ if κ_i , $i < 2$ are composable configurations, and $\kappa = \kappa_0 \parallel \kappa_1$.

Lemma (AC): Let $\kappa_i = \langle\langle \alpha_i \mid \mu_i \rangle\rangle_{\chi_i}^{\rho_i}$, $i < 3$ be pairwise composable configurations. And let $\kappa_\emptyset = \langle\langle \emptyset \mid \emptyset \rangle\rangle$ be the empty configuration. Then

$$\kappa_0 \parallel \kappa_1 = \kappa_1 \parallel \kappa_0$$

$$\kappa_0 \parallel \kappa_\emptyset = \kappa_0$$

$$(\kappa_0 \parallel \kappa_1) \parallel \kappa_2 = \kappa_0 \parallel (\kappa_1 \parallel \kappa_2)$$

Proof: Using the AC properties of set union, the only thing to check is the equality of external actors for the two associations. It is easy to see that

$$\begin{aligned} & (((\chi_0 \cup \chi_1) - (\rho_0 \cup \rho_1)) \cup \chi_2) - (\rho_0 \cup \rho_1 \cup \rho_2) \\ &= (\chi_0 \cup \chi_1 \cup \chi_2) - (\rho_0 \cup \rho_1 \cup \rho_2) \\ &= (\chi_0 \cup ((\chi_1 \cup \chi_2) - (\rho_1 \cup \rho_2))) - (\rho_0 \cup \rho_1 \cup \rho_2) \end{aligned}$$

□

5.1.1. Projection and Merging of Sequences

We define projection and merging operations to relate computations of composite configurations with those of their components. In the remainder of this subsection we relax our notation convention slightly. We say that π is *computation for* κ if π is a computation sequence or path with first element κ . Let $\pi = [\kappa_i \xrightarrow{l_i} \kappa_{i+1} \mid i < \bowtie]$ be a computation for κ and let κ be decomposed as $\kappa^0 \parallel \kappa^1$. Then we can decompose each $\kappa_i = \kappa_i^0 \parallel \kappa_i^1$ keeping newly created actors/receptionists and messages in the component that generated them. Given such a decomposition, we can choose points to move messages sent across the internal boundary from the sender to receiver message pool such that receives are always from the local pool. We call such a choice of points an *i/o-marking* relative to the decomposition (κ^0, κ^1) . Given a decomposition and i/o-marking we can project π to obtain computations

for κ^j for $j < 2$. This projection preserves fairness of paths if the i/o-marking moves all messages sent from one component to the other, not just those that are eventually received. Such i/o-markings are called *fair*.

Definition (sequence/path decomposition): Let $\pi = [\kappa_i \xrightarrow{l_i} \kappa_{i+1} \mid i < \infty]$ be a computation sequence or path for κ . A decomposition (κ^0, κ^1) of κ naturally induces a decomposition of π – a sequence of pairs $[(\kappa_i^0, \kappa_i^1) \mid i < \infty]$ such that (κ_i^0, κ_i^1) is a decomposition of κ_i for $i < \infty$. The decomposition of π is defined by induction on i as follows. For $i = 0$ we take the given decomposition of κ . The decomposition of κ_{i+1} is obtained from the decomposition (κ_i^0, κ_i^1) of κ_i in one of two ways according to the form of l_i .

- (1) l_i is a **exec** transition with focus $a \in \text{Dom}(\alpha_i^j)$, a **rcv** transition with focus $a \in \text{Dom}(\alpha_i^j)$ of $m \in \mu_i^j$, an **input** to a in j , or an **output** from j . In this case, κ_{i+1}^j is the unique configuration such that $\kappa_i^j \xrightarrow{l_i} \kappa_{i+1}^j$, and $\kappa_{i+1}^{1-j} = \kappa_i^{1-j}$.
- (2) l_i is a **rcv** transition with focus $a \in \text{Dom}(\alpha_i^j)$ of $m \in \mu_i^{1-j}$. In this case, κ_{i+1}^j is the unique configuration such that $\kappa_i^j \xrightarrow{L_i} \kappa_{i+1}^j$, where $L_i = [\text{in} : m, l_i]$, and κ_{i+1}^{j-1} is the unique configuration such that $\kappa_i^{j-1} \xrightarrow{\text{out} : m} \kappa_{i+1}^{j-1}$.

Definition (i/o-marking): Let $\pi = [\kappa_i \xrightarrow{l_i} \kappa_{i+1} \mid i < \infty]$ be a computation sequence or path for κ , and let (κ^0, κ^1) be a decomposition of κ . An i/o-marking of π relative to (κ^0, κ^1) is a set, IO , of pairs (k, m) such that the conditions (1-3) below hold.

- (1) If $(k, m) \in IO$, then $k < \infty$, and m is generated by l_i for some $i \leq k$ such that the focus of l_i and the target of the message are in different components of the path decomposition.
- (2) If $(k, m) \in IO$ and $(k', m) \in IO$, then $k = k'$.
- (3) If l_i is a case 2 **rcv** of m generated in step $i' < i$, then there is some k with $i' \leq k < i$ such that $(k, m) \in IO$.

IO is said to be *fair* if the converse to (1) also holds, i.e. if condition (4) below also holds.

- (4) If m is generated by l_i for some $i < \infty$ such that the focus of l_i and the target of the message are in different components of the path decomposition, then $(k, m) \in IO$, for some k such that $i \leq k$.

An element (k, m) of IO is called a mark at point k in π . There may be more than one such mark at any given point k . A *disambiguation* of IO is a family $\ll$ of total orderings $\ll_k$ on the marks at point k in IO . We will also call the pair $IO, \ll$ an i/o-marking.

Definition (i/o-restriction): Let $\pi = [\kappa_i \xrightarrow{l_i} \kappa_{i+1} \mid i < \infty]$ be a computation sequence or path for κ , and let $IO, \ll$ be an i/o-marking of π relative to decomposition (κ^0, κ^1) of κ . Let ν be an initial segment of π . $IO', \ll'$ is an i/o-restriction (or, just restriction) of $IO, \ll$ to ν if $IO' \subseteq IO$, $\ll'$ is the restriction of $\ll$ to IO' , and if $(k, m) \in IO'$, then

- $k \leq \text{Len}(\nu)$
- if $(k', m') \in IO$ and $k' < k$ or $k' = k$ and $m' \ll_k m$, then $(k', m') \in IO'$

Definition (projection): If π is a computation for κ and $IO, \ll$ is an i/o-marking for π relative to the decomposition $\kappa = \kappa^0 \parallel \kappa^1$, then the projection $\pi]_{j}^{IO, \ll}$ onto component

j for $j < 2$ is defined as follows. Let $\pi = [\kappa_i \xrightarrow{l_i} \kappa_{i+1} \mid i < \infty]$ with decomposition $[(\kappa_i^0, \kappa_i^1) \mid i < \infty]$ induced by $\kappa = \kappa^0 \parallel \kappa^1$. Then

$$\pi \downarrow_j^{IO, \ll} = [\kappa_i^j \xrightarrow{L_i^j * M_i^j} \kappa_{i+1}^j \mid i < \infty]$$

where

- L_i^j is $[l_i]$ if l_i located in j , and $[\]$ if l_i located in $1 - j$, and
- M_i^j is a sequence of i/o transition labels corresponding to the marks at point i in IO . Let (i, m_k) be the k -th element in the ordering $\ll_i$. Then the k element of M_i^j is given by
 - $\langle \text{in} : m_k \rangle$ if the target of m_k is in j
 - $\langle \text{out} : m_k \rangle$ if the target of m_k is in $1 - j$

Lemma (project-computation): If π is a computation for κ , and $IO, \ll$ is an i/o-marking for π relative to the decomposition $\kappa = \kappa^0 \parallel \kappa^1$, then the projection $\pi \downarrow_j^{IO, \ll}$ is a computation for κ^j , for $j < 2$. Furthermore if $\pi \in \mathcal{F}(\kappa)$ and IO is a fair marking, then $\kappa^j \in \mathcal{F}(\kappa^j)$, for $j < 2$.

Proof: Let

$$\pi \downarrow_j^{IO, \ll} = [\kappa_i^j \xrightarrow{L_i^j * M_i^j} \kappa_{i+1}^j \mid i < \infty]$$

be the projection of π onto j relative to the decomposition $\kappa = \kappa^0 \parallel \kappa^1$ and the marking $IO, \ll$. Then by the construction of the decomposition and the definitions of M_i^j, L_i^j $\kappa_i^j \xrightarrow{L_i^j * M_i^j} \kappa_{i+1}^j$ is a (multi-step) labelled transition for $i < \infty$. Thus the projections are (possibly infinite) labelled computation sequences.

Assume $\pi \in \mathcal{F}(\kappa)$ and IO is a fair marking. Suppose l is enabled in κ_i^j , i.e. $\kappa_i^j \xrightarrow{l} \kappa'$ for some κ' . If l is an internal transition, or a **out** transition to an actor external to the composite system, then l is enabled in κ_i , and by fairness of π , either l is l_k for some $k \geq i$, or l is a **rcv** by a and at some $k \geq i$, **rcv**'s by a are permanently disabled in π . In the first case l_k will be a transition in $\pi \downarrow_j^{IO, \ll}$, and in the second case **rcv**'s by a are permanently disabled in $\pi \downarrow_j^{IO, \ll}$ at the projection of k . If $l = \langle \text{out} : m \rangle$ with the target of m a receptionist of $1 - j$, then $(k, m) \in IO$ for some k by fairness of IO , and by the definition of projection l will occur in the projection of l_k . $\square$

Lemma (project-restrict): Let π be a computation for κ , and let $IO, \ll$ be an i/o-marking for π relative to the decomposition $\kappa = \kappa^0 \parallel \kappa^1$.

- If ν is an initial segment of π , then any restriction, $IO', \ll'$, of $IO, \ll$ to ν is an i/o-marking for ν relative to the decomposition $\kappa = \kappa^0 \parallel \kappa^1$, and $\nu \downarrow_j^{IO', \ll'}$ is an initial segment of $\pi \downarrow_j^{IO, \ll}$ for $j < 2$.
- If ν_0 is an initial segment of $\pi \downarrow_0^{IO, \ll}$ there there is an initial segment, ν , of π and a the restriction $IO', \ll'$ of $IO, \ll$ to ν such that $\nu_0 = \nu \downarrow_0^{IO', \ll'}$.

Definition (complimentary): We say l_0, l_1 are complimentary if $l_j = \langle \text{in} : m \rangle$ and $l_{1-j} = \langle \text{out} : m \rangle$ for some $j < 2$.

Definition (merged): The relation $sMerged(\nu^0, \nu^1, \nu)$ on computation sequences is defined by induction on the combined lengths of ν^0, ν^1 . In this definition we use the notation $\nu \xrightarrow{l} \kappa$ to stand for the sequence $[\kappa_0 \xrightarrow{l_0} \kappa_1 \xrightarrow{l_1} \dots \xrightarrow{l_n} \kappa_n \xrightarrow{l} \kappa]$ where $\nu = [\kappa_i \xrightarrow{l_i} \kappa_{i+1} \mid i < n]$.

- (0) $sMerged([\kappa_0], [\kappa_1], [\kappa_0 \parallel \kappa_1])$ if κ_0, κ_1 are composable.
- (1) $sMerged(\nu_0, \nu_1, \nu) \Rightarrow sMerged(\nu_0 \xrightarrow{l} \kappa_0, \nu_1, \nu \xrightarrow{l} \kappa_0 \parallel \kappa_1)$
if $\text{Last}(\nu_1) = \kappa_1$, κ_0, κ_1 are composable, and l is not output to receptionist of κ_1
- (2) $sMerged(\nu_0, \nu_1, \nu) \Rightarrow sMerged(\nu_0, \nu_1 \xrightarrow{l} \kappa_1, \nu \xrightarrow{l} \kappa_0 \parallel \kappa_1)$
if $\text{Last}(\nu_0) = \kappa_0$, κ_0, κ_1 are composable, and l is not output to receptionist of κ_0
- (3) $sMerged(\nu_0, \nu_1, \nu) \Rightarrow sMerged(\nu_0, \xrightarrow{l_0} \kappa_0, \nu_1 \xrightarrow{l_1} \kappa_1, \nu)$
if l_0, l_1 are complimentary

Lemma (merged-cseq): If $\nu_j \in \mathcal{T}(\kappa_j)$ for $j < 2$ and $sMerged(\nu_0, \nu_1, \nu)$, then

- (1) $\nu \in \mathcal{T}(\kappa_0 \parallel \kappa_1)$, and
- (2) there exists a unique i/o-marking, IO , and disambiguation $\ll$, such that $\nu_j = \nu \downarrow_j^{IO, \ll}$ for $j < 2$.

Proof: (1) follows by induction on the definition of $sMerged$. It is easy to check that in each case ν is a well-formed computation sequence.

An i/o-marking, IO , and disambiguation $\ll$ is constructed by induction on the combined lengths of ν_0, ν_1 . If $\nu_j = [\kappa_j]$, then $\nu = [\kappa_0 \parallel \kappa_1]$. In this case, $IO = \emptyset = \ll$, uniquely. Otherwise there are two cases to consider according to whether or not the final transition of ν_j is an output to a receptionist of ν_{1-j} for some $j < 2$.

For the first case, suppose $\nu_0 = \nu'_0 \xrightarrow{l_0} \kappa'_0$ and l_0 is an output to a receptionist of ν_1 . Then by definition of $sMerged$, we must have $\nu_1 = \nu'_1 \xrightarrow{l_1} \kappa'_1$, l_0, l_1 are complimentary, and $sMerged(\nu'_0, \nu'_1, \nu)$. By induction let $IO', \ll'$ be the unique disambiguated i/o-marking such that $\nu'_0 = \nu \downarrow_0^{IO', \ll'}$, and $\nu'_1 = \nu \downarrow_1^{IO', \ll'}$. In this case, $IO = IO' \cup \{(k, m)\}$ where $k = \text{Len}(\nu)$ and m is the message transmitted by l_0, l_1 . $<$ is $<'$ with $<_k$ extended to make m last in the ordering of marks at point k . The case when the final transition of ν_1 is an output to ν_0 is analogous.

For the second case, by definition of $sMerged$, we must have $\nu = \nu' \xrightarrow{l} \kappa'$, where $\nu_j = \nu'_j \xrightarrow{l} \kappa'_j$, $\nu'_{1-j} = \nu_{1-j}$, and $\kappa' = \kappa'_j \parallel \text{Last}(\nu_{1-j})$, for some $j < 2$, and $sMerged(\nu'_0, \nu'_1, \nu')$. By induction let $IO', \ll'$ be the unique disambiguated i/o-marking such that $\nu'_0 = \nu' \downarrow_0^{IO', \ll'}$, and $\nu'_1 = \nu' \downarrow_1^{IO', \ll'}$. In this case, take $IO = IO'$ and $\ll = \ll'$.

□

Lemma (project-merged): If κ_0 and κ_1 are composable, $\nu \in \mathcal{T}(\kappa_0 \parallel \kappa_1)$, and IO is an i/o-marking of ν relative to the decomposition (κ_0, κ_1) , then $sMerged(\nu]_0^{IO, \ll}, \nu]_1^{IO, \ll}, \nu)$ for any disambiguation $\ll$ of IO .

Proof : The proof is by induction on the length of ν plus the size of IO . If $\nu = [\kappa_0 \parallel \kappa_1]$ then we are done by clause (0) of the definition of $sMerged$. Suppose $\nu = \nu' \xrightarrow{l_n} \kappa_{n+1}$ has $n+1$ transitions. If $(n, m) \in IO$ then let $IO' = IO - \{(n, m)\}$, and let $\ll'$ be the restriction of $\ll$ to IO' . Suppose the target of m is in component 1, (the other case is similar). Thus $\nu]_0^{IO, \ll} = \nu]_0^{IO', \ll'} \xrightarrow{\langle \text{out} : m \rangle} \kappa_{n+1}^0$, $\nu]_1^{IO, \ll} = \nu]_1^{IO', \ll'} \xrightarrow{\langle \text{in} : m \rangle} \kappa_{n+1}^1$, and by induction $sMerged(\nu]_0^{IO', \ll'}, \nu]_1^{IO', \ll'}, \nu)$. Hence by clause (3) of the definition $sMerged(\nu]_0^{IO, \ll}, \nu]_1^{IO, \ll}, \nu)$. If there is no m such that $(n, m) \in IO$, then suppose the focus of l_n is in component 0, (the other case is similar). Thus $\nu]_0^{IO, \ll} = \nu']_0^{IO, \ll} \xrightarrow{l_n} \kappa_{n+1}^0$, $\nu]_1^{IO, \ll} = \nu']_1^{IO, \ll}$, and by induction $sMerged(\nu']_0^{IO, \ll}, \nu']_1^{IO, \ll}, \nu')$. Hence by clause (1) of the definition $sMerged(\nu]_0^{IO, \ll}, \nu]_1^{IO, \ll}, \nu)$.

□

Definition ($\mathcal{T}(\kappa_0) \parallel \mathcal{T}(\kappa_1)$): The composition of the computation trees of composable configurations κ_0, κ_1 is defined as follows.

$$\mathcal{T}(\kappa_0) \parallel \mathcal{T}(\kappa_1) = \{\nu \mid (\exists \nu_0 \in \mathcal{T}(\kappa_0), \nu_1 \in \mathcal{T}(\kappa_1))(sMerged(\nu_0, \nu_1, \nu))\}$$

Theorem (Composition of Computation Trees): The computation tree of the composition of actor configurations is the composition of the computation trees of the components.

$$\mathcal{T}(\kappa_0 \parallel \kappa_1) = \mathcal{T}(\kappa_0) \parallel \mathcal{T}(\kappa_1)$$

Proof : by (merged-cseq) and (project-merged). □

5.1.2. The Merging of Paths

Now we extend the composition result to computation paths. We first give two definitions of merged paths and show they are equivalent.

Definition (path-merge-1): $pMerged_1(\pi_0, \pi_1, \pi)$ iff the initial configurations of π_0, π_1 induce a decomposition of π and there is a fair, disambiguated i/o-marking $IO, \ll$ of π for this decomposition such that π_j is the projection of π determined by $IO, \ll$ onto component j for $j < 2$.

Definition (path-merge-2): $pMerged_2(\pi_0, \pi_1, \pi)$ iff for each initial sequence ν of π there are initial sequences ν_j of π_j for $j < 2$ such that $sMerged(\nu_0, \nu_1, \nu)$, and for each initial sequence ν_j of π_j (for $j < 2$) there are initial sequences ν of π and ν_{1-j} of π_{1-j} such that $sMerged(\nu_0, \nu_1, \nu)$.

Lemma (pm12): $pMerged_1(\pi_0, \pi_1, \pi)$ iff $pMerged_2(\pi_0, \pi_1, \pi)$.

Proof (pm12): For the only if direction, assume $pMerged_1(\pi_0, \pi_1, \pi)$. Let $[(\kappa_i^0, \kappa_i^1) \mid i < \boxtimes]$ be the decomposition of π induced by the initial configurations of π_0, π_1 , let $IO, \ll$ be the fair, disambiguated i/o-marking of π for this decomposition with projections π_0, π_1 . Suppose ν is an initial segment of π , then by (project-restrict) the restriction of $IO, \ll$

to ν is a **i/o**-marking for ν with corresponding projections ν_j , being initial segments of the projections π_j of π and by (**project-merged**) $sMerged(\nu_0, \nu_1, \nu)$. Suppose ν_0 is an initial segment of π_0 . Then by (**project-restrict**) ν_0 is the projection onto component 0 determined by some restriction of $IO, \ll$ to some initial segment ν of π . Take ν_1 to be the projection of ν onto component 1 as above. $\square_{\text{onlyif}}$

For the if direction, assume $pMerged_2(\pi_0, \pi_1, \pi)$ and let $\kappa_0, \kappa_1, \kappa$ be the corresponding initial configurations. Then (κ_0, κ_1) is a decomposition of κ . For each initial segment ν of π , let $IO_\nu, \ll_\nu$ be the maximal **i/o**-marking such that $\nu \upharpoonright_j^{IO_\nu, \ll_\nu}$ is an initial segment of π_j for $j < 2$. Note that these marking form a chain. Let $IO, \ll$ be the union of this chain of markings. We claim (i) that $IO, \ll$ is a fair **i/o**-marking for π , relative to the decomposition (κ_0, κ_1) of κ , and (ii) that $\pi_j = \pi \upharpoonright_j^{IO, \ll}$. (ii) is easy. To see (i) we show that each of the conditions of the definition of and **i/o**-marking hold. (1) and (2) follow from the fact that if $(k, m), (k', m) \in IO$, then $(k, m), (k', m) \in IO_\nu$ for some initial segment ν of π . (3) follows from the fact that if l_i is a case 2 **rcv** of π then it is a case 2 **rcv** of some initial segment ν . For (4), suppose that m is generated by l_i in component 0 with target in component 1. Then by fairness of π_0 there is some initial segment ν_0 that contains the generation and output of m . Choose ν_1, ν initial segments of π_1, π such that $pMerged(\nu_0, \nu_1, \nu)$. Then by maximality IO_ν contains (k, m) for some k . $\square_{\text{if}}$

$\square_{\text{pm12}}$

Henceforth we will write $pMerged$ without a subscript to mean the relation given by either of the above definitions. Note that, as suggested by the if-direction argument, it is necessary to require that each initial segment of π_j be mergable with some initial segment of π_{1-j} in (**path-merge-2**). Otherwise the situation could arise where $[\kappa_0^1 \xrightarrow{l} \kappa_1^1]$ is an initial sequence of π_1 , κ_0^1 is quiescent, and l is an input. Then π could just be π_0 with the extra actors tacked on, without violating any of the other conditions.

Using the definition of path merging and the facts about projections we can extend the tree composition results to fair computation sets.

Definition ($\mathcal{F}(\kappa_0) \parallel \mathcal{F}(\kappa_1)$): The composition of the sets of fair computation paths of composable configurations κ_0, κ_1 is defined as follows.

$$\mathcal{F}(\kappa_0) \parallel \mathcal{F}(\kappa_1) = \{\pi \mid (\exists \pi_0 \in \mathcal{F}(\kappa_0), \pi_1 \in \mathcal{F}(\kappa_1))(pMerged(\pi_0, \pi_1, \pi))\}$$

Theorem (Composition of Fair Computations): The set of fair computation paths of the composition of actor configurations is the composition of the fair computation paths of the components.

$$\mathcal{F}(\kappa_0 \parallel \kappa_1) = \mathcal{F}(\kappa_0) \parallel \mathcal{F}(\kappa_1)$$

Proof: For the left-to-right containment, assume $\pi \in \mathcal{F}(\kappa_0 \parallel \kappa_1)$. Let $IO, \ll$ be any fair **i/o**-marking of π for the decomposition $(\kappa_0 \parallel \kappa_1)$ of κ . Then by (**project-computation**) $\pi \upharpoonright_j^{IO, \ll} \in \mathcal{F}(\kappa_j)$ for $j < 2$ and hence $\pi \in \mathcal{F}(\kappa_0) \parallel \mathcal{F}(\kappa_1)$.

For the right-to-left containment, assume $\pi \in \mathcal{F}(\kappa_0) \parallel \mathcal{F}(\kappa_1)$ and let $\pi_j \in \mathcal{F}(\kappa_j)$ be such that $pMerged(\pi_0, \pi_1, \pi)$. Thus $\pi_j = \pi \upharpoonright_j^{IO, \ll}$ for $j < 2$ for some fair **i/o**-marking of π for the decomposition $(\kappa_0 \parallel \kappa_1)$ of κ . We only need to show that π is fair. Suppose l is

enabled at stage i in π then it is enabled at the corresponding point in one of the component paths, and by fairness of the component paths either the transition occurs at some point in the component or it becomes permanently disabled at some point in the component. The same will be true in π . $\square$

We give one further characterization of merging of computation paths in terms of their input, output transitions. Informally, π_0 and π_1 are mergable just if they have disjoint sets of actors, and if there are complimentary subsequences of their i/o transitions that include all output from one path to actors of the other path.

Definition (mergable): Let $\pi_j = [\kappa_i^j \xrightarrow{l_i} \kappa_{i+1}^j \mid i < \bowtie^j]$ be computation paths with $\kappa_i^j = \langle\langle \alpha_i^j \mid \mu_i^j \rangle\rangle_{\chi_i^j}^{\rho_i^j}$ for $i < \bowtie^j$ and $j < 2$. π_0 and π_1 are mergable just if the following conditions hold.

- (1) $\bigcup_{i < \bowtie^0} \text{Dom}(\alpha_i^0) \cap \bigcup_{i < \bowtie^1} \text{Dom}(\alpha_i^1) = \emptyset$, and
- (2) if $a \in \chi_i^j \cap \text{Dom}(\alpha_k^{1-j})$ then $a \in \rho_{k'}^{1-j}$ for some $k' \geq k$.
- (3) There are subsequences, $\xi^j = [\xi_i^j \mid i < \bowtie^j]$, of $\bowtie^j$ such that
 - (3.1) $l_{\xi_i^0}^0$ and $l_{\xi_i^1}^1$ are complimentary, for $i < \bowtie$.
 - (3.2) if $l_i^j = \langle \text{out} : m \rangle$ with the target of m a receptionist of π^{1-j} then i is in the range of ξ^j .

We call ξ^0, ξ^1 a *synchronization* for π_0, π_1 .

A synchronization for a mergable pair of paths determines a set of mergings of the paths in which complimentary i/o transitions become internal boundary crossings.

Definition (Mergings): Let $\pi_j = [\kappa_i^j \xrightarrow{l_i} \kappa_{i+1}^j \mid i < \bowtie^j]$ for $j < 2$ be mergable with synchronization ξ^0, ξ^1 . The *Mergings* $(\pi_0, \xi^0, \pi_1, \xi^1)$ is the set of computation paths $\pi = (\kappa_0^0 \parallel \kappa_0^1; \zeta)$ where $\zeta = [L_i \mid i < \bowtie]$, L_i is any interleaving of L_i^0, L_i^1 , and $L_i^j = [l_m^j \mid \xi_i^j < m < \xi_{i+1}^j]$. (In the case $\bowtie$ is finite we define $\xi_{\bowtie}^j$ to be $\bowtie^j$.)

Lemma (mergings-merged):

- (1) If $p\text{Merged}(\pi_0, \pi_1, \pi)$ then π_j for $j < 2$ are mergable.
- (2) If $\pi_j = [\kappa_i^j \xrightarrow{l_i} \kappa_{i+1}^j \mid i < \bowtie^j] \in \mathcal{F}(\kappa_0^j)$ for $j < 2$ are mergable with synchronization ξ^0, ξ^1 , then $p\text{Merged}(\pi_0, \pi_1, \pi)$ for $\pi \in \text{Mergings}(\pi_0, \xi^0, \pi_1, \xi^1)$.

Proof : For (1) it is easy to see that the hygiene conditions (1-2) of the definition of mergable are satisfied. Let $IO, \ll$ be a fair i/o-marking for π with projections π_j for $j < 2$, given by the definition of $p\text{Merged}$. This naturally induces a synchronization pair.

For (2) we need to find a fair i/o-marking $IO, \ll$ for π relative to the decomposition (κ_0^0, κ_0^1) such that $\pi_j = \pi_j^{IO, \ll}$ for $j < 2$. Let $\pi = (\kappa_0^0 \parallel \kappa_0^1; [L_i \mid i < \bowtie])$. Intuitively we mark each $i < \bowtie$ with the message m exchanged at stage i (i.e. the message part of ξ_i^j). Flattening the sequence of multi-steps we obtain a corresponding marking IO of the single step representation of the path. Because some of the L^i may be empty, some points may have multiple marks. These ordering in the multi-step sequence induces an ordering $\ll$ of the set of marks at each point in the single step sequence. Since the π_j are fair, any message

generated in π_j to an actor in π_{1-j} will be output, and hence have a mark in IO . Thus IO is fair.

□

5.2. Equivalence of Actor Configurations

We first define *interaction equivalence* for actor configurations. This is a natural extension of the notion of trace equivalence to actor configurations. Traditionally, traces are defined to be finite sequences of events or actions. Two objects are trace equivalent if they have the same set of traces. Three points distinguish interaction equivalence from the traditional trace equivalence.

- (1) Since trace equivalence is defined for finite traces only, it cannot account for liveness properties such as fairness. Thus we must also consider infinite traces. Related notions in the literature include failure sets and completed traces [7, 15, 9].
- (2) Only fair traces should be considered.
- (3) A trace semantics should more directly reflect the behavior of an actor configuration considered as a component. In particular, they should allow for the dynamic addition of external actors to the environment as the configuration evolves. To our knowledge the present work is the first attempt to seriously address this problem.

We then extend the observational equivalence on expressions to actor configurations. As with expressions, this is accomplished by considering closed configurations obtained by adding observers. We also extend interaction equivalence to expressions. We show that interaction equivalence implies observational equivalence, both for expressions and configurations. The converse is an open problem.

5.2.1. Interaction Equivalence

Informally, two actor configurations are interaction equivalent, written $\kappa_0 \cong_i \kappa_1$, if their fair computations give rise to the same set of i/o transition sequences. To make this precise we first define the i/o projection operations on transition labels, computation sequences, paths, and trees.

Definition (i/o):

$$\mathbf{i/o}(l) = \begin{cases} [\text{out} : a, m] & \text{if } l = \text{out} : a, m \\ [\text{in} : a, m] & \text{if } l = \text{in} : a, m \\ [] & \text{otherwise} \end{cases}$$

$$\mathbf{i/o}([\kappa_i \xrightarrow{l_i} \kappa'_i \mid i < \bowtie]) = *[\mathbf{i/o}(l_i) \mid i < \bowtie] = \mathbf{i/o}(l_0) * \mathbf{i/o}(l_1) * \dots * \mathbf{i/o}(l_i) \dots$$

$$\mathbf{i/o}(T) = \{\mathbf{i/o}(\pi) \mid \pi \in T\} \quad \text{for any set, } T, \text{ of paths}$$

Definition ($\kappa_0 \cong_i \kappa_1$): Let κ_0, κ_1 be actor configurations with the same receptionists. Then

$$\kappa_0 \cong_i \kappa_1 \Leftrightarrow \mathbf{i/o}(\mathcal{F}(\kappa_0)) = \mathbf{i/o}(\mathcal{F}(\kappa_1))$$

Lemma ($\cong_i$ -equiv): The relation $\cong_i$ is an equivalence relation on configurations (transitive, reflexive, symmetric).

Lemma ($\cong_i$ -compose): If $\kappa_0 \cong_i \kappa_1$ and κ is composable with κ_j for $j < 2$ then $\kappa_0 \parallel \kappa \cong_i \kappa_1 \parallel \kappa$.

Proof : It suffices to show that for each $\pi'_0 \in \mathcal{F}(\kappa_0 \parallel \kappa)$, we can find $\pi'_1 \in \mathcal{F}(\kappa_1 \parallel \kappa)$, such that $\mathbf{i/o}(\pi'_0) = \mathbf{i/o}(\pi'_1)$. Assume $\pi'_0 \in \mathcal{F}(\kappa_0 \parallel \kappa)$, and find $\pi_0 \in \mathcal{F}(\kappa_0)$, $\pi \in \mathcal{F}(\kappa)$ such that $pMerged(\pi_0, \pi, \pi'_0)$ (pick a fair $\mathbf{i/o}$ -marking and project). By (**mergings-merged**) π_0 is mergable with π . Since $\kappa_0 \cong_i \kappa_1$ we can find $\pi_1 \in \mathcal{F}(\kappa_1)$ such that $\mathbf{i/o}(\pi_0) = \mathbf{i/o}(\pi_1)$, and π_1 satisfies the conditions (1-2) for mergability with π . Thus π_1 is mergable with π and any such merging π'_1 is a fair computation for $\kappa_1 \parallel \kappa$. By careful bookkeeping we can choose and interleaving of segments between synchronization points we can find a merging $\pi'_1 \in \mathcal{F}(\kappa_1 \parallel \kappa)$ such that $\mathbf{i/o}(\pi'_0) = \mathbf{i/o}(\pi'_1)$. $\square$

Interaction equivalence naturally extends to expressions as follows. Two expressions are interaction equivalent just if replacing some occurrences of one by the other in a configuration results in interaction equivalent configurations.

Definition ($e_0 \cong_i e_1$): $e_0 \cong_i e_1$ just if $\kappa_0 \cong_i \kappa_1$ for all κ_i of the form

$$\langle\langle \alpha, [C[e_i]]_a \mid \mu \rangle\rangle_\chi^\rho$$

Using interaction equivalence we can state a conjecture concerning the connection between certain configurations and expressions that construct them. This strengthens the intuition that, at least statically, there is a strong connection between expressions and configurations. For simplicity we consider the case of a single actor configuration.

Conjecture (io-exp): Let $\kappa_j = \langle\langle (b_j(a))_a \mid \emptyset \rangle\rangle_\chi^a$, and let e_j be expressions constructing κ_j .

$$e_j = \mathbf{let}\{a := \mathbf{newadr}()\}\mathbf{let}\{z := \mathbf{initbeh}(a, b_j(a))\}a$$

Then

$$\kappa_0 \cong_i \kappa_1 \Rightarrow e_0 \cong_i e_1$$

We note that it is not necessarily the case that $b_0(x) \cong b_1(x)$. We will see an example in the next section. The intuition behind the conjecture is the following. Let O be an observing context for e_j , and let $\pi \in \mathcal{F}(O[e_0])$. Without loss we treat executions of e_0 as single steps. We annotate each configuration in π to mark the remaining occurrences of e_0 (in descendants of holes that remain untouched). We also list actors a_i that have been created by executions of e_0 and associated with each a_i we list any actors created as a result of a_i receiving a message. We consider each a_i and its associates as a subconfiguration. We mark for each a_i the subpath π_i^0 corresponding to transitions internal to the subconfiguration, inserting phantom $\mathbf{i/o}$ messages to represent the crossing of internal boundaries. We claim that $\pi_i^0 \in \mathcal{F}(\kappa_0)$ and hence we can find $\pi_i^1 \in \mathcal{F}(\kappa_1)$ such that $\mathbf{i/o}(\pi_i^0) = \mathbf{i/o}(\pi_i^1)$. We further claim that each π_i^0 can be replaced by the corresponding π_i^1 and that the result will be a fair computation for $O[e_1]$. The hard part is getting the path into canonical form so that the joint replacement can be done without getting confused.

5.2.2. Observational Equivalence

Now we extend the notion of observational equivalence to configurations.

Definition (Observing Configurations): For an actor configuration $\kappa = \langle\langle \alpha \mid \mu \rangle\rangle_\chi^\rho$ the observing configurations are configurations over the extended language of the form $\kappa' = \langle\langle \alpha' \mid \mu' \rangle\rangle_\rho^\chi$ such that κ' is composable (in the sense of §5.1) with κ .

We are interested in observing internal **event** transitions rather than interactions with the environment. Thus we define an operation $\text{Hide}(\kappa)$ hiding all the receptionists of a configuration.

Definition ($\text{Hide}(\kappa)$): $\text{Hide}(\langle\langle \alpha \mid \mu \rangle\rangle_\chi^\rho) = \langle\langle \alpha \mid \mu \rangle\rangle_\chi^\emptyset$

Definition ($\kappa_0 \cong \kappa_1$): For $\kappa_0 = \langle\langle \alpha_0 \mid \mu_0 \rangle\rangle_\chi^\rho$ and $\kappa_1 = \langle\langle \alpha_1 \mid \mu_1 \rangle\rangle_\chi^\rho$, $\kappa_0 \cong \kappa_1$ iff $\text{Obs}(\text{Hide}((\kappa_0 \parallel \kappa')) = \text{Obs}(\text{Hide}((\kappa_1 \parallel \kappa')))$ for all observing configurations κ' for κ_j , $j < 2$.

Note the following. Firstly, $\text{Hide}((\kappa \parallel \kappa'))$ is a closed configuration for observer κ' for κ . Secondly, no two closed configurations can be distinguished by an external observer.

Using observational equivalence configurations we can extend the property of congruence with respect to expression construction to congruence with respect to configuration construction. Namely, replacing an expression occurring in a configuration by an observationally equivalent one yields an equivalent configuration.

Theorem (exp-cfg): If $e_0 \cong e_1$ then

- (i) $\kappa_0 = \langle\langle \alpha, [C[e_0]]_a \mid \mu \rangle\rangle_\chi^\rho \cong \langle\langle \alpha, [C[e_1]]_a \mid \mu \rangle\rangle_\chi^\rho = \kappa_1$
- (ii) $\kappa'_0 = \langle\langle \alpha, (\lambda x.C[e_0])_a \mid \mu \rangle\rangle_\chi^\rho \cong \langle\langle \alpha, (\lambda x.C[e_1])_a \mid \mu \rangle\rangle_\chi^\rho = \kappa'_1$

Proof : (i) We need to show that $\text{Obs}(\text{Hide}(\kappa_0 \parallel \kappa')) = \text{Obs}(\text{Hide}(\kappa_1 \parallel \kappa'))$ for any observing κ' . Note however that $\text{Hide}(\kappa_0 \parallel \kappa') = O[e_0]$ and $\text{Hide}(\kappa_1 \parallel \kappa') = O[e_1]$ for some $O \in \mathbb{O}$, so the result follows directly from the definition of $\cong$.

(ii) We need to show that $\text{Obs}(\text{Hide}(\kappa'_0 \parallel \kappa')) = \text{Obs}(\text{Hide}(\kappa'_1 \parallel \kappa'))$ for any observing κ' . Pick any $\pi_0 \in \mathcal{F}(\text{Hide}(\kappa'_0 \parallel \kappa'))$, then we must find $\pi_1 \in \mathcal{F}(\text{Hide}(\kappa'_1 \parallel \kappa'))$ such that $\text{obs}(\pi_0) = \text{obs}(\pi_1)$. There are two cases to consider. Either actor a never becomes active, or it becomes active first after k steps of computation. In the first case, the e_i are never touched, so both computations proceed uniformly, thus their observation and fairness behavior both correspond. In the second case, consider the step where a receives its first message:

$$\langle\langle \alpha', (\lambda x.C[e_0])_a \mid \mu, \langle a \Leftarrow cv \rangle \rangle\rangle \xrightarrow{\langle \text{rcv} : a, cv \rangle} \langle\langle \alpha', [\text{app}(\lambda x.C[e_0], cv)]_a \mid \mu \rangle\rangle = O[e_0]$$

Factor $\pi_0 = \nu[e_0] * \pi'_0$, where $\pi'_0 \in \mathcal{F}(O[e_0])$ and $\nu[]$ denotes a sequence where each configuration in the sequence contains a hole that computes uniformly in the hole. Thus, $\text{Obs}(O[e_0]) = \text{Obs}(O[e_1])$ because $e_0 \cong e_1$ and O is a configuration context. This means

by the definition of *Obs* there is a path $\pi'_1 \in \mathcal{F}(O[e_1])$, such that $obs(\pi'_0) = obs(\pi'_1)$. Let $\pi_1 = \nu[e_1] * \pi'_1$. Then, $\pi_1 \in \mathcal{F}(\text{Hide}(\kappa_1 \parallel \kappa'))$, since by construction it is a computation for $(\text{Hide}(\kappa_1 \parallel \kappa'))$, and because π fair implies $\nu * \pi$ is fair for any ν such that $\nu * \pi$ is a computation path. Moreover, $obs(\pi_0) = obs(\pi_1)$ because any **event** transitions in $\nu[e_0]$ also occur in $\nu[e_1]$, and $obs(\pi'_0) = obs(\pi'_1)$ by hypothesis.

□

Theorem (io-op): If two configurations are interaction equivalent, then they are observationally equivalent. Similarly for expressions.

$$(1) \quad \kappa_0 \cong_i \kappa_1 \Rightarrow \kappa_0 \cong \kappa_1$$

$$(2) \quad e_0 \cong_i e_1 \Rightarrow e_0 \cong e_1$$

Proof (1): Let κ be any observing configuration for κ_0, κ_1 , and let $\pi'_0 \in \mathcal{F}(\text{Hide}(\kappa_0 \parallel \kappa))$. Then $\pi'_0 \in \mathcal{F}(\kappa_0 \parallel \kappa)$ and by **(composition of fair computations)** $\pi'_0 \in \mathcal{F}(\kappa_0) \parallel \mathcal{F}(\kappa)$. Consequently $(\exists \pi_0 \in \mathcal{F}(\kappa_0), \pi \in \mathcal{F}(\kappa))(pMerged(\pi_0, \pi, \pi'_0))$, so by **(mergings-merged)** the π_0 and π are mergable, and since all **events** take place in π we have that $obs(\pi'_0) = obs(\pi)$. Since $\kappa_0 \cong_i \kappa_1$ we can find $\pi_1 \in \mathcal{F}(\kappa_1)$ such that $i/o(\pi_0) = i/o(\pi_1)$. Hence π_1 is mergable with π and, by reasoning similar to the above, for any merging π'_1 we have $obs(\pi'_0) = obs(\pi'_1)$. Also π'_1 has no input transitions, since the only inputs of π_1, π are synchronized with outputs from the opposite component. Thus $\pi'_1 \in \mathcal{F}(\text{Hide}(\kappa_1 \parallel \kappa))$. □₁

Proof (2): Assume $e_0 \cong_i e_1$ and let O be an observing context for e_0, e_1 . Let a^* be and actor not in the domain of O and let O^* be obtained from O by replacing all occurrences of **event**() by **send**(a^*, nil). Then $\kappa_j = O^*[e_j]$ is a configuration with no receptionists and one external actor, a^* and $\kappa_0 \cong_i \kappa_1$. Let $\kappa^* = \langle\langle (\lambda m. \text{event}())_{a^*} \mid \emptyset \rangle\rangle_{\emptyset}^{a^*}$. Then κ^* is an observing configuration for κ_j for $j < 2$. Furthermore, $Obs(\text{Hide}(\kappa_j \parallel \kappa^*)) = Obs(O[e_j])$ for $j < 2$. By (1) $Obs(\text{Hide}(\kappa_0 \parallel \kappa^*)) = Obs(\text{Hide}(\kappa_1 \parallel \kappa^*))$. □₂

□_{io-op}

The converse of **(io-op)** is an open problem.

5.3. Transforming Paths

Now we develop some basic tools for manipulating paths. The main objective is to be able to treat finite sequences of transition labels as defining single multi-step transitions, and to establish conditions under which we may assume that a path has been put into suitable canonical form.

We let $\langle \text{exec} : a \rangle$ stand for any transition (label) with focus a , other than **rcv**. This implicitly excludes **in** and **out**, as they are not considered to have focus actors.

Definition (Independence –): Two transitions are independent if whenever they can occur consecutively, they can occur in either order, and both orders define the same multi-step transition. Formally, $l_0 - l_1$ just if

$$\kappa \xrightarrow{[l_0, l_1]} \kappa' \Leftrightarrow \kappa \xrightarrow{[l_1, l_0]} \kappa'$$

for all configurations κ, κ' .

Note that two transitions are (vacuously) considered independent if they can not occur adjacently. For example **rcv** is independent of **send**, **newadr**, **initbeh** at the same location because the transition immediately following a **rcv** must be **beta-v**, and no **exec** other than a **become** can precede an **rcv**.

Theorem (Independence): The following pairs of transitions are independent:

$$\langle \text{in} : m \rangle - \begin{cases} \langle \text{rcv} : a, cv \rangle & \text{if } m \neq \langle a \Leftarrow cv \rangle \\ \langle \text{exec} : a \rangle & \end{cases}$$

$$\langle \text{out} : m \rangle - \begin{cases} \langle \text{rcv} : a, cv \rangle \\ \langle \text{exec} : a \rangle & \text{except } \langle \text{send} : a, m \rangle \end{cases}$$

$$\langle \text{rcv} : a, cv \rangle - \begin{cases} \langle \text{in} : m' \rangle & \text{if } m' \neq m \\ \langle \text{out} : m' \rangle \\ \langle \text{exec} : a' \rangle & \text{except } \langle \text{send} : a', \langle a \Leftarrow cv \rangle \rangle \end{cases}$$

$$\langle \text{init} : a, a' \rangle - \begin{cases} \langle \text{in/out} : m \rangle \\ \langle \text{rcv} : a_0, cv \rangle & \text{if } a_0 \neq a' \\ \langle \text{exec} : a_0 \rangle & \text{if } a_0 \neq a' \end{cases}$$

$$\langle \text{bec} : a, a' \rangle - \begin{cases} \langle \text{in/out} : m \rangle \\ \langle \text{rcv} : a_0, cv \rangle & \text{if } a_0 \neq a \\ \langle \text{exec} : a_0 \rangle & \text{if } a_0 \neq a \text{ and } a_0 \neq a' \end{cases}$$

$$\langle \text{send} : a, m \rangle - \begin{cases} \langle \text{in} : m' \rangle \\ \langle \text{out} : m' \rangle & \text{if } m' \neq m \\ \langle \text{rcv} : a_0, cv \rangle & \text{if } m \neq \langle a_0 \Leftarrow cv \rangle \\ \langle \text{exec} : a' \rangle & \text{if } a \neq a' \end{cases}$$

$$\langle \text{new} : a, a' \rangle - \begin{cases} \langle \text{in/out} : m \rangle \\ \langle \text{rcv} : a_0, cv \rangle \\ \langle \text{exec} : a_0 \rangle & \text{if } a_0 \neq a \end{cases}$$

$$\langle \text{fun} : a \rangle - \begin{cases} \langle \text{in/out} : m \rangle \\ \langle \text{rcv} : a', cv \rangle & \text{if } a' \neq a \\ \langle \text{exec} : a' \rangle & \text{if } a' \neq a \end{cases}$$

Proof (Independence): Each case must be checked. Consider the first case, $\langle \text{in} : m \rangle$. This step depends on nothing since it is an input transition, so it only must be checked that it is not permuted to be before a step that depends directly on it. The only step that directly depends on it is a receive of the same message, by inspection of the transition rules. Thus, all other transitions are independent of $\langle \text{in} : m \rangle$. The other cases may be argued similarly. $\square$

An event thread, I , at a in π is a finite subsequence of transitions of π directly initiated by receipt of a message by a and executed by a and its clones. In other words an event thread at a in π is a sequence in the computation of the configuration obtained by restricting the configuration of π at the point of the message receipt to the focus actor a . We further require that there are no receive transitions, other than the initial one, and no input or output transitions.

Definition (event thread): Let

$$\begin{aligned} \mathcal{F}(\kappa) &= [\kappa_i \xrightarrow{l_i} \kappa_{i+1} \mid i < \bowtie], \text{ and } \pi \in \mathcal{F}(\kappa), \\ I &= [i_j \mid j < n] \text{ such that } j < j' < n \Rightarrow i_j < i_{j'}, \\ L(I, \pi) &= [l_j \mid j < n], \text{ the transition sequence corresponding to } I \text{ in } \pi. \end{aligned}$$

Then I is an *event thread* at a in π (initiated at i_0) if

- (1) $l_{i_0} = \text{rcv}(a, cv)$ for some cv ,
- (2) $L(I, \pi)$ contains no **rcv**, **in** or **out** transitions after the first **rcv**, and
- (3) $L(I, \pi)$ is a computation for $\ll \alpha_{i_0} \rfloor \{a\} \mid \langle a \Leftarrow cv \rangle \gg$.

Condition (3) makes explicit the structure of an event thread, running the actor in a configuration with only itself and no receives can only compute itself and its clones.

Definition (macro-step): Let $\pi \in \mathcal{F}(\kappa)$. A macro step at a in π is a tail of an event thread at a in π , i.e. an event thread with some of the initial transitions possibly omitted.

Definition (permutation): Let $\pi = \kappa; \zeta \in \mathcal{F}(\kappa)$. $\pi' = \kappa; \zeta' \in \mathcal{F}(\kappa)$ is a permutation of π if there is a bijection p on $\{0 \leq i < \bowtie\}$ where $\bowtie$ is the length (domain) of ζ and ζ' such that $\zeta' = \zeta \circ p$, i.e. $\zeta'(i) = \zeta(p(i))$ for $0 \leq i < \bowtie$.

Theorem (macro-step): Let $\pi \in \mathcal{F}(\kappa)$, and let I be a macro step in π . Then we can find $\pi' \in \mathcal{F}(\kappa)$ and a bijection p such that π' is a permutation of π via p , and letting I' be the image of I under p , $L(I', \pi')$ is a multi-step of π' : $I' = [k + i \mid 0 \leq i < \text{Len}(I)]$ for some k .

Corollary (macro-step): The π' given by (macro-step) also satisfies:

- (io) $\text{i/o}(\pi) = \text{i/o}(\pi')$
- (ev) $\text{obs}(\pi) = \text{obs}(\pi')$

Proof (macro-step): This is just iterated application of (independence) to bubble up steps to occur immediately after the initial **rcv**. $\square$

Theorem (infinite-macro-steps): Let $\pi \in \mathcal{F}(\kappa)$, and let I_j for $j \in J$ and index set J be a family (possibly infinite) of pairwise disjoint macro steps in π . (Two macro steps I, I' in π are disjoint if they have an empty intersection.) Then we can find $\pi' \in \mathcal{F}(\kappa)$ and a bijection p such that

- (1) π' is a permutation of π via p ;
- (2) letting I'_j be the image of I_j under p , then $L(I'_j, \pi')$ is a multi-step transition of π' for $j \in J$

Corollary (infinite-macro-steps): Let π' be as in (infinite-macro-steps), then

- (io) $i/o(\pi) = i/o(\pi')$
- (ev) $obs(\pi) = obs(\pi')$

Proof: p is computed by induction on the index set. It will be the identity up to the point where the first macro step begins. The next group of transitions will be those of the macro step. Then add the length of the macro step to transitions not included until the next macro step begins and so on. It is easy to see that the result is a computation. Enabledness of transitions is not changed, and every transition of π occurs either as part of a macro step or delayed by a finite amount corresponding to the sum of the lengths of the preceding (finite number of) macro steps. Thus fairness is also preserved. (1) is ensured by construction, and (2) is ensured by the fact that the two computations are related by a bijection on the sequence of transition labels. $\square$

6. Examples of Configuration Equivalence

In this section we give two examples that illustrate a method for establishing equivalence of configurations. This first example is an instance of a simple transformation that eliminates indirection in returning a reply to a message. The second example involves a variety of ways in which recursion can be expressed. Equivalence is established by proving interaction equivalence. This is accomplished by establishing a causality relation between messages input from the environment and messages output to the environment. Then we show that for each configuration and each possible pattern of input/output augmented with causality there is a fair computation path with that pattern. The work of establishing the causality relation is made easier by showing that computations can be put into a simple canonical form.

6.1. Indirection Elimination

We begin with a two actor system with one receptionist and one external actor. The receptionist takes requests for transforming data, applies one operation, and sends the result to the other internal actor. The second actor applies a second operation and sends the result back to the receptionist, who returns it to the external actor. This is transformed into a system in which the second actor is modified to return its results directly to the external actor. The receptionist is unchanged. We show that this transformation is sound, i.e. that the two systems are equivalent, using the ideas sketched above. This approach also works for transformations such as fusion or splitting of internal actors [33]. We assume that $\mathbf{do?}$, $\mathbf{ret?}$ are true on disjoint sets of messages, and that $\mathbf{ret?}(\mathbf{mkret}(v))$ is true. We also assume that f and g are functions that, when applied to communicable values, have finite

computations, with no observable effects, returning communicable values. Let procedures **b0**, **b1** be defined by

$$\begin{aligned}
 b_0 &= \lambda x. \lambda a. \mathbf{rec}(\lambda b. \lambda m. \mathbf{seq}(\mathbf{become}(b), \\
 &\quad \mathbf{if}(\mathbf{do?}(m), \\
 &\quad \quad \mathbf{send}(a, f(m)), \\
 &\quad \mathbf{if}(\mathbf{ret?}(m), \mathbf{send}(x, m), \mathbf{nil})))) \\
 b_1 &= \lambda c. \mathbf{rec}(\lambda b. \lambda m. \mathbf{seq}(\mathbf{become}(b), \mathbf{send}(c, \mathbf{mkret}(g, m))))
 \end{aligned}$$

Now we construct the two systems, each with receptionist a_0 and external actor x . They differ only in whether the parameter c in the internal actors behavior is instantiated to a_0 (indirect reply) or to x (direct reply). Thus we define behaviors B_0 and $B_1(c)$, and $\kappa^{(c)}$, for $c \in \{a_0, x\}$.

$$B_0 = \mathbf{b0}(x, a_1), \quad B_1(c) = \mathbf{b1}(c), \quad \kappa^{(c)} = \left\langle\!\left\langle (B_0)_{a_0}, (B_1(c))_{a_1} \mid \emptyset \right\rangle\!\right\rangle_x^{a_0}$$

The soundness of the transformation from $\kappa^{(a_0)}$ to $\kappa^{(x)}$ is expressed by the following theorem.

Theorem (rem.indir): $\kappa^{(a_0)} \cong_i \kappa^{(x)}$

Corollary (rem.indir): $\kappa^{(a_0)} \cong \kappa^{(x)}$

The proof uses two key lemmas. The first lemma (**ri-1**) says that if we are given a (possibly infinite) sequence ζ of input and output transitions for $\kappa^{(c)}$, and a map giving for each occurrence of an output transition in ζ the (unique) input transition occurrence to which it is a reply, then we can find $\pi \in \mathcal{F}(\kappa^{(c)})$ such that $\mathbf{i/o}(\pi) = \zeta$, for $c \in \{a_0, x\}$. The second lemma (**ri-2**) says that if $\pi \in \mathcal{F}(\kappa^{(c)})$, then we can find a map γ , such that $\mathbf{i/o}(\pi), \gamma$ satisfy the conditions of (**ri-1**).

Lemma (ri-1): Let ζ be a (possibly infinite) sequence of input and output transitions for $\kappa^{(c)}$

$$\zeta = [l_i \mid i < \infty] \quad \text{where } l_i \in \langle \mathbf{in} : \langle a_0 \Leftarrow c\mathbb{V} \rangle \rangle \cup \langle \mathbf{out} : \langle x \Leftarrow c\mathbb{V} \rangle \rangle \text{ for } i < \infty$$

and define

$$\begin{aligned}
 \mathbf{in}_\zeta &= \{i \mid \zeta(i) \in \langle \mathbf{in} : \langle a_0 \Leftarrow c\mathbb{V} \rangle \rangle\} \\
 \mathbf{out}_\zeta &= \{i \mid \zeta(i) \in \langle \mathbf{out} : \langle x \Leftarrow c\mathbb{V} \rangle \rangle\}.
 \end{aligned}$$

Suppose γ maps each output to its causing input, and each input that satisfies **ret?** or **do?** causes an output with appropriate contents. I.e., $\gamma \in [\mathbf{out}_\zeta \rightarrow \mathbf{in}_\zeta]$ is an injection such that

- (1) $\gamma(i) < i$ for $i \in \mathbf{out}_\zeta$
- (2) if $\zeta(j) = \langle \mathbf{in} : \langle a_0 \Leftarrow cv \rangle \rangle$ then $\gamma(i) = j$ for some i with $\zeta(i) = \langle \mathbf{out} : \langle x \Leftarrow cv' \rangle \rangle$, and
 - (do) if **do?**(cv), then $cv' = \mathbf{mkret}(g(f(cv)))$
 - (ret) if **ret?**(cv), then $cv' = cv$.

Then we can find $\pi \in \mathcal{F}(\kappa^{(c)})$ such that $\mathbf{i/o}(\pi) = \zeta$, for $c \in \{a_0, x\}$.

Lemma (ri-2): For $c \in \{a_0, x\}$, if $\pi \in \mathcal{F}(\kappa^{(c)})$, then we can find γ , such that $\mathbf{i/o}(\pi), \gamma$ satisfy the conditions of (ri-1).

Before we prove these lemmas we show how they are used to prove the soundness theorem.

Proof (rem.indir): It suffices to show that, for $c, c' \in \{a_0, x\}$, if $\pi \in \mathcal{F}(\kappa^{(c)})$ then we can find $\pi' \in \mathcal{F}(\kappa^{(c')})$ such that $\mathbf{i/o}(\pi) = \mathbf{i/o}(\pi')$. Assume $\pi \in \mathcal{F}(\kappa^{(c)})$. Then by (ri-2) we obtain γ such that $\mathbf{i/o}(\pi_c), \gamma$ satisfy the conditions of (ri-1). Then by (ri-1) we obtain $\pi' \in \mathcal{F}(\kappa^{(c')})$ such that $\mathbf{i/o}(\pi') = \mathbf{i/o}(\pi)$, as required. $\square_{\text{rem.indir}}$

The proof of the lemmas is greatly simplified by showing that any fair computation path for one of the initial configurations can be replaced (preserving fairness and observable trace) by another computation path (for the same initial configuration) that is in a simple canonical form in which the processing of each message received by an internal actor can be thought of as a single multi-step. Thus there are only input, output, and receive transitions to consider. To describe the canonical form, we first define the multi-step transitions $R_0(cv)$ and $R_1(cv, c)$.

Definition (R_0): Let π be a computation for $\kappa^{(c)}$ in which a message $\langle a_0 \Leftarrow cv \rangle$ occurs at some stage. Let I be the maximal event thread at a_0 in π beginning with $\langle \mathbf{rcv} : a_0, cv \rangle$. Then $R_0(cv) = [l_i \mid i \in I]$. Thus, if

$$\kappa = \left\langle\left\langle (B_0)_{a_0}, (B_1(c))_{a_1} \mid \{\langle a_0 \Leftarrow cv \rangle\} \right\rangle\right\rangle_x^{a_0}$$

then $\kappa \xrightarrow{R_0(cv)} \kappa'$ by execution transitions focused at a_0 (no input, output, or receive transitions), no execution transitions are enabled in κ' . From the definition of B_0 we can see that

$$\kappa' = \begin{cases} \left\langle\left\langle (B_0)_{a_0}, (B_1(c))_{a_1} \mid \{\langle a_1 \Leftarrow cv' \rangle\} \right\rangle\right\rangle_x^{a_0} & \text{if } \mathbf{do?}(cv) \text{ and } cv' = f(cv) \\ \left\langle\left\langle (B_0)_{a_0}, (B_1(c))_{a_1} \mid \{\langle x \Leftarrow cv' \rangle\} \right\rangle\right\rangle_x^{a_0} & \text{if } \mathbf{ret?}(cv) \text{ and } cv' = cv \\ \left\langle\left\langle (B_0)_{a_0}, (B_1(c))_{a_1} \mid \emptyset \right\rangle\right\rangle_x^{a_0} & \text{if neither } \mathbf{ret?}(cv) \text{ nor } \mathbf{do?}(cv). \end{cases}$$

Note that the definition of R_0 is independent of the particular computation, since the behavior of a_0 does not change.

Definition (R_1): $R_1(cv)$ is obtained from the maximal event thread beginning with $\langle \mathbf{rcv} : a_1, cv \rangle$ as in the definition of R_0 . Thus, if

$$\kappa = \left\langle\left\langle (B_0)_{a_0}, (B_1(c))_{a_1} \mid \{\langle a_1 \Leftarrow cv \rangle\} \right\rangle\right\rangle_x^{a_0}$$

then $\kappa \xrightarrow{R_1(cv)} \kappa'$ by execution transitions focused at a_1 (no input, output, or receive transitions), and no execution transitions remain enabled in κ' . From the definition of $B_1(c)$ we can see that

$$\kappa' = \left\langle\left\langle (B_0)_{a_0}, (B_1(c))_{a_1} \mid \{\langle c \Leftarrow \mathbf{mkret}(g(cv)) \rangle\} \right\rangle\right\rangle_x^{a_0}.$$

Lemma (canon-ri): Let $\pi \in \mathcal{F}(\kappa^{(c)})$. Then we can find $\pi' \in \mathcal{F}(\kappa^{(c)})$ such that $\mathbf{i}/\mathbf{o}(\pi) = \mathbf{i}/\mathbf{o}(\pi')$ and $\pi' = [\kappa_i \xrightarrow{L_i} \kappa_{i+1} \mid i \in \mathbb{N}]$ where L_i is one of the following:

- an input of the form $\langle \mathbf{in} : \langle a_0 \Leftarrow cv \rangle \rangle$;
- an output of the form $\langle \mathbf{out} : \langle x \Leftarrow cv \rangle \rangle$; or
- one of the multi-step transitions $R_0(cv)$ or $R_1(cv)$.

Proof : By (infinite macro-steps). $\square$

Proof (ri-1): Assume the conditions of (ri-1) hold for ζ, γ . Define π by inserting multi-step transitions: (1) just after each input of a no-op message; and (2) just before each output transition. Suppose $\zeta(j) = \langle \mathbf{in} : \langle a_0 \Leftarrow cv \rangle \rangle$ where neither $\mathbf{do?}(cv)$ nor $\mathbf{ret?}(cv)$ hold. Then insert $R_0(cv)$ just after j . Suppose $i \in \mathbf{out}_\zeta$, $\gamma(i) = j$, and $\zeta(j) = \langle \mathbf{in} : \langle a_0 \Leftarrow cv \rangle \rangle$. Then we define the sequence L_i to insert as follows. $L_i = [R_0(cv)] \circ L'$ where L' is empty unless $\mathbf{do?}(cv)$, in which case

$$L' = [R_1(f(cv)), R_0(\mathbf{mkret}(g(f(cv))))], \quad \text{if } c = a_0, \text{ and}$$

$$L' = [R_1(f(cv))], \quad \text{if } c = x.$$

It is easy to see that the result of the insertions π is a computation path for $\kappa^{(c)}$ with $\mathbf{i}/\mathbf{o}(\pi) = \zeta$. To see that π is fair, note that the only transitions that are enabled and not taken immediately are receives of input messages that cause outputs. This is because any input that causes no output is received immediately, and any message to x that appears in the message pool is output immediately following the multi-step transition that enables it. For the remaining input messages there is a corresponding output by the conditions on γ and its receipt occurs in the multi-step $\square$

Proof (ri-2): Suppose $\pi \in \mathcal{F}(\kappa^{(c)})$ and let $\zeta = \mathbf{i}/\mathbf{o}(\pi)$. By (canon-ri) we may assume $\pi = [\kappa_i \xrightarrow{L_i} \kappa_{i+1} \mid i \in \mathbb{N}]$ where L_i is either an input, an output, or one of the multi-step transitions $R_j(cv)$. Thus the interesting configurations in π only differ in the message component. We annotate messages with the index of the input event of the causing message. An input message causes itself, and the cause annotation of the message, if any, generated by $R_j(cv)$ is the same as the annotation of the message processed. Thus, ignoring the unchanging parts of the configurations:

$$\begin{aligned} & \mu_i \xrightarrow{\langle \mathbf{in} : \langle a_0 \Leftarrow cv \rangle \rangle} \mu_i \cup \{ \langle a_0 \Leftarrow cv \rangle^i \} \\ & \mu_i \{ \langle a_0 \Leftarrow cv \rangle^j \} \xrightarrow{R_0(cv)} \mu_i \cup \begin{cases} \{ \langle a_1 \Leftarrow f(cv) \rangle^j \} & \text{if } \mathbf{do?}(cv) \\ \{ \langle x \Leftarrow cv \rangle^j \} & \text{if } \mathbf{ret?}(cv) \\ \{ \} & \text{otherwise} \end{cases} \\ & \mu_i \{ \langle a_1 \Leftarrow cv \rangle^j \} \xrightarrow{R_1(cv)} \mu_i \cup \{ \langle c \Leftarrow g(cv) \rangle^j \} \end{aligned}$$

For $i \in \mathbf{out}_\varphi$, let i' be the index of the output transition in π and let j' be the annotation of the message output at i . Let j be the index of the input transition j' in ζ and define $\gamma(i) = j$. Then it is easy to see from the definitions of the multi-step transitions that ζ, γ satisfies the conditions of (ri-1). $\square$

6.2. Varieties of Recursion as Patterns of Message Passing

We define a trivial recursive procedure, **cd**, which takes a number, counts down to 0 and returns 0. We then define various actors that have the count-down behavior. An actor with count-down behavior responds to a message containing a number and a caller, by counting from that number down to 0 and sending a message containing 0 to the caller. The behaviors B_0 and B_1 do this by simply applying **cd** to the number. The difference is that B_0 sends the reply before becoming ready for the next message, while B_1 becomes ready for the next message immediately, leaving the work of the current message to a clone. This increases the opportunity for parallelism – i.e the number of actors with work to do. B_2 implements the count-down recursion by decrementing the number and sending itself a message with the decremented number and the caller. When the number reaches 0, B_2 replies to the caller. In this way an actor with behavior B_2 may interleave working on requests from various callers. B_3 creates a new actor to handle each message. Again, this is a way of increasing parallelism. The new actor has behavior B_2 . It could equally well have been given behavior B_0 or B_1 . Finally B_4 also creates a new actor for each message. This new actor implements a loop with loop variable i (the current value of the number counting down). It goes once around the loop each time it receives a ‘tick’ (any message at all).

In fact, these actors are all equivalent to one that, for messages of the appropriate form, returns 0 to the caller immediately. The point is not to analyze the functional behavior, but to analyze the message patterns used to express the recursion. The examples can easily be elaborated to behaviors that compute more interesting functions using the same recursion patterns. We chose this simple case to focus on methods for proving equivalence of the various recursion patterns.

To avoid anomalies, we choose behaviors which ignore messages that are not of the expected form, i.e. messages that are not pairs whose first component is a number. This is done using the test **bad?**.

```

bad? =  $\lambda x.$ if(pr( $x$ ), if(isnat( $1^{\text{st}}$ ( $x$ )), nil, t), t)
cd = rec( $\lambda d \lambda x.$ if(eq( $x$ , 0), 0,  $d(x - 1)$ ))
 $B_0$  = rec( $\lambda b. \lambda m.$ 
    if(bad?( $m$ ),
        become( $b$ ),
        let{ $n := 1^{\text{st}}$ ( $m$ ),  $c := 2^{\text{nd}}$ ( $m$ )}seq(send( $c$ , cd( $n$ )), become( $b$ ))))
 $B_1$  = rec( $\lambda b. \lambda m.$ 
    if(bad?( $m$ ),
        become( $b$ ),
        let{ $n := 1^{\text{st}}$ ( $m$ ),  $c := 2^{\text{nd}}$ ( $m$ )}seq(become( $b$ ), send( $c$ , cd( $n$ ))))

```

$$B_2 = \lambda a. \mathbf{rec}(\lambda b. \lambda m. \\
\quad \mathbf{if}(\mathbf{bad?}(m), \\
\quad \quad \mathbf{become}(b), \\
\quad \quad \mathbf{let}\{n := 1^{\mathbf{st}}(m), c := 2^{\mathbf{nd}}(m)\} \\
\quad \quad \quad \mathbf{seq}(\mathbf{become}(b), \\
\quad \quad \quad \mathbf{if}(\mathbf{eq}(n, 0), \mathbf{send}(c, n), \mathbf{send}(a, \mathbf{pr}(n-1, c)))))$$

$$B_3 = \mathbf{rec}(\lambda b. \lambda m. \\
\quad \mathbf{if}(\mathbf{bad?}(m), \\
\quad \quad \mathbf{become}(b), \\
\quad \quad \mathbf{let}\{n := 1^{\mathbf{st}}(m), c := 2^{\mathbf{nd}}(m)\} \\
\quad \quad \quad \mathbf{seq}(\mathbf{become}(b), \\
\quad \quad \quad \mathbf{let}\{a := \mathbf{newadr}()\} \\
\quad \quad \quad \mathbf{seq}(\mathbf{initbeh}(a, B_2(a)), \mathbf{send}(a, \mathbf{pr}(n, c)))))$$

$$B_{4x} = \lambda a. \lambda c. \mathbf{rec}(\lambda b. \lambda i. \lambda m. \\
\quad \mathbf{if}(\mathbf{eq}(i, 0), \\
\quad \quad \mathbf{send}(c, 0), \\
\quad \quad \mathbf{seq}(\mathbf{become}(b(i-1)), \mathbf{send}(a, \mathbf{nil}))))$$

$$B_4 = \mathbf{rec}(\lambda b. \lambda m. \\
\quad \mathbf{if}(\mathbf{bad?}(m), \\
\quad \quad \mathbf{become}(b), \\
\quad \quad \mathbf{let}\{n := 1^{\mathbf{st}}(m), c := 2^{\mathbf{nd}}(m)\} \\
\quad \quad \quad \mathbf{seq}(\mathbf{become}(b), \\
\quad \quad \quad \mathbf{let}\{a := \mathbf{newadr}()\} \\
\quad \quad \quad \mathbf{seq}(\mathbf{initbeh}(a, B_{4x}(a, c, n)), \mathbf{send}(a, \mathbf{nil}))))$$

Define the configurations $\kappa^{(j)}$ for $j \leq 4$ by

$$\kappa^{(j)} = \left\langle\!\left\langle (B_j)_a \mid \emptyset \right\rangle\!\right\rangle_{\emptyset}^{\{a\}} \quad \text{for } j \in \{0, 1, 3, 4\}$$

$$\kappa^{(2)} = \left\langle\!\left\langle (B_2(a))_a \mid \emptyset \right\rangle\!\right\rangle_{\emptyset}^{\{a\}}$$

Theorem (cd-c): The configurations $\kappa^{(j)}$ for $j \leq 4$ are interaction equivalent (and hence observationally equivalent).

Define the expressions $e^{(j)}$ for $j \leq 4$ by

$$e^{(j)} = \mathbf{let}\{a := \mathbf{newadr}\} \mathbf{initbeh}(a, B_j) \quad \text{for } j \in \{0, 1, 3, 4\}$$

$$e^{(2)} = \mathbf{let}\{a := \mathbf{newadr}\} \mathbf{initbeh}(a, B_2(a))$$

By **(io-exp)** we have the following corollary, expressing the equivalences in terms of expressions constructing the various actor systems.

Corollary (cd-e): The expressions $e^{(j)}$ for $j \leq 4$ are observationally equivalent.

The proof of **(cd-c)** is similar to that for the indirection removal example. It relies on two lemmas that show that the observable behavior is characterized by the possible i/o paths and maps from output to causing input.

Lemma (cd-1): Let ζ be a (possibly infinite) sequence of input/output transitions:

$$\zeta = [l_i \mid i < \infty] \quad \text{where } l_i \in \langle \text{in} : \langle a \Leftarrow \text{pr}(\mathbb{N}, \mathbb{A}d) \rangle \rangle \cup \langle \text{out} : \langle \mathbb{A}d \Leftarrow 0 \rangle \rangle \text{ for } i < \infty$$

and define

$$\text{in}_\zeta = \{i \mid \zeta(i) \in \langle \text{in} : \langle a \Leftarrow \text{pr}(\mathbb{N}, \mathbb{A}d) \rangle \rangle\}$$

$$\text{out}_\zeta = \{i \mid \zeta(i) \in \langle \text{out} : \langle \mathbb{A}d \Leftarrow 0 \rangle \rangle\}$$

Suppose $\gamma \in [\text{out}_\zeta \rightarrow \text{in}_\zeta]$ is an injection such that

- (1) $\gamma(i) < i$ with $i \in \text{out}_\zeta$, and
- (2) if $\zeta(j) = \langle \text{in} : \langle a \Leftarrow \text{pr}(n, c) \rangle \rangle$ with $n \in \mathbb{N}$, then $j = \gamma(i)$ for some i with $\zeta(i) = \langle \text{out} : \langle c \Leftarrow 0 \rangle \rangle$.

Then we can find $\pi \in \mathcal{F}(\kappa^{(j)})$ such that $\text{i/o}(\pi) = \zeta$, for each case $j \leq 4$.

We interpret (1) to mean that γ maps each output to its causing input, and (2) to mean that each input of the correct form causes an output.

Lemma (cd-2): For each case $j \leq 4$, if $\pi \in \mathcal{F}(\kappa^{(j)})$, then we can find γ , such that $\text{i/o}(\pi), \gamma$ satisfy the conditions of **(cd-1)**.

The equivalence of configurations $\kappa^{(j)}$ for $j \leq 4$, follows easily from **(cd-1, cd-2)** as in the previous example.

Proof (cd-c): It suffices to show that for any $i, j \leq 4$, if $\pi^i \in \mathcal{F}(\kappa^{(i)})$ then we can find $\pi^j \in \mathcal{F}(\kappa^{(j)})$ such that $\text{i/o}(\pi^i) = \text{i/o}(\pi^j)$. Assume $\pi^i \in \mathcal{F}(\kappa^{(i)})$. Then by **(cd-2)** we obtain γ such that $\text{i/o}(\pi^i), \gamma$ satisfy the conditions of **(cd-1)**. Then by **(cd-1)** we obtain $\pi^j \in \mathcal{F}(\kappa^{(j)})$ such that $\text{i/o}(\pi^j) = \text{i/o}(\pi^i)$, as required. $\square_{\text{cd-c}}$

Again, the key to proving the lemmas is to find a suitable canonical form for the fair computations of each initial configuration. The points of interest are input, output, receive, and count-down steps. We define the multi-step transitions $R_j(a, m, a')$ and $Cd_j(a', c, n)$. The parameter a in R_j and a' in Cd_j are the focus of the multi-step transitions. The parameter a' in R_j is the actor that will carry out the count down (a when j is 0 or 2 and a clone or newly created actor when j is 1, 3, or 4).

Definition (R_j, Cd_j): If $\text{bad?}(cv)$, then

$$\left\langle \left\langle \alpha, (B_j)_a \mid \mu, \langle a \Leftarrow cv \rangle \right\rangle \right\rangle_\chi \xrightarrow{R_j(a, cv, a)}^a \left\langle \left\langle \alpha, (B_j)_a \mid \mu \right\rangle \right\rangle_\chi^a$$

Otherwise the multi-step transitions are defined by rules of the form

$$\alpha_0 \mid \mu_0 \xrightarrow{L} \alpha_1 \mid \mu_1$$

where α_0 will specify the state of a single actor, the focus of L , and μ_0 specifies the messages consumed, if any. α_1 will specify the resulting state of the focus actor, as well as any newly created/cloned actors. μ_1 will be the messages generated, if any. Each such rule abbreviates a transition

$$\langle\langle \alpha \cup \alpha_0 \mid \mu \cup \mu_0 \rangle\rangle_{\chi} \xrightarrow{L} \langle\langle \alpha \cup \alpha_1 \mid \mu \cup \mu_1 \rangle\rangle_{\chi}$$

where $\text{Dom}(\alpha) \cap \text{Dom}(\alpha_j) = \emptyset$ and $\cup$ on messages is multiset union. L is defined as the least multi-step transition leading from the left hand configuration of the rule to the right hand configuration.

$$E_0(c, n) = \mathbf{seq}(\mathbf{send}(c, \mathbf{cd}(n)), \mathbf{become}(B_0))$$

$$(B_0)_a \mid \langle a \Leftarrow \mathbf{pr}(n, c) \rangle \xrightarrow{R_0(a, \mathbf{pr}(n, c), a)} [E_0(c, n)]_a \mid \emptyset$$

$$[E_0(c, n)]_a \mid \emptyset \xrightarrow{Cd_0(a, c, n)} \begin{cases} (B_0)_a \mid \langle c \Leftarrow 0 \rangle & \text{if } n = 0 \\ [E_0(c, n-1)]_a \mid \emptyset & \text{if } n > 0 \end{cases}$$

$$E_1(c, n) = \mathbf{send}(c, \mathbf{cd}(n))$$

$$(B_1)_a \mid \langle a \Leftarrow \mathbf{pr}(n, c) \rangle \xrightarrow{R_1(a, \mathbf{pr}(n, c), a')} (B_1)_a, [E_1(c, n)]_{a'} \mid \emptyset$$

a' fresh

$$[E_1(c, n)]_{a'} \mid \emptyset \xrightarrow{Cd_1(a', c, n)} \begin{cases} [\mathbf{nil}]_{a'} \mid \langle c \Leftarrow 0 \rangle & \text{if } n = 0 \\ [E_1(c, n-1)]_{a'} \mid \emptyset & \text{if } n > 0 \end{cases}$$

$$(B_2(a))_a \mid \langle a \Leftarrow \mathbf{pr}(n, c) \rangle \xrightarrow{R_2(a, \mathbf{pr}(n, c), a)} (B_2)_a \mid m$$

where $m = \langle c \Leftarrow 0 \rangle$ if $n = 0$, and $m = \langle a \Leftarrow \mathbf{pr}(n-1, c) \rangle$ if $n > 0$

$$Cd_2(a, c, n) = R_2(a, \mathbf{pr}(c, n), a)$$

$$(B_3)_a \mid \langle a \Leftarrow \mathbf{pr}(n, c) \rangle \xrightarrow{R_3(a, \mathbf{pr}(n, c), a')} (B_3)_a, (B_2(a'))_{a'} \mid \langle a' \Leftarrow \mathbf{pr}(n, c) \rangle$$

a' fresh

$$Cd_3(a', c, n) = R_2(a', \mathbf{pr}(c, n), a')$$

$$(B_4)_a \mid \langle a \Leftarrow \mathbf{pr}(n, c) \rangle \xrightarrow{R_4(a, \mathbf{pr}(n, c), a')} (B_4)_a, (B_{4x}(a', c, n))_{a'} \mid \langle a' \Leftarrow \mathbf{nil} \rangle$$

a' fresh

$$(B_{4x}(a', c, 0))_{a'} \mid \langle a' \Leftarrow \mathbf{nil} \rangle \xrightarrow{Cd_4(a', c, 0)} [\mathbf{nil}]_{a'} \mid \langle c \Leftarrow 0 \rangle$$

$$(B_{4x}(a', c, n+1))_{a'} \mid \langle a' \Leftarrow \mathbf{nil} \rangle \xrightarrow{Cd_4(a', c, n+1)} (B_{4x}(a', c, n))_{a'} \mid \langle a' \Leftarrow \mathbf{nil} \rangle$$

Lemma (canon-cd): Let $\pi \in \mathcal{F}(\kappa^{(j)})$. Then we can find $\pi' \in \mathcal{F}(\kappa^{(j)})$ such that $\mathbf{i/o}(\pi) = \mathbf{i/o}(\pi')$ and $\pi' = [\kappa_i \xrightarrow{L_i} \kappa_{i+1} \mid i \in \mathbb{N}]$ where L_i is one of the following:

- an input of the form $\langle \text{in} : \langle a \Leftarrow cv \rangle \rangle$;
- an output of the form $\langle \text{out} : \langle c \Leftarrow 0 \rangle \rangle$; or
- one of the multi-step transitions $R_j(a, n, a')$ or $Cd_j(a', c, n)$.

Proof : By (infinite macro-steps). $\square$

Proof (cd-1):

Assume ζ, γ satisfy the conditions of (cd-1). For $k \leq 4$ we construct $\pi^{(k)}$ by inserting multi-step transitions as follows. Suppose $\zeta(j) = \langle \text{in} : \langle a_0 \Leftarrow cv \rangle \rangle$ where $\text{bad?}(cv)$ holds. Then insert $R_j(a, cv, a)$ just after this transition. Suppose $i \in \text{out}_\zeta$, $\gamma(i) = j$, and $\zeta(j) = \langle \text{in} : \langle a \Leftarrow \text{pr}(n, c) \rangle \rangle$. Then for $\pi^{(k)}$ insert L_k just before $\zeta(i)$ where L_k is defined as follows.

$$L_0 = [R_0(a, \text{pr}(n, c), a), Cd_0(a, c, n - k) \mid k \leq n]$$

$$L_2 = [R_2(a, \text{pr}(n - k, c), a) \mid k \leq n]$$

$$L_j = [R_j(a, \text{pr}(n, c), a'), Cd_j(a', c, n - k) \mid k \leq n] \quad \text{for } j \in \{1, 3, 4\}, \text{ where } a' \text{ is fresh}$$

Clearly $\pi^{(k)}$ is a complete computation from $\kappa^{(k)}$. So we need only show fairness. From the definitions of R_k, Cd_k it is easy to see that any enabled transition other than receipt of an external message by a or output of a message sent to an external actor is enabled and taken during L_k . At any point there is at most one message waiting to be sent to an external actor. The output transition occurs just after the generating L_k . Finally for any $j \in \text{in}_\zeta$ either $\zeta(i)$ is bad and is received (discarded) immediately, or is some $i \in \text{out}_\zeta$ such that $\gamma(i) = j$ and hence the receipt of that message by a initiates the L_k inserted just before $\zeta(i)$. $\square_{\text{cd-1}}$

Proof (cd-2): Assume $\pi \in \mathcal{F}(\kappa^{(k)})$, and let $\zeta = \text{i/o}(\pi)$. By (cannon-cd) we may assume $\pi = [\kappa_i \xrightarrow{L_i} \kappa_{i+1} \mid i \in \mathbb{N}]$ where $\kappa_i = \langle \langle \alpha_i \mid \mu_i \rangle \rangle_a^a$, and L_i is either an input, an output, or one of the multi-step transitions $R_j(a, cv, a')$, or $Cd_j(a', c, n)$. We want to construct γ to satisfy the conditions of (cd-1). To do this we form an annotated version, π' , of π in which each message (occurrence) and each executing actor state, in each configuration is tagged with the index of the input event causing this occurrence or state (via a chain of receives and sends). This is done by elaborating the (multi-step) transition rules as follows.

$$\begin{aligned} & \langle \langle \alpha_i \mid \mu_i \rangle \rangle_\chi^a \xrightarrow{\langle \text{in} : m \rangle} \langle \langle \alpha_i \mid \mu_i, m^i \rangle \rangle_{\chi'}^a \\ (B_0)_a \mid \langle a \Leftarrow \text{pr}(n, c) \rangle^i & \xrightarrow{R_0(a, \text{pr}(n, c), a)} [E_0(c, n)]_a^i \mid \emptyset \\ [E_0(c, n)]_a^i \mid & \xrightarrow{Cd_0(a, c, n)} \begin{cases} (B_0)_a \mid \langle c \Leftarrow 0 \rangle^i & \text{if } n = 0 \\ [E_0(c, n - 1)]_a^i \mid \emptyset & \text{if } n > 0 \end{cases} \\ (B_1)_a \mid \langle a \Leftarrow \text{pr}(n, c) \rangle^i & \xrightarrow{R_1(a, \text{pr}(n, c), a')} (B_1)_a, [E_1(c, n)]_{a'}^i \mid \emptyset \\ [E_1(c, n)]_{a'}^i \mid \emptyset & \xrightarrow{Cd_1(a', c, n)} \begin{cases} [\text{nil}]_{a'} \mid \langle c \Leftarrow 0 \rangle^i & \text{if } n = 0 \\ [E_1(c, n - 1)]_{a'}^i \mid \emptyset & \text{if } n > 0 \end{cases} \\ (B_2(a))_a \mid \langle a \Leftarrow \text{pr}(n, c) \rangle^i & \xrightarrow{R_2(a, \text{pr}(n, c), a)} (B_2)_a \mid m^i \end{aligned}$$

$$\begin{aligned}
& (B_3)_a \mid \langle a \Leftarrow \mathbf{pr}(n, c) \rangle^i \xrightarrow{R_3(a, \mathbf{pr}(n, c), a)} (B_3)_a, (B_2(a'))_{a'} \mid \langle a' \Leftarrow \mathbf{pr}(n, c) \rangle^i \\
& (B_4)_a \mid \langle a \Leftarrow \mathbf{pr}(n, c) \rangle^i \xrightarrow{R_1(a, \mathbf{pr}(n, c), a')} (B_4)_a, (B_{4x}(a', c, n))_{a'} \mid \langle a' \Leftarrow \mathbf{nil} \rangle^i \\
& (B_{4x}(a', c, 0))_{a'} \mid \langle a' \Leftarrow \mathbf{nil} \rangle^i \xrightarrow{Cd_4(a', c, 0)} [\mathbf{nil}]_{a'} \mid \langle c \Leftarrow 0 \rangle^i \\
& (B_{4x}(a', c, n+1))_{a'} \mid \langle a' \Leftarrow \mathbf{nil} \rangle^i \xrightarrow{Cd_4(a', c, n+1)} (B_{4x}(a', c, n))_{a'} \mid \langle a' \Leftarrow \mathbf{nil} \rangle^i
\end{aligned}$$

For $i \in \mathbf{out}_\varphi$, let i' be the index of the output transition in π and let j' be the annotation of the message output at i . Let j be the index of the input transition j' in ζ and define $\gamma(i) = j$. Then it is easy to see from the definitions of the multi-step transitions that ζ, γ satisfies the conditions of **(ri-1)**. $\square_{\text{cd-2}}$

Using the analysis of the count-down behaviors, and the methods for establishing expression equivalence we can in fact show that the behaviors (lambda abstractions) B_j for $j \in \{0, 1, 3, 4\}$ are equivalent. We can not directly compare B_2 as it requires an additional argument. But, for example it is easy to see that $\lambda x. B_0$ is not equivalent to B_2 , because the equivalence of $\kappa^{(2)}$ to the others depended on applying B_2 to the address of the actor in which it occurs. An observing context that distinguishes between $\lambda x. B_0$ and B_2 is

$$\llbracket (\lambda x. \mathbf{event}())_{a_0}, (\lambda x. \mathbf{sink})_{a_1}, [\mathbf{app}(\mathbf{app}(\bullet, a_0), \mathbf{pr}(1, a_1))] \mid \emptyset \rrbracket$$

An alternative to filtering out messages of the wrong form by explicit tests in the behaviors B_i is to use a filter actor as receptionist. For this we would obtain B'_i from B_i by omitting the **bad?** test; replacing **if(bad?(m), become(b), b_i)** by b_i in each case. In the system configuration we would replace the working receptionist actor by a filter actor that does the checking and forwards messages that pass the test to the working actor. For example, define

$$\mathbf{cdin} = \lambda z. \mathbf{rec}(\lambda b. \lambda m. \mathbf{seq}(\mathbf{become}(b), \mathbf{if}(\mathbf{bad?}(m), \mathbf{nil}, \mathbf{send}(z, m))))$$

and replace the κ^j definitions by

$$\kappa^{(j)} = \llbracket (B'_j)_{a'}, (\mathbf{cdin}(a'))_a \mid \emptyset \rrbracket_{\emptyset}^{\{a\}}$$

(with additional arg of a' to B'_j when $j = 2$). This has the advantage that the test is only done once per message and is part of the interface to the system rather than the working part. However the behaviors B'_j are not going to be equivalent in a general context.

7. Proving Expression Equivalence

In this section we develop three instances of a general method for proving expressions observationally equivalent. The method is a minor generalization of the techniques used to prove the **(ciu)** theorem (a form of context lemma) in [18].

- (1) The first instance of the method treats equivalence of expressions that have a common reduct – i.e. expressions that step in 0 or more steps to the same expression having the same effects (sends, becomes, creation of new actors, initializing new actors). This is called the *common reduct case*.
- (2) The second instance of the method is an elaboration of the first, treating expressions that reduce to lambda abstractions that are application equivalent – i.e. have a common reduct when applied to any value. This is called the *two stage reduction case*.
- (3) The third instance of the method treats equivalence of reduction contexts. This is called the *equivalence of reduction contexts case*.

We provide examples of the use of these techniques by using them to establish the equational laws stated in §4. We note that these methods and proofs can easily be modified to prove interaction equivalence, instead of observational equivalence of expressions. For simplicity we have only treated observational equivalence here.

7.1. The General Method

Each of these three methods is based on the idea of using *configuration templates* to establish a correspondence between the fair computations of configurations containing the entities to be proved equivalent. A configuration template is simply a configuration with holes, i.e. schematic variables, that may be instantiated by various sorts of syntactic entities. Observing contexts correspond to a special case of configuration templates.

The first step then is to choose a class of configuration templates CT such that $e_0 \cong e_1$ if $Obs(ct[e_0]) = Obs(ct[e_1])$ for all templates $ct \in CT$. To establish the equality of observations, it is sufficient to construct, for each $\pi_0 \in \mathcal{F}(ct[e_0])$, a $\pi_1 \in \mathcal{F}(ct[e_1])$ such that $obs(\pi_0) = obs(\pi_1)$ and conversely. We call such a construction a *path correspondence*. The crucial fact concerning configuration templates is that one can compute symbolically with them in the sense that computation is parametric in the holes. Perhaps an appropriate name for this form of computation is *uniform computation or reduction*.

A suitable class of configuration templates is obtained by extending each syntactic class to allow holes and defining appropriate notions of hole filling. Decomposition theorems and schematic reduction rules are then developed. In each of the three methods the only essential difference is the type and number of holes needed:

- (1) For the common reduct case we define templates by adding holes for expressions. We call these holes *expression holes*, and they are denoted by $\circ$.
- (2) For the two stage reduction case we need not only expression holes but also a countable family of holes for lambda abstractions. We call these holes *abstraction holes* and they are denoted by $\triangleright_j$ for $j \in \mathbb{N}$. Note that these holes are filled by values, specifically by lambda abstractions, not simply by expressions.

- (3) For the equivalence of reduction contexts we need an entirely new class of holes. These holes will be for reduction contexts. We call them *reduction context holes*, and denote them by $\diamond$. Note that these holes will be filled by reduction contexts and are *not* to be confused with redex holes. As far as we are aware the introduction of holes that are filled by contexts is completely novel.

For each case, syntactic classes X are annotated with the signs of the sorts of holes they contain: ${}^\circ X$ for expression holes; ${}^{\circ\triangleright} X$ for expression and lambda abstraction holes; and ${}^\diamond X$ for reduction context holes. We prefix the names of these classes by E-, LE-, or R- respectively. Thus E-expressions are expression templates with holes for expressions, ${}^\circ \mathbb{E}$ is the set of E-expressions, and we let ${}^\circ e$ range over ${}^\circ \mathbb{E}$. Similarly for other syntactic classes and hole types.

The idea underlying the construction of a path correspondence to establish equivalence is the same for each of the three cases. It relies on the ability to localize differences in computations as multi-steps, and to use holes to formalize the aspects of computation that are independent of the local differences. Consider the case of proving expressions equivalent using templates with expression holes. We consider fair computation paths starting from an E-configuration with holes filled by one of the expressions, say e_0 . For each such path, π_0 , we show how to obtain a sequence of E-configurations satisfying two conditions. The first is that filling the holes in the sequence of E-configurations with e_0 (and filling in transition labels) yields π_0 . The second is that filling the holes in the sequence of E-configurations with e_1 (and filling in transition labels) yields a fair computation path with the same observation. The other two cases are simple variations on this idea.

One of the keys ideas in uniform computation is to insure that transitions commute with hole filling; except of course when the *hole is touched*, i.e. information about the contents of the hole is required to carry out the step. Consider the schematic redex $\mathbf{app}(\lambda x.\bullet, v)$. We need a notation that allows us to carry out this reduction in such a way that filling the hole and the reducing gives the same result as reducing and then filling the hole. For this purpose we associate with each hole a substitution to be applied when the hole is filled. The domain of the substitution also determines the variables of an expression that are trapped at the hole. This localizes trapping and allows renaming of lambda-variables even in the presence of holes (which is not the case for traditional notions of expression context). A detailed development of this notation can be found in [30, 32]. We use $\circ[\sigma]$ to denote an expression hole with associated substitution σ (which may in turn have expressions holes in its range), a similar notation holds for the other classes of holes: $\triangleright_j[\sigma]$ for abstraction holes, and $\diamond[\sigma]$ for reduction holes.

To simplify definitions of syntactic classes we treat $\mathbf{app}$ on a par with elements of $\mathbb{F}_2$. We use Θ_n for syntactic operations of arity n , and Θ_n^e to indicate the operations of the extended language (i.e. Θ_0 extended to include $\mathbf{event}$). Thus:

Definition ($\Theta_n \Theta_n^e$):

$$\begin{aligned} \Theta_2 &= \mathbb{F}_2 \cup \{\mathbf{app}\} & \Theta_n &= \mathbb{F}_n & \text{for } n \neq 2 \\ \Theta_0^e &= \Theta_0 \cup \{\mathbf{event}\} & \Theta_n^e &= \Theta_n & \text{for } n \neq 0 \end{aligned}$$

7.2. Common Reduct Case

We now treat the common reduct case in depth. The other two cases follow in the same manner and we allow ourselves to be a little more terse.

7.2.1. E-Syntax

As mentioned above, syntactic classes, X , with expression holes are indicated by the mark ${}^\circ X$. Metavariables ranging over these classes are indicated by the same mark, and we prefix the names of these classes by E-. Thus we have E-expressions where ${}^\circ e$ ranges over ${}^\circ \mathbb{E}$, E-configurations where ${}^\circ \kappa$ ranges over ${}^\circ \mathbb{K}$, etc. We first define the E- analogs of expression, value expression, and value substitution.

Definition (${}^\circ \mathbb{E}$ ${}^\circ \mathbb{V}$ ${}^\circ \mathbb{S}$):

$${}^\circ \mathbb{V} = \text{At} \cup \mathbb{X} \cup \lambda \mathbb{X}. {}^\circ \mathbb{E} \cup \mathbf{pr}({}^\circ \mathbb{V}, {}^\circ \mathbb{V})$$

$${}^\circ \mathbb{E} = {}^\circ \mathbb{V} \cup \Theta_n^e({}^\circ \mathbb{E}^n) \cup \circ[{}^\circ \mathbb{S}]$$

$${}^\circ \mathbb{S} = \mathbb{X} \xrightarrow{f} {}^\circ \mathbb{V}$$

λ is the only binding operator, and free variables of E-expressions are defined as follows:

Definition ($\text{FV}({}^\circ e)$ $\text{FV}({}^\circ \sigma)$):

$$\text{FV}({}^\circ e) = \begin{cases} \text{FV}({}^\circ \sigma) & \text{if } {}^\circ e = \circ[{}^\circ \sigma] \\ \{{}^\circ e\} & \text{if } {}^\circ e \in \mathbb{X} \\ \text{FV}({}^\circ e_0) - \{z\} & \text{if } {}^\circ e = \lambda z. {}^\circ e_0 \\ \text{FV}({}^\circ e_0) \cup \dots \cup \text{FV}({}^\circ e_n) & \text{if } {}^\circ e = \theta({}^\circ e_1, \dots, {}^\circ e_n) \text{ and } \theta \in \Theta_n^e \end{cases}$$

$$\text{FV}({}^\circ \sigma) = \bigcup_{x \in \text{Dom}({}^\circ \sigma)} \text{FV}({}^\circ \sigma(x))$$

The variables in the domain of occurrences of ${}^\circ \sigma$ are neither free or bound. In particular, renaming of bound variables only applies to the range of a substitution associated with a hole, not to its domain.

Definition (${}^\circ e[{}^\circ \sigma]$ ${}^\circ \sigma_1 \odot {}^\circ \sigma_2$): Substitution is extended to E-expressions as follows:

$${}^\circ e[{}^\circ \sigma] = \begin{cases} \circ[{}^\circ \sigma \odot {}^\circ \sigma'] & \text{if } {}^\circ e = \circ[{}^\circ \sigma'] \\ {}^\circ e & \text{if } {}^\circ e \in \mathbb{X} - \text{Dom}({}^\circ \sigma) \\ {}^\circ \sigma({}^\circ e) & \text{if } {}^\circ e \in \text{Dom}({}^\circ \sigma) \\ \lambda z. {}^\circ e_0[{}^\circ \sigma](\text{Dom}({}^\circ \sigma) - \{z\}) & \text{if } {}^\circ e = \lambda z. {}^\circ e_0 \text{ and } z \notin \text{FV}({}^\circ \sigma) \\ \theta({}^\circ e_0[{}^\circ \sigma], \dots, {}^\circ e_n[{}^\circ \sigma]) & \text{if } {}^\circ e = \theta({}^\circ e_1, \dots, {}^\circ e_n) \text{ and } \theta \in \Theta_n^e \end{cases}$$

$${}^\circ \sigma_1 \odot {}^\circ \sigma_2 = \lambda x \in \text{Dom}({}^\circ \sigma_2). {}^\circ \sigma_2(x)[{}^\circ \sigma_1]$$

As defined here substitution is a partial operation. Using renaming substitutions we can define α renaming in the usual way. We consider E-expressions (and entities containing them) to be equivalent if they differ only by α renaming. Thus, for any substitution we can always choose an α variant so that substitution is defined.

Expression hole filling is defined by induction on the structure of ${}^\circ e$. Like substitution, we avoid capture of free variables in e by lambda binding. We let ${}^\circ e[\circ := e]$ be the result of filling expression holes in ${}^\circ e$ with e . All capture is done at hole occurrences.

Definition (${}^{\circ}e[\circ := e]$):

$${}^{\circ}e[\circ := e] = \begin{cases} e[\lambda x \in \text{Dom}({}^{\circ}\sigma). {}^{\circ}\sigma(x)[\circ := e]] & \text{if } {}^{\circ}e = \circ[{}^{\circ}\sigma] \\ \theta({}^{\circ}e_1[\circ := e], \dots, {}^{\circ}e_n[\circ := e]) & \text{if } {}^{\circ}e = \theta({}^{\circ}e_1, \dots, {}^{\circ}e_n) \text{ and } \theta \in \Theta_n^e \\ {}^{\circ}v & \text{if } {}^{\circ}e = {}^{\circ}v \in \text{At} \cup \mathbb{X} \\ \lambda x. {}^{\circ}e[\circ := e] & \text{if } {}^{\circ}e = \lambda x. {}^{\circ}e \text{ and } x \text{ not free in } e \end{cases}$$

Lemma (fil-subst): Hole filling and substitution commute.

$${}^{\circ}e[{}^{\circ}\sigma][\circ := e'] = {}^{\circ}e[\circ := e'][{}^{\circ}\sigma[\circ := e']]$$

if $\text{Dom}({}^{\circ}\sigma) \cap \text{FV}(e') = \emptyset$.

Proof : By induction on the structure of ${}^{\circ}e$. We assume the names of bound variables in ${}^{\circ}e$ have been chosen not to conflict with any free variables in e' , or the range of ${}^{\circ}\sigma$, or the domain of ${}^{\circ}\sigma$. As examples, we consider the cases where ${}^{\circ}e$ is a lambda abstraction or a hole. If ${}^{\circ}e = \lambda z. {}^{\circ}e_0$ then

$$\begin{aligned} {}^{\circ}e[{}^{\circ}\sigma][\circ := e'] &= (\lambda z. {}^{\circ}e_0[{}^{\circ}\sigma])[\circ := e'] \\ &= \lambda z. ({}^{\circ}e_0[{}^{\circ}\sigma][\circ := e']) && \text{by hygiene assumptions} \\ &= \lambda z. ({}^{\circ}e_0[\circ := e'][{}^{\circ}\sigma[\circ := e']]) && \text{by the Induction Hypothesis} \\ &= (\lambda z. {}^{\circ}e_0[\circ := e'])[{}^{\circ}\sigma[\circ := e']] && \text{by hygiene assumptions} \\ &= {}^{\circ}e[\circ := e'][{}^{\circ}\sigma[\circ := e']] \end{aligned}$$

If ${}^{\circ}e = \circ[{}^{\circ}\sigma']$ then

$$\begin{aligned} {}^{\circ}e[{}^{\circ}\sigma][\circ := e'] &= \circ[{}^{\circ}\sigma'][{}^{\circ}\sigma][\circ := e'] \\ &= (\circ[\lambda z \in \text{Dom}({}^{\circ}\sigma'). {}^{\circ}\sigma'(z)[{}^{\circ}\sigma]])[\circ := e'] \\ &= e'[\lambda z \in \text{Dom}({}^{\circ}\sigma'). {}^{\circ}\sigma'(z)[{}^{\circ}\sigma][\circ := e']] \\ &= e'[\lambda z \in \text{Dom}({}^{\circ}\sigma'). {}^{\circ}\sigma'(z)[\circ := e']][{}^{\circ}\sigma[\circ := e']] && \text{by the Induction Hypothesis} \\ &= e'[{}^{\circ}\sigma'[\circ := e'] \odot {}^{\circ}\sigma[\circ := e']] \\ &= \circ[{}^{\circ}\sigma'][\circ := e'] [{}^{\circ}\sigma[\circ := e']] && \text{by hygiene assumptions} \\ &= {}^{\circ}e[\circ := e'] [{}^{\circ}\sigma[\circ := e']] \end{aligned}$$

□

Next we define analogs of redex and reduction context.

Definition (${}^{\circ}\mathbb{R}$ ${}^{\circ}\mathbb{E}_{\text{rdx}}$):

$$\begin{aligned} {}^{\circ}\mathbb{R} &= \{\square\} \cup \Theta_{m+n+1}({}^{\circ}\mathbb{V}^m, {}^{\circ}\mathbb{R}, {}^{\circ}\mathbb{E}^n) \\ {}^{\circ}\mathbb{E}_{\text{rdx}} &= \Theta_n^e({}^{\circ}\mathbb{V}^n) \end{aligned}$$

Note that E-reduction contexts possess two types of holes, consequently we must disambiguate the process of hole filling. Note that the unique occurrence of a redex hole is not adorned with a substitution, consequently the process of filling the redex hole, $\square$, with the E-expression, ${}^\circ e$, remains unchanged, and we denote it by ${}^\circ R[\square := {}^\circ e]$.

In the case of multiple hole filling we write ${}^\circ R[\circ := e_0][\square := e]$ for the result of filling the expression holes with e_0 , and the redex hole with e .

Lemma (E-properties):

- (1) ${}^\circ R[\circ := e_0][\square := e] = {}^\circ R[\square := e][\circ := e_0]$
- (2) Filling an E-expression, E-reduction context, or E-redex with an expression yields an expression, reduction context, or redex, respectively.

7.2.2. E-Expression Decomposition

We now give a decomposition lemma for E-expressions: An E-expression ${}^\circ e$ is either an E-value (element of ${}^\circ \mathbb{V}$) or it can be decomposed uniquely into an E-reduction context filled with either an E-redex or with an expression hole.

Lemma (E-expression decomposition):

- (0) ${}^\circ e \in {}^\circ \mathbb{V}$, or
- (1) $(\exists ! {}^\circ R, {}^\circ r)({}^\circ e = {}^\circ R[\square := {}^\circ r])$, or
- (2) $(\exists ! {}^\circ R, {}^\circ \sigma)({}^\circ e = {}^\circ R[\square := \circ[{}^\circ \sigma]])$

Proof : An easy induction on the structure of ${}^\circ e$. We consider two example cases. First, suppose ${}^\circ e = \circ[{}^\circ \sigma]$. Then we have case (2) with ${}^\circ R = \square$. Second, suppose ${}^\circ e = \theta({}^\circ e_1, \dots, {}^\circ e_n)$. If ${}^\circ e_i \in {}^\circ \mathbb{V}$ for $1 \leq i \leq n$, then we have case (1) with ${}^\circ R = \square$ (and ${}^\circ r = {}^\circ e$). If ${}^\circ e_i \notin {}^\circ \mathbb{V}$ for some $1 \leq i \leq n$, assume k to be the least such i . Then by the induction hypothesis, ${}^\circ e_k$ decomposes either as (i) ${}^\circ R'[\square := {}^\circ r]$, or as (ii) ${}^\circ R'[\square := \circ[{}^\circ \sigma]]$. Taking ${}^\circ R = \theta({}^\circ e_1, \dots, {}^\circ e_{k-1}, {}^\circ R', {}^\circ e_{k+1}, \dots, {}^\circ e_n)$ we obtain the desired decomposition of ${}^\circ e$. $\square$

7.2.3. E-Configurations

An E-configuration ${}^\circ \kappa$ is formed like a configuration, using E-expressions and E-values instead of simple expressions and values.

Definition (${}^\circ \mathbb{K}$):

$${}^\circ \mathbb{K} = \left\langle \left\langle {}^\circ \mathbb{A}c \mid {}^\circ \mathbb{M} \right\rangle \right\rangle_x^\rho$$

where

$${}^\circ \mathbb{A}c = \mathbb{A}d \xrightarrow{f} \mathbb{A}s$$

$${}^\circ \mathbb{A}s = (\lambda x. {}^\circ \mathbb{E}) \cup [{}^\circ \mathbb{E}] \cup \{({}^\circ \mathbb{A}d)\}$$

$${}^\circ \mathbb{M} = \langle {}^\circ \mathbb{V} \Leftarrow {}^\circ \mathbb{V} \rangle$$

and the constraints specified in the definition of actor configurations in §2 are satisfied.¹ We let ${}^\circ\kappa$ range over ${}^\circ\mathbb{K}$, and ${}^\circ\alpha$ range over ${}^\circ\text{Ac}$. Filling expression holes of an E-configuration, E-actor map, E-actor state, E-multiset of messages, and E-messages is defined in the obvious manner. Let ${}^\circ X$ stand generically for an element of one of these E-syntactic classes, then we define ${}^\circ X[\circ := e]$ as follows:

Definition (${}^\circ X[\circ := e]$):

$${}^\circ X[\circ := e] = \begin{cases} \left\langle \left\langle {}^\circ\alpha[\circ := e] \mid {}^\circ\mu[\circ := e] \right\rangle \right\rangle_\chi^\rho & \text{if } {}^\circ X = \left\langle \left\langle {}^\circ\alpha \mid {}^\circ\mu \right\rangle \right\rangle_\chi^\rho \\ \lambda x \in \text{Dom}({}^\circ\alpha). {}^\circ\alpha(x)[\circ := e] & \text{if } {}^\circ X = {}^\circ\alpha \\ ((\lambda x. {}^\circ e)[\circ := e]) & \text{if } {}^\circ X = (\lambda x. {}^\circ e) \\ [{}^\circ e[\circ := e]] & \text{if } {}^\circ X = [{}^\circ e] \\ (?_a) & \text{if } {}^\circ X = (?_a) \\ \{ {}^\circ m[\circ := e] \mid {}^\circ m \in {}^\circ\mu \} & \text{if } {}^\circ X = {}^\circ\mu \\ \langle {}^\circ v_0[\circ := e] \Leftarrow {}^\circ v_1[\circ := e] \rangle & \text{if } {}^\circ X = \langle {}^\circ v_0 \Leftarrow {}^\circ v_1 \rangle \end{cases}$$

An E-configuration, ${}^\circ\kappa$, is *closing* for e if ${}^\circ\kappa[\circ := e]$ is a closed configuration. Dually an expression e is a *valid filling* for an E-configuration, ${}^\circ\kappa$, if ${}^\circ\kappa[\circ := e]$ is a configuration. As for atoms and variables, the notion of communicable value remains unchanged and we do not introduce new notation for these. In particular, although messages may have holes, a message with a hole can effectively be ignored. This is because holes in E-values must occur inside λ 's and hence filling these holes cannot yield communicable values or actor addresses. Thus a message with a hole can never be delivered or output. The next lemma expresses the fact that closing E-configurations make just the same observations as simple observing contexts.

Lemma (ocx): $e_0 \cong e_1$ iff $\text{Obs}({}^\circ\kappa[e_0]) = \text{Obs}({}^\circ\kappa[e_1])$ for all ${}^\circ\kappa$ that close e_0, e_1 .

Proof : The backward implication is easy to see, since $\mathbb{O}$ is (with suitable translation to account for trapping at holes rather than at lambdas) a subset of ${}^\circ\mathbb{K}$. The idea for the proof of the forward implication is to define for each configuration context ${}^\circ\kappa$, an observing context O whose computations give rise to the same set of observations. In fact O evolves to ${}^\circ\kappa$ in a finite number of steps. For an E-expression ${}^\circ e$ we define ${}^\circ e^\dagger$ to be the result of recursively replacing decorated holes $\circ[\sigma]$ by applications $\text{app}(\dots \text{app}(\lambda x_1 \dots \lambda x_n. \bullet, {}^\circ\sigma(x_1)^\dagger), \dots, {}^\circ\sigma(x_n)^\dagger)$ where $\{x_1, \dots, x_n\} = \text{Dom}({}^\circ\sigma)$. Let ${}^\circ\kappa = \left\langle \left\langle {}^\circ\alpha \mid {}^\circ\mu \right\rangle \right\rangle$, let $A = [a_i \mid i < n] = \text{Dom}({}^\circ\alpha)$, and define $O = \left\langle \left\langle [e_{\circ\kappa}]_{\hat{a}} \mid \emptyset \right\rangle \right\rangle$ where $\hat{a} \notin A$, and $e_{\circ\kappa}$ is constructed as follows. Let

$$E = \{i < n \mid (\exists {}^\circ e_i)({}^\circ\alpha(a_i) = [{}^\circ e_i])\}, \text{ and let } n_E \text{ be the cardinality of } E.$$

$$I = \{i < n \mid (\exists {}^\circ v_i)({}^\circ\alpha(a_i) = ({}^\circ v_i))\}.$$

$$B_i(a_0, \dots, a_{n-1}) = {}^\circ v_i^\dagger, \quad \text{if } {}^\circ\alpha(a_i) = ({}^\circ v_i).$$

$$B_i(a_0, \dots, a_{n-1}) = \lambda a. \text{seq}(\text{send}(a, 0), {}^\circ e_i^\dagger), \quad \text{if } {}^\circ\alpha(a_i) = [{}^\circ e_i].$$

$$\mu = \{\langle z_j \Leftarrow {}^\circ v_j \rangle \mid j < m\}$$

¹ The only condition whose meaning is altered in this general setting is (2), where the free variables of any holes must be taken into consideration.

Define

$$\begin{aligned}
W_{\circ\kappa} &= \mathbf{rec}(\lambda b. \lambda k. \lambda m. \mathbf{if}(\mathbf{eq}(k, 0), \mathbf{seq}(\mathbf{send}(z_j, {}^\circ v_j^\dagger)_{j < m}), \mathbf{become}(b(k-1)))) \\
e_{\circ\kappa} &= \mathbf{let}\{a_i := \mathbf{newadr}()\}_{i < n} \\
&\quad \mathbf{seq}(\mathbf{initbeh}(a_i, B_i(a_0, \dots, a_{n-1}))_{i \in I \cup E}, \\
&\quad \mathbf{send}(a_i, \hat{a})_{i \in E}, \\
&\quad \mathbf{become}(W_{\circ\kappa}(n_E)))
\end{aligned}$$

Now, we claim that any computation of ${}^\circ\kappa[\circ := e]$ has a corresponding computation (with same observations) of $O[e]$ obtained by accepting all the startup messages, sending and accepting the acks, and completing the computation of the initializing actor (which can then be ignored). Conversely any computation of $O[e]$ has a corresponding computation of ${}^\circ\kappa[\circ := e]$ obtained by ignoring the finite amount of initializing activity. This argument can be made more rigorous by decomposing paths in either system and then replacing hole contents and making local transformation of non-uniform transition steps. This process is explained in more detail in the proof of the theorem (**fun-red-eq**) below. $\square$

7.2.4. E-Reduction

The reduction relations $\stackrel{\lambda}{\mapsto}$ and $\mapsto$ are extended to the generalized domains in the obvious fashion, simply by liberally annotating metavariables with ${}^\circ$'s, modulo the extension of substitution to E-expressions. As examples, we give the (**beta-v**), (**br**), and (**eq**) clauses of $\stackrel{\lambda}{\mapsto}$ and the internal transitions for $\mapsto$ on closed E-configurations.

Definition ($\stackrel{\lambda}{\mapsto}$):

$$\begin{aligned}
(\text{beta-v}) \quad & {}^\circ R[\square := \mathbf{app}(\lambda x. {}^\circ e, {}^\circ v)] \stackrel{\lambda}{\mapsto} {}^\circ R[\square := {}^\circ e[x := {}^\circ v]] \\
(\text{br}) \quad & {}^\circ R[\square := \mathbf{br}({}^\circ v_1, {}^\circ v_2, {}^\circ v)] \stackrel{\lambda}{\mapsto} \begin{cases} {}^\circ R[\square := {}^\circ v_1] & \text{if } {}^\circ v \in {}^\circ\mathbb{V} - \{\mathbf{nil}\} \\ {}^\circ R[\square := {}^\circ v_2] & \text{if } {}^\circ v = \mathbf{nil} \end{cases} \\
(\text{eq}) \quad & {}^\circ R[\square := \mathbf{eq}({}^\circ v_0, {}^\circ v_1)] \stackrel{\lambda}{\mapsto} \begin{cases} {}^\circ R[\square := \mathbf{t}] & \text{if } {}^\circ v_0 = {}^\circ v_1 \in \mathbf{At} \\ {}^\circ R[\square := \mathbf{nil}] & \text{if } {}^\circ v_0, {}^\circ v_1 \in \mathbf{At} \text{ and } {}^\circ v_0 \neq {}^\circ v_1 \end{cases}
\end{aligned}$$

Definition ($\mapsto$):

$$\begin{aligned}
\langle \mathbf{fun} : a \rangle \quad & {}^\circ e \stackrel{\lambda}{\mapsto} {}^\circ e' \Rightarrow \langle\langle {}^\circ\alpha, [{}^\circ e]_a \mid {}^\circ\mu \rangle\rangle \mapsto \langle\langle {}^\circ\alpha, [{}^\circ e']_a \mid {}^\circ\mu \rangle\rangle \\
\langle \mathbf{new} : a, a' \rangle \quad & \langle\langle {}^\circ\alpha, [{}^\circ R[\square := \mathbf{newadr}()]]_a \mid {}^\circ\mu \rangle\rangle \mapsto \langle\langle {}^\circ\alpha, [{}^\circ R[\square := a']]_a, (?_a)_{a'} \mid {}^\circ\mu \rangle\rangle \quad a' \text{ fresh} \\
\langle \mathbf{init} : a, a' \rangle \quad & \langle\langle {}^\circ\alpha, [{}^\circ R[\square := \mathbf{initbeh}(a', {}^\circ v)]]_a, (?_a)_{a'} \mid {}^\circ\mu \rangle\rangle \mapsto \langle\langle {}^\circ\alpha, [{}^\circ R[\square := \mathbf{nil}]]_a, ({}^\circ v)_{a'} \mid {}^\circ\mu \rangle\rangle \\
\langle \mathbf{bec} : a, a' \rangle \quad & \langle\langle {}^\circ\alpha, [{}^\circ R[\square := \mathbf{become}({}^\circ v)]]_a \mid {}^\circ\mu \rangle\rangle \mapsto \langle\langle {}^\circ\alpha, [{}^\circ R[\square := \mathbf{nil}]]_{a'}, ({}^\circ v)_a \mid {}^\circ\mu \rangle\rangle \quad a' \text{ fresh} \\
\langle \mathbf{send} : a, m \rangle \quad & \langle\langle {}^\circ\alpha, [{}^\circ R[\square := \mathbf{send}({}^\circ v_0, {}^\circ v_1)]]_a \mid {}^\circ\mu \rangle\rangle \mapsto \langle\langle {}^\circ\alpha, [{}^\circ R[\square := \mathbf{nil}]]_a \mid {}^\circ\mu, \langle {}^\circ v_0 \Leftarrow {}^\circ v_1 \rangle \rangle\rangle \\
\langle \mathbf{rcv} : a, cv \rangle \quad & \langle\langle {}^\circ\alpha, ({}^\circ v)_a \mid \langle a \Leftarrow cv \rangle, {}^\circ\mu \rangle\rangle \mapsto \langle\langle {}^\circ\alpha, [\mathbf{app}({}^\circ v, cv)]_a \mid {}^\circ\mu \rangle\rangle
\end{aligned}$$

7.2.5. E-Uniform Computation

The notion of E-uniformity of computation is made precise in the following definitions and lemmas. The basic idea is that given a decomposition of a configuration as an E-configuration with holes filled by a given expression, then any transition step leading from that configuration is either independent of what appears in the holes, or it explicitly uses information about the contents of some hole occurrence.

Definition (E-hole touching): Let ${}^\circ\kappa = \langle\langle {}^\circ\alpha \mid {}^\circ\mu \rangle\rangle$. We say that ${}^\circ\kappa$ touches a hole at a if ${}^\circ\alpha(a) = [{}^\circ R[\Box := \circ[{}^\circ\sigma]]]$ for some ${}^\circ R$, ${}^\circ\sigma$.

We say that a transition $\kappa \xrightarrow{l} \kappa'$ touches a hole relative to a decomposition $\kappa = {}^\circ\kappa[\circ := e]$ if l has focus a and ${}^\circ\kappa$ touches a hole at a .

Lemma (E-Uniform Computation):

- (1) If ${}^\circ\kappa \xrightarrow{l} {}^\circ\kappa'$, then ${}^\circ\kappa[\circ := e] \xrightarrow{l} {}^\circ\kappa'[\circ := e]$ for any valid filling expression e .
- (2) If ${}^\circ\kappa$ has no transition with focus a (and a is an actor of ${}^\circ\kappa$), then either ${}^\circ\kappa$ touches a hole at a or ${}^\circ\kappa[\circ := e]$ has no transition with focus a for any valid filling expression e .
- (3) If $\kappa \xrightarrow{l} \kappa'$ and $\kappa = {}^\circ\kappa[\circ := e]$, then either the transition touches a hole or we can find ${}^\circ\kappa'$ such that $\kappa' = {}^\circ\kappa'[\circ := e]$ and ${}^\circ\kappa \xrightarrow{l} {}^\circ\kappa'$.

Proof (1): This is proved by considering cases on the transition rule applied. The only interesting case is (**beta-v**). This follows from (**fl-subst**) $\Box_1$

Proof (2): Assume ${}^\circ\kappa = \langle\langle {}^\circ\alpha \mid {}^\circ\mu \rangle\rangle$ has no transition with focus a , and ${}^\circ\kappa$ does not touch a hole at a . Then one of the following holds:

- (i) ${}^\circ\alpha(a) = (?'_a)$
- (ii) ${}^\circ\alpha(a) = ({}^\circ v)$ and ${}^\circ\mu$ contains no messages deliverable to a
- (iii) ${}^\circ\alpha(a) = [{}^\circ v]$
- (iv) ${}^\circ\alpha(a) = [{}^\circ R[\Box := \mathbf{initbeh}({}^\circ v_0, {}^\circ v_1)]]$ where ${}^\circ v_0$ is not the address of an uninitialized actor created by a
- (v) ${}^\circ\alpha(a) = [{}^\circ R[\Box := {}^\circ R]]$ where ${}^\circ R$ is a non-actor redex that is stuck.

In each of these cases, it easy to see that there will be no transition with focus a enabled when the expressions holes are filled. $\Box_2$

Proof (3): Assume $\kappa = \langle\langle \alpha \mid \mu \rangle\rangle \xrightarrow{l} \kappa' = \langle\langle \alpha' \mid \mu' \rangle\rangle$ and $\kappa = {}^\circ\kappa[\circ := e]$. Thus ${}^\circ\kappa = \langle\langle {}^\circ\alpha \mid {}^\circ\mu \rangle\rangle$ where $\alpha = {}^\circ\alpha[\circ := e]$ and $\mu = {}^\circ\mu[\circ := e]$. We want to find ${}^\circ\alpha'$, ${}^\circ\mu'$ such that ${}^\circ\kappa \xrightarrow{l} {}^\circ\kappa' = \langle\langle {}^\circ\alpha' \mid {}^\circ\mu' \rangle\rangle$, $\alpha' = {}^\circ\alpha'[\circ := e]$, and $\mu' = {}^\circ\mu'[\circ := e]$. Since we are considering closed configurations there are no i/o transitions. Thus, we need to consider only two cases **rcv** transitions and **exec** transitions. We split the **exec** transitions into functional and actor primitives.

Receive: $l = \langle \mathbf{rcv} : a, cv \rangle$, $\langle a \Leftarrow cv \rangle \in \mu$, and $\alpha(a) = (v)$. Thus ${}^\circ\alpha(a) = ({}^\circ v)$ with $v = {}^\circ v[\circ := e]$. Thus we let ${}^\circ\alpha' = {}^\circ\alpha\{a := [\mathbf{app}({}^\circ v, cv)]\}$, and ${}^\circ\mu = {}^\circ\mu' \cup \{\langle a \Leftarrow cv \rangle\}$.

Execution-lambda: $l = \langle \text{fun} : a \rangle$, $\alpha(a) = [R[\Box := r]]$ and $r \xrightarrow{\lambda} e'$. Thus ${}^\circ\alpha(a) = [{}^\circ R[\Box := {}^\circ r]]$ with $R = {}^\circ R[\Box := e]$, and $r = {}^\circ r[\Box := e]$. Thus we want to find ${}^\circ e'$ such that ${}^\circ r \xrightarrow{\lambda} {}^\circ e'$. Then ${}^\circ\alpha' = {}^\circ\alpha\{a := [{}^\circ R[\Box := {}^\circ e']]\}$ and ${}^\circ\mu' = {}^\circ\mu$. If $r = \mathbf{app}(\lambda z.e_0, v)$ (z chosen fresh), then $e' = e_0[z := v]$, and ${}^\circ r = \mathbf{app}(\lambda z.{}^\circ e_0, {}^\circ v)$ where $e_0 = {}^\circ e_0[\Box := e]$ and $v = {}^\circ v[\Box := e]$. Take ${}^\circ e' = {}^\circ e_0[z := {}^\circ v]$ and use (**fil-subst**). If $r = \mathbf{eq}(v_0, v_1)$, then ${}^\circ r = \mathbf{eq}({}^\circ v_0, {}^\circ v_1)$ where $v_j = {}^\circ v_j[\Box := e]$ for $j < 2$. e' is **t** or **nil** and we may take ${}^\circ e' = e'$. The remaining cases are similar.

Execution-actor: If $l = \langle \text{send} : a \rangle$, then $\alpha(a) = [R[\Box := \text{send}(v_0, v_1)]]$ $\alpha'(a) = [R[\Box := \text{nil}]]$, $\mu' = \mu \cup \{\langle v_0 \Leftarrow v_1 \rangle\}$. Also ${}^\circ\alpha(a) = [{}^\circ R[\Box := \text{send}({}^\circ v_0, {}^\circ v_1)]]$ where $R = {}^\circ R[\Box := e]$, and $v_j = {}^\circ v_j[\Box := e]$ for $j < 2$. Take ${}^\circ\alpha' = {}^\circ\alpha\{a := [{}^\circ R[\Box := \text{nil}]]\}$ and ${}^\circ\mu' = {}^\circ\mu \cup \{\langle {}^\circ v_0 \Leftarrow {}^\circ v_1 \rangle\}$.

If $l = \langle \text{become} : a, a' \rangle$, then a' is fresh, $\alpha(a) = [R[\Box := \text{become}(v)]]$ $\alpha'(a) = (v)$, $\alpha'(a') = [R[\Box := \text{nil}]]$, and $\mu' = \mu$. Also ${}^\circ\alpha(a) = [{}^\circ R[\Box := \text{become}({}^\circ v)]]$ where $R = {}^\circ R[\Box := e]$, and $v = {}^\circ v[\Box := e]$. Take ${}^\circ\alpha' = {}^\circ\alpha\{a := ({}^\circ v), a' := [{}^\circ R[\Box := \text{nil}]]\}$ and ${}^\circ\mu' = {}^\circ\mu$.

If $l = \langle \text{new} : a, a' \rangle$, then a' is fresh, $\alpha(a) = [R[\Box := \text{newadr}()]]$ $\alpha'(a) = [R[\Box := \text{nil}]]$, $\alpha'(a') = (?_a)$, and $\mu' = \mu$. Also ${}^\circ\alpha(a) = [{}^\circ R[\Box := \text{newadr}()]]$ where $R = {}^\circ R[\Box := e]$. Take ${}^\circ\alpha' = {}^\circ\alpha\{a := [{}^\circ R[\Box := \text{nil}]], a' := (?_a)\}$ and ${}^\circ\mu' = {}^\circ\mu$.

If $l = \langle \text{init} : a, a' \rangle$, then $\alpha(a) = [R[\Box := \text{initbeh}(a', v)]]$, $\alpha(a') = (?_a)$, $\alpha'(a) = [R[\Box := \text{nil}]]$, $\alpha'(a') = (v)$, and $\mu' = \mu$. Also ${}^\circ\alpha(a) = [{}^\circ R[\Box := \text{initbeh}(a', {}^\circ v)]]$, where $R = {}^\circ R[\Box := e]$, and $v = {}^\circ v[\Box := e]$, and ${}^\circ\alpha(a') = (?_a)$. Take ${}^\circ\alpha' = {}^\circ\alpha\{a := [{}^\circ R[\Box := \text{nil}]], a' := ({}^\circ v)\}$ and ${}^\circ\mu' = {}^\circ\mu$.

□₃ □_E–uniform

7.2.6. The (fun-red-eq) Theorem

Now we have enough notation to describe the construction of path correspondences for expressions with uniform common reducts. We first consider the case of expressions that reduce via purely functional reductions. Then we show how this construction can be modified to allow for reduction of actor primitives.

Theorem (fun-red-eq): If for each ${}^\circ\sigma$ with domain containing the free variables of e_0, e_1 , either $e_j[{}^\circ\sigma]$ hangs for $j < 2$, or there is some ${}^\circ e_c$ such that $e_j[{}^\circ\sigma]$ reduces in 0 or more $\xrightarrow{\lambda}$ steps to ${}^\circ e_c$ uniformly, then $e_0 \cong e_1$.

Corollary (fun-red-eq): The following laws are instances of (**fun-red-eq**): (**red-exp**), (**beta-v**), (**ift**), (**ifn**), (**ifelim**), (**isprt**), (**isprn**), (**fst**), and (**snd**).

Proof: Assume that for each closing ${}^\circ\sigma$ there is ${}^\circ e_{c,j}$ such that $e_j[{}^\circ\sigma] \xrightarrow{\lambda} \dots \xrightarrow{\lambda} {}^\circ e_{c,j}$ $j < 2$, uniformly, and either ${}^\circ e_{c,j}$ is (uniformly) stuck for $j < 2$, or ${}^\circ e_{c,0} = {}^\circ e_{c,1}$. In either case we call ${}^\circ e_{c,j}$ the common reduct. We want to show that $e_0 \cong e_1$. By (**ocx**) it is sufficient to show that $\text{Obs}({}^\circ\kappa[\Box := e_0]) = \text{Obs}({}^\circ\kappa[\Box := e_1])$ for any ${}^\circ\kappa$ that is a closing E-configuration for e_0 and e_1 . To do this, we show that for any $\pi_0 \in \mathcal{F}({}^\circ\kappa[\Box := e_0]) = [\kappa_i \xrightarrow{h} \kappa_{i+1} \mid i \in \mathbb{N}]$ we can find $\pi_1 \in \mathcal{F}({}^\circ\kappa[\Box := e_1])$ such that $\text{obs}(\pi_0) = \text{obs}(\pi_1)$. (The case with 0 and 1 interchanged is symmetric.)

Informally, by the uniformity property of computations, we see that replacing occurrences of e_0 by e_1 has no effect on a computation except where a hole is touched. Using (**infinite macro-steps**) we can localize non-uniform steps so that when a hole is touched,

reduction to a common reduct occurs in a single multi-step (which involves no event transitions). Thus we may obtain a computation for ${}^{\circ}\kappa[\circ := e_1]$ by replacing occurrences of e_0 by e_1 and replacing multi-step transitions reducing $e_0[\circ\sigma]$ to its common reduct by multi-step transitions reducing $e_1[\circ\sigma]$ to its common reduct. To insure completeness/fairness of the result, we need to take account of the case where a hole $\circ[\circ\sigma]$ is exposed, but the multi-step for $e_0[\circ\sigma]$ is trivial and hence does not appear as a transition. We do this by inserting the corresponding multi-step for $e_1[\circ\sigma]$ at the point where the hole is first exposed. Such holes then effectively disappear, since they are either filled with a stuck expression or with the same expression. Below we make this informal argument more rigorous, by the following steps:

- (1) We analyze the configurations occurring in π_0 and record occurrences of e_0 in holes descending from ${}^{\circ}\kappa$. This gives us decompositions ${}^{\circ}\kappa_i[\circ := e_0]$ of κ_i . In the cases where a hole is touched such that e_0 is its common reduct, we fill that hole with e_0 giving a new E-configuration ${}^{\circ}\kappa'_i$ with one less hole, such that ${}^{\circ}\kappa'_i[\circ := e_0]$ is κ_i . This process of filling holes with common reducts continues until the transition l_i is either uniform or touches a hole in which e_0 is not its common reduct. We also record subsequences of transitions corresponding to uniform reduction of such occurrences of e_0 to its common reduct.
- (2) Using (**infinite-macro-steps**) we may assume that the path is expressed in terms of multi-step transitions such that the recorded subsequences of transitions corresponding to non-trivial reduction to a common reduct are single multi-steps. We also insert copies of κ_i for each hole that is filled with a common reduct, remembering the corresponding decomposition, and insert empty multi-steps between these copies. We also insert a copy of κ_i and a connecting empty multi-step for each hole that occurs in a reduction context that is not touched – because the occurrence of e_0 is stuck, or because it is a value and placing it in the redex hole produces either a value or a stuck state.
- (3) Form π_1 by filling the holes of ${}^{\circ}\kappa_i$ (and remaining holes in additional copies) with e_1 and replacing multi-steps for e_0 by corresponding multi-steps for e_1 (empty multi-steps may expand to non-trivial reductions of occurrences of e_1 to its common reduct).

It is easy to see that π_1 is a computation path. The argument that it is complete and fair relies on the insertion of multi-steps, and uses the same case analysis that was used in the uniform computation lemma.

Step (1) We analyze and decompose π_0 to obtain

- (i) for each $i < \infty$ a series of decompositions ${}^{\circ}\kappa_{i,j}$ for $j \leq n_i$ such that $\kappa_i = {}^{\circ}\kappa_{i,j}[\circ := e_0]$ for $j \leq n_i$ and such that ${}^{\circ}\kappa_{i,n_i} \xrightarrow{l_i} {}^{\circ}\kappa_{i+1,0}$ uniformly, or the transition touches a hole in which e_0 has non-trivial reduction to its common reduct. (We call this entering the hole.) n_i will be 0 except in the case of a hole touched in which e_0 is its own common reduct. Then we fill that hole with the common reduct and redecompose.
- (ii) The set I of indices of transitions that enter holes
- (iii) The map J from I to the sequence of indices of transitions corresponding to the thread of computation that carries out the reduction to the common reduct.

This is done incrementally by defining sequences I_n, J_n by induction on n and taking $I = \bigcup_{n < \infty} I_n$, and $J = \bigcup_{n < \infty} J_n$. At stage (i, j) we have defined I_i, J_i , and ${}^{\circ}\kappa_{i,j}$. If $j = n_i$, then the next stage is $(i + 1, 0)$ otherwise it is $(i, j + 1)$.

Stage (0, 0) $I_0 = \emptyset$, and J_0 is the empty map. ${}^\circ\kappa_{0,0} = {}^\circ\kappa$.

At stage (i, j) There are four cases to consider:

- (1) l_i is execution in a hole;
- (2) l_i is uniform wrt ${}^\circ\kappa_{i,j}$
- (3) l_i touches a hole and
 - (3.1) enters the hole
 - (3.2) does not enter the hole

Case 1: This case occurs if i is an element of $M = J_i(m)$ for some $m \in I_i$. Thus $n_i = j$ and $l_i = \langle \text{fun} : a \rangle$ for some a . We move to stage $(i+1, 0)$ with $I_{i+1} = I_i$, $J_{i+1} = J_i$, ${}^\circ\mu_{i+1,0} = {}^\circ\mu_{i,j}$, and ${}^\circ\alpha_{i+1,0} = {}^\circ\alpha_{i,j}\{a := [{}^\circ R[\Box := {}^\circ e_{m,k+1}]]\}$ where ${}^\circ R$, and ${}^\circ e_{m,k+1}$ are obtained as follows. Let k be the index of i in M . The hole is entered at stage (m, n_m) with ${}^\circ\alpha_{m,n_m}(a) = [{}^\circ R[\Box := \circ[{}^\circ\sigma]]]$. Let ${}^\circ e_{m,0} = e_0[{}^\circ\sigma]$, let n be the length of M , and let $[{}^\circ e_{m,k} \xrightarrow{\lambda} {}^\circ e_{m,k+1} \mid k < n]$ be the thread of computation reducing $e_0[{}^\circ\sigma]$ to its common reduct ${}^\circ e_{m,n}$. Note that ${}^\circ\alpha_{i,j}(a) = [{}^\circ R[\Box := {}^\circ e_{m,k}]]$.

Case 2: $n_i = j$ and we move to stage $(i+1, 0)$ with $I_{i+1} = I_i$, $J_{i+1} = J_i$, ${}^\circ\kappa_{i+1,0}$ such that ${}^\circ\kappa_{i,j} \xrightarrow{l_i} {}^\circ\kappa_{i+1,0}$ uniformly according to the uniformity lemma.

Case 3.1: In this case $l_i = \langle \text{fun} : a \rangle$ for some a . $n_i = j$ and we move to stage $(i+1, 0)$ with $I_{i+1} = I_i \cup \{i\}$, $J_{i+1} = J_i\{i := M\}$, ${}^\circ\mu_{i+1} = {}^\circ\mu_i$, and ${}^\circ\alpha_{i+1} = {}^\circ\alpha_i\{a := [{}^\circ R[\Box := {}^\circ e_{i,1}]]\}$ where M , ${}^\circ R$, and ${}^\circ e_{i,1}$ are obtained as follows. Let ${}^\circ\alpha_{i,j}(a) = [{}^\circ R[\Box := \circ[{}^\circ\sigma]]]$, let ${}^\circ e_{i,0} = e_0[{}^\circ\sigma]$, and let $[{}^\circ e_{i,k}[{}^\circ\sigma] \xrightarrow{\lambda} {}^\circ e_{i,k+1} \mid k < n+1]$ be the thread of computation reducing $e_0[{}^\circ\sigma]$ to its common reduct ${}^\circ e_{i,n+1}$. By fairness, there is a sequence of indices $M = [i_k \mid k < n+1]$ with $i_0 = i$, $i_k < i_{k+1}$ for $k < n+1$ such that M is the macro step corresponding to the above lambda reduction.

Case 3.2: We move to stage $(i, j+1)$ with $I_{i+1} = I_i$, $J_{i+1} = J_i$, ${}^\circ\mu_{i,j+1} = {}^\circ\mu_{i,j}$, and ${}^\circ\alpha_{i,j+1} = {}^\circ\alpha_{i,j}\{a := [{}^\circ R[\Box := e_0[{}^\circ\sigma]]]\}$ where a , ${}^\circ R$, ${}^\circ\sigma$ are obtained as follows. a is the focus of l_i , and ${}^\circ\alpha_{i,j}(a) = [{}^\circ R[\Box := \circ[{}^\circ\sigma]]]$, with $e_0[{}^\circ\sigma]$ equal to its common reduct.

Step (2) The family $J(i)$ for $i \in I$ satisfies the conditions of **(infinite-macro-step)**. Hence we may assume that π_0 has the form

$$[{}^\circ\kappa_{i,0}[\circ := e_0] \xrightarrow{[]} \dots \xrightarrow{[]} {}^\circ\kappa_{i,n_i} \xrightarrow{L_i} {}^\circ\kappa_{i+1,0}[\circ := e_0]] \mid i < \bowtie$$

where each L_i is either a single (uniform) transition, or a multi-step reduction of an occurrence of e_0 to its common reduct, and the E-configurations obtained by the above decomposition method. For each i , if ${}^\circ\alpha_{i,0}(a) = [{}^\circ R[\Box := \circ[{}^\circ\sigma]]]$, and there are no transitions L_j for $i \leq j$ with focus a , and i is the least such index, we insert before each transition leading from ${}^\circ\alpha_{i,0}$ an empty transition ${}^\circ\kappa_{i,0}[\circ := e_0] \xrightarrow{[]} {}^\circ\kappa_{i,0}[\circ := e_0]$.

Step (3) We let π_1 be the path

$$[{}^\circ\kappa_{i,0}[\circ := e_1] \xrightarrow{L_{i,0}} \dots \xrightarrow{L_{i,n_i}^{-1}} {}^\circ\kappa_{i,n_i}[\circ := e_1] \xrightarrow{L_i^1} {}^\circ\kappa_{i+1,0}[\circ := e_1]] \mid i < \bowtie$$

where $L_i^!$ is L_i if L_i is a single (uniform) transition; $L_i^!$ is the corresponding macro-step reduction of the occurrence of e_1 to its common reduct, if L_i is a macro-step or an empty transition.

Clearly π_1 is a complete computation path. Also the transitions are the same except for points where holes are touched, but these differences are not observable. Thus $obs(\pi_0) = obs(\pi_1)$.

It remains to check that fairness has been preserved. Suppose some transition l is enabled at stage i in π_1 . We have three cases:

Receive: Suppose l is receipt of $\langle a \Leftarrow cv \rangle$. Then ${}^\circ\alpha_i(a) = (\lambda x. {}^\circ e)$ and hence l is enabled in π_0 at stage i . If l fires in π_0 at stage $i' \geq i$, then it also fires at this stage in π_1 . Suppose l never fires in π_0 . Then by fairness, there is some $i' > i$ such that ${}^\circ\alpha_{i'}(a)$ is an executing state for $j \geq i'$. By construction l is permanently disabled at i' in π_1 as well.

Uniform Execution: Suppose l is an execution step with focus a where ${}^\circ\alpha_i(a) = [{}^\circ R[\Box := {}^\circ r]]$. Then l is enabled in π_0 at i , it can not be disabled, and must occur in π_0 at some stage $i' \geq i$ and hence will occur in π_1 at that stage.

Hole Touching: Suppose l is an execution step by a with ${}^\circ\alpha_i(a) = [{}^\circ e]$ where ${}^\circ e = {}^\circ R[\Box := {}^\circ \sigma]$. First assume $e_1[{}^\circ \sigma]$ reduces. If ${}^\circ e[\Box := e_0]$ does not reduce, then by construction, the transition is taken in π_1 as soon as it is enabled. If ${}^\circ e[\Box := e_0]$ reduces, then a transition will eventually be taken at a in π_0 , and the l will be taken at the corresponding point in π_1 . Suppose $e_1[{}^\circ \sigma]$ does not reduce. Then it must be a value, hence the common reduct. Hence the reduction of e_0 is enabled in π_0 and will eventually be taken. ${}^\circ R$ has the form ${}^\circ R_0[\Box := \theta({}^\circ v^m, {}^\circ \sigma, {}^\circ e^n)]$. If all the E-expressions ${}^\circ e^n$ are E-values, then l must be reduction of the redex in ${}^\circ R_0$ and this is also enabled now, in π_0 . Otherwise consider decomposition of the first non-value element of ${}^\circ e^n$ and repeat this argument. Since we are now looking at a smaller E-expression, we eventually reach the point where the step enabled in π_1 corresponds to one in π_0 and hence will occur eventually. $\square$

7.2.7. The Proof of the (hang-infin) Theorem

We now prove (hang-infin), which says that any two expressions that hang or have infinite computations are observationally equivalent.

Proof (hang-infin): Assume $e_0, e_1 \in \text{Hang} \cup \text{Infin}$. We want to show that $e_0 \cong e_1$. Let ${}^\circ\kappa = \langle \langle {}^\circ\alpha \mid {}^\circ\mu \rangle \rangle$ be a closing E-configuration for e_0 and e_1 . Assume $\pi_0 \in \mathcal{F}({}^\circ\kappa[\Box := e_0]) = [\kappa_i \xrightarrow{l_i} \kappa_{i+1} \mid i \in \mathbb{N}]$. We want to find ${}^\circ\alpha_i$, and L_i , such that $\kappa_i = {}^\circ\kappa_i[\Box := e_0]$ where ${}^\circ\kappa_i = \langle \langle {}^\circ\alpha_i \mid {}^\circ\mu_i \rangle \rangle$ and, letting $\pi_1 = [{}^\circ\kappa_i[\Box := e_1] \xrightarrow{L_i} {}^\circ\kappa_{i+1}[\Box := e_1] \mid i \in \mathbb{N}]$, we have $\pi_1 \in \mathcal{F}({}^\circ\kappa[\Box := e_1])$ and $obs(\pi_0) = obs(\pi_1)$. (Actually, we let holes in π_j be filled by any expression of the same class as e_j .) For the base case we have ${}^\circ\alpha_0 = {}^\circ\alpha$. Assume we have ${}^\circ\alpha_i$. Suppose $e_0 \in \text{Hang}$. Let a be the focus of l_i . We first consider each a' other than a such that ${}^\circ\alpha_i(a') = {}^\circ R[\Box := {}^\circ \sigma]$. If $e_1 \in \text{Hang}$ then we just insert any steps needed to reach the stuck state (we assume that they are already macroized for π_0). If $e_1 \in \text{Infin}$, then insert the step to reach the next element of its infinite sequence. Now we consider the transition label l_i . If it does not touch a hole, then ${}^\circ\alpha_{i+1}$ is given by the uniform transition lemma. Suppose ${}^\circ\alpha_i(a) = {}^\circ R[\Box := {}^\circ \sigma]$. Then ${}^\circ\alpha_{i+1}$ has the same decomposition, just possible different expressions (of the same class) filling the holes. $\square_{h-i}$

7.2.8. The Proofs of the Actor Primitive Laws

We show how to modify the construction for the purely functional case to establish equivalence where reductions may involve actor primitives. The notion of common reduct is a fragment of an E-configuration which must be merged with the parent configuration. In the equivalences considered below, with one exception, no extra steps need to be inserted since in every instance either both expressions step, or both hang. Thus we only need to construct $\circ\alpha_{i+1}$ for the hole touching case. Fairness follows easily using the same argument as for the functional case. The exception is the commuting of **newadr** with **initbeh**, which will be treated in more detail below.

Proof (triv): Suppose e_0, e_1 are taken from the pair $\vartheta(\bar{x}), \text{seq}(\vartheta(\bar{x}), \text{nil})$ where $\vartheta \in \{\text{send}, \text{become}, \text{initbeh}\}$. Suppose l_i is an execution by a with $\circ\alpha_i(a) = [\circ R[\Box := \circ[\circ\sigma]]]$. We define $\circ\alpha_{i+1}$ for each form $\vartheta(\bar{x})$.

(**send**(x_0, x_1)) $\circ\alpha_{i+1} = \circ\alpha_i\{a := [\circ R[\Box := \text{nil}]]\}$. Also, $\circ\mu_{i+1} = \circ\mu_i \cup \{<\circ v_0 \Leftarrow \circ v_1>\}$, where $\circ v_j = x_j[\circ\sigma]$. (Note that any holes appearing in messages must be inside lambdas and hence the message is ill-formed and can be ignored.)

(**become**(x_0)) $\circ\alpha_{i+1} = \circ\alpha_i\{a := (\circ\sigma(x_0)), a' := [\circ R[\Box := \text{nil}]]\}$ where a' is a new (anonymous) actor identifier.

(**initbeh**(x_0, x_1)) $\circ\alpha_{i+1} = \circ\alpha_i\{a := [\circ R[\Box := \text{nil}]], \circ\sigma(x_0) := (\circ\sigma(x_1))\}$. Since the step occurs in π_0 , we may assume $a' = \circ\sigma(x_0) \in \text{Dom}(\circ\alpha_i)$, and $\circ\alpha_i(a') = (?_a)$.

□_{triv}

Proof (can-b): Suppose e_0, e_1 are taken from the pair

$$\text{seq}(\text{become}(v_0), \text{become}(v_1)), \quad \text{seq}(\text{become}(v_0), \text{nil}).$$

Assume l_i is an execution by a with $\circ\alpha_i(a) = [\circ R[\Box := \circ[\circ\sigma]]]$. Define

$$\circ\alpha_{i+1} = \circ\kappa_i\{a := (v_0[\circ\sigma]), a' := [\circ R[\Box := \text{nil}]]\}$$

where a' is a new anonymous actor identifier. We ignore the additional anonymous actor with state $(v_1[\circ\sigma])$ in the case of two becomes, since it is carried along unchanged in the remainder of the computation. □

Proof (can-i): Suppose e_0, e_1 are taken from the pair

$$\text{seq}(\text{initbeh}(v, v_0), \text{initbeh}(v, v_1)), \quad \text{seq}(\text{initbeh}(v, v_0), \text{stuck}).$$

(The equivalence with **stuck** replaced by **bot** follows from (**hang-infin**).) Assume l_i is an execution by a with $\circ\alpha_i(a) = [\circ R[\Box := \circ[\circ\sigma]]]$. Define

$$\circ\alpha_{i+1} = \circ\alpha_i\{a := [\circ R[\Box := \text{bot}]], \circ\sigma(v) := (\circ\sigma(v_0))\}$$

Since the step occurs in π_0 , we may assume $a' = \circ\sigma(v) \in \text{Dom}(\circ\alpha_i)$, and $\circ\alpha_i(a') = (?_a)$. Officially, in π_1 $\circ\alpha_{i+1}(a)$ should be $[\circ R[\Box := \text{initbeh}(v, v_1)[\circ\sigma]]]$. But, as in the proof of (**hang-infin**) we treat stuck expressions as indistinguishable. □

Proof (commutes): Suppose e_0, e_1 are taken from one of the pairs in the commutes lemma, except (**n-i**). Assume l_i is an execution by a with $\circ\alpha_i(a) = [\circ R[\Box := \circ[\circ\sigma]]]$.

(s-s)

$\text{let}\{x_0 := \text{send}(v_0, v_1)\}\text{let}\{x_1 := \text{send}(v_2, v_3)\}e$
 $\text{let}\{x_1 := \text{send}(v_2, v_3)\}\text{let}\{x_0 := \text{send}(v_0, v_1)\}e$

Define

${}^\circ\alpha_{i+1} = {}^\circ\alpha_i\{adr := [{}^\circ R[\Box := e\{x_0 := \text{nil}, x_1 := \text{nil}\}][{}^\circ\sigma]]\}$
 ${}^\circ v_j = v_j[{}^\circ\sigma] \quad \text{for } j < 4$
 ${}^\circ\mu_{i+1} = {}^\circ\mu_i \cup \{<{}^\circ v_0 \Leftarrow {}^\circ v_1>, <{}^\circ v_2 \Leftarrow {}^\circ v_3>\}$

$\Box_{s-s}$

(s-n)

$\text{let}\{x_0 := \text{send}(v_0, v_1)\}\text{let}\{x_1 := \text{newadr}()\}e$
 $\text{let}\{x_1 := \text{newadr}()\}\text{let}\{x_0 := \text{send}(v_0, v_1)\}e$

Let a_1 be fresh and define

${}^\circ\alpha_{i+1} = {}^\circ\alpha_i\{a := [{}^\circ R[\Box := e\{x_0 := \text{nil}, x_1 := a_1\}][{}^\circ\sigma]]a_1 := (?_a)\}$
 ${}^\circ v_j = v_j[{}^\circ\sigma] \quad \text{for } j < 2$
 ${}^\circ\mu_{i+1} = {}^\circ\mu_i \cup \{<{}^\circ v_0 \Leftarrow {}^\circ v_1>\}$

$\Box_{s-n}$

(n-n)

$\text{let}\{x_0 := \text{newadr}()\}\text{let}\{x_1 := \text{newadr}()\}e$
 $\text{let}\{x_1 := \text{newadr}()\}\text{let}\{x_0 := \text{newadr}()\}e$

Let a_0, a_1 be fresh and define

${}^\circ\alpha_{i+1} = {}^\circ\alpha_i\{a := [{}^\circ R[\Box := e\{x_0 := a_0, x_1 := a_1\}][{}^\circ\sigma]]a_0 := (?_a)a_1 := (?_a)\}$

$\Box_{n-n}$

(n-i) To complete the proof, we treat the case (n-i). Here e_0, e_1 are taken from the pair

$\text{let}\{x_0 := \text{newadr}()\}\text{let}\{x_1 := \text{initbeh}(v, v_1)\}e$
 $\text{let}\{x_1 := \text{initbeh}(v, v_1)\}\text{let}\{x_0 := \text{newadr}()\}e$

We first consider possible dangling transitions. This can only happen if e_0 does the initialization first. Suppose ${}^\circ\alpha_i(a) = [{}^\circ R[\Box := {}^\circ[{}^\circ\sigma]]]$, such that when $\text{initbeh}(v, v_1)$ is placed in the expression hole no transition is enabled. Then we insert the **newadr** transition in L_i and remove the hole, leaving some stuck expression in its place. As before we do not distinguish between different stuck expressions placed in the hole. Also, in π_1 there will be an extra uninitialized actor carried along untouched for each such insertion. These can also be ignored. Now we consider the transition l_i . Assume its focus is a and ${}^\circ\alpha_i(a) = [{}^\circ R[\Box := {}^\circ[{}^\circ\sigma]]]$. Since we have removed holes with stuck initializations, we may assume that $\text{initbeh}(v, v_1)$ at this point. Let a_0 be fresh and define

${}^\circ\alpha_{i+1} = {}^\circ\alpha_i\{a := [{}^\circ R[\Box := e\{x_0 := a_0, x_1 := \text{nil}\}][{}^\circ\sigma]]a_0 := (?_a)a_1 := (v_1[{}^\circ\sigma])\}$

$\Box_{n-i}$

$\Box_{\text{commutes}}$

7.3. Equivalence by Two Stage Reduction

There is one remaining equivalence to establish using common reducts:

$$(\mathbf{if.lam}) \quad \lambda x. \mathbf{if}(v, e_1, e_2) \cong \mathbf{if}(v, \lambda x. e_1, \lambda x. e_2) \quad x \notin \mathbf{FV}(v)$$

The intuitive reasoning behind this equivalence is that for any closing substitution (allowing holes, and actor addresses in the range) the two expressions reduce to equivalent lambda expressions. In fact these lambda expressions have the property that when applied to any argument they reduce to a common expression.

The method developed so far requires reduction to a common local configuration in one stage. Thus we must elaborate the notion of a template to provide for two stages. Specifically, we add a family of holes for lambda-abstractions, which we denote by $\triangleright_j$ for $j \in J$ for some $J \in \mathbb{N} \cup \{\omega\}$.

7.3.1. LE-Syntax

Syntactic classes X with both expression and lambda holes are indicated by the mark ${}^{\circ\triangleright}X$, and we prefix the names of these classes by LE-, thus we have LE-expressions, LE-configurations, etc. The defining clauses are as before with two exceptions: lambda holes are added to the clause generating values; and $\mathbf{app}(\triangleright_j[{}^{\circ\triangleright}\sigma], {}^{\circ\triangleright}v)$ is omitted from the class of LE-redexes. The latter exception is made in order to preserve the property that redexes reduce uniformly.

Definition (${}^{\circ\triangleright}\mathbb{V}, {}^{\circ\triangleright}\mathbb{E}, {}^{\circ\triangleright}\mathbb{S}, {}^{\circ\triangleright}\mathbb{R}, {}^{\circ\triangleright}\mathbb{E}_{\text{rdx}}$):

$${}^{\circ\triangleright}\mathbb{V} = \mathbf{At} \cup \mathbf{X} \cup \lambda \mathbf{X}. {}^{\circ\triangleright}\mathbb{E} \cup \mathbf{pr}({}^{\circ\triangleright}\mathbb{V}, {}^{\circ\triangleright}\mathbb{V}) \cup \triangleright_{\mathbb{N}}[{}^{\circ\triangleright}\mathbb{S}]$$

$${}^{\circ\triangleright}\mathbb{E} = {}^{\circ\triangleright}\mathbb{V} \cup \Theta_n^e({}^{\circ\triangleright}\mathbb{E}^n) \cup \{\circ[{}^{\circ\triangleright}\mathbb{S}]\}$$

$${}^{\circ\triangleright}\mathbb{S} = \mathbf{X} \xrightarrow{f} {}^{\circ\triangleright}\mathbb{V}$$

$${}^{\circ\triangleright}\mathbb{R} = \{\square\} \cup \Theta_{m+n+1}({}^{\circ\triangleright}\mathbb{V}^m, {}^{\circ\triangleright}\mathbb{R}, {}^{\circ\triangleright}\mathbb{E}^n)$$

$${}^{\circ\triangleright}\mathbb{E}_{\text{rdx}} = \Theta_n^e({}^{\circ\triangleright}\mathbb{V}^n) - \mathbf{app}(\triangleright_j[{}^{\circ\triangleright}\mathbb{S}], {}^{\circ\triangleright}\mathbb{V})$$

Note that lambda holes can occur in the range of a value substitution, and as arguments in redices, except in the function position of an application. Using the double index convention, we write ${}^{\circ\triangleright}e[\triangleright_j := \varphi_j]$ to indicate the simultaneous filling of the holes $\triangleright_j$ with the corresponding lambdas φ_j from some previously specified family $\{\varphi_j\}_{j \in J}$ of lambda abstractions. The definitions of substitution, free variables, and hole filling are entirely analogous to the common reduct case and we omit them.

The decomposition lemma is modified as follows. An LE-expression ${}^{\circ\triangleright}e$ is either an LE-value expression (element of ${}^{\circ\triangleright}\mathbb{V}$), or it can be decomposed uniquely into an LE-reduction context with redex hole filled with either an LE-redex, an LE-expression hole, or an application of a lambda hole (to an LE-value).

Lemma (LE-expression decomposition):

- (0) ${}^{\circ\triangleright}e \in {}^{\circ\triangleright}\mathbb{V}$, or
- (1) $(\exists! {}^{\circ\triangleright}R, {}^{\circ\triangleright}r)({}^{\circ\triangleright}e = {}^{\circ\triangleright}R[\square := {}^{\circ\triangleright}r])$, or
- (2) $(\exists! {}^{\circ\triangleright}R, {}^{\circ\triangleright}\sigma)({}^{\circ\triangleright}e = {}^{\circ\triangleright}R[\square := \circ[{}^{\circ\triangleright}\sigma]])$, or
- (3) $(\exists! {}^{\circ\triangleright}R, {}^{\circ\triangleright}\sigma, {}^{\circ\triangleright}v)({}^{\circ\triangleright}e = {}^{\circ\triangleright}R[\square := \mathbf{app}(\triangleright_j[{}^{\circ\triangleright}\sigma], {}^{\circ\triangleright}v)])$

7.3.2. LE-computation

The definition of LE-configurations and LE-reduction are the natural extensions of E-configurations and E-reduction to the situation with lambda abstraction holes added. The definition of hole touching and the uniform computation lemmas generalize easily to this situation.

Definition (${}^{\circ}\mathbb{K}$):

$${}^{\circ}\mathbb{K} = \left\langle\left\langle {}^{\circ}\text{Ac} \mid {}^{\circ}\mathbb{M} \right\rangle\right\rangle_{\chi}^{\rho}$$

where

$${}^{\circ}\text{Ac} = \text{Ad} \xrightarrow{f} {}^{\circ}\text{As}$$

$${}^{\circ}\text{As} = (\lambda x. {}^{\circ}\mathbb{E}) \cup [{}^{\circ}\mathbb{E}] \cup \{(?_{\text{Ad}})\}$$

$${}^{\circ}\mathbb{M} = \langle {}^{\circ}\mathbb{V} \Leftarrow {}^{\circ}\mathbb{V} \rangle$$

and the constraints specified in the definition of actor configurations in §2. are satisfied. We let ${}^{\circ}\kappa$ range over ${}^{\circ}\mathbb{K}$, and ${}^{\circ}\alpha$ range over ${}^{\circ}\text{Ac}$. Filling expression and abstraction holes of an LE-configuration, LE-actor map, LE-actor state, LE-multiset of messages, and LE-messages is defined in the obvious manner.

An LE-configuration, ${}^{\circ}\kappa$, is closing for e and a family $\{\varphi_j\}_{j \in J}$ of lambda abstractions if $({}^{\circ}\kappa[\circ := e])[\triangleright_j := \varphi_j]$ is a closed configuration.

Definition (LE-Reduction): The reduction relations $\xrightarrow{\lambda}$ and $\mapsto$ are extended to the generalized domains in the obvious fashion, simply by liberally annotating metavariables with ${}^{\circ}$'s. We omit the details.

Definition (LE-hole touching): If ${}^{\circ}\kappa = \left\langle\left\langle {}^{\circ}\alpha \mid {}^{\circ}\mu \right\rangle\right\rangle$, then ${}^{\circ}\kappa$ touches a hole at a if ${}^{\circ}\alpha(a) = [{}^{\circ}e]$ and either ${}^{\circ}e = {}^{\circ}R[\square := \circ[{}^{\circ}\sigma]]$ or ${}^{\circ}e = {}^{\circ}R[\square := \mathbf{app}(\triangleright_j[{}^{\circ}\sigma], {}^{\circ}v)]$. A transition from ${}^{\circ}\kappa[\circ := e][\triangleright_j := \varphi_j]$ touches a hole at a if the focus actor of the transition is a and ${}^{\circ}\kappa$ touches a hole at a .

Note that since an abstraction hole must be filled with a value, they are not touched in the same ways as arbitrary expression holes, in particular if the transition is a $\langle \mathbf{fun} : a \rangle$ execution step where ${}^{\circ}\alpha(a) = [{}^{\circ}e]$ and ${}^{\circ}e = {}^{\circ}R[\square := \mathbf{app}(\lambda x. {}^{\circ}e', \triangleright_j[{}^{\circ}\sigma])]$, then this is not considered touching the hole, $\triangleright_j$.

The (**E-Uniform Computation**) lemma generalizes to the situation with added abstraction holes.

Lemma (LE-Uniform Computation):

- (1) If ${}^{\circ}\kappa \xrightarrow{l} {}^{\circ}\kappa'$, then ${}^{\circ}\kappa[\circ := e][\triangleright_j := \varphi_j] \xrightarrow{l} {}^{\circ}\kappa'[\circ := e][\triangleright_j := \varphi_j]$ for any valid filling expression e and family of lambda abstractions φ_j .
- (2) If ${}^{\circ}\kappa$ has no transition with focus a (and a is an actor of ${}^{\circ}\kappa$), then either ${}^{\circ}\kappa$ touches a hole at a or ${}^{\circ}\kappa[\circ := e][\triangleright_j := \varphi_j]$ has no transition with focus a for any valid filling expression e and family of lambda abstractions φ_j .
- (3) If $\kappa \xrightarrow{l} \kappa'$ and $\kappa = {}^{\circ}\kappa[\circ := e][\triangleright_j := \varphi_j]$, then either the transition touches a hole or we can find ${}^{\circ}\kappa'$ such that $\kappa' = {}^{\circ}\kappa'[\circ := e][\triangleright_j := \varphi_j]$ and ${}^{\circ}\kappa \xrightarrow{l} {}^{\circ}\kappa'$.

Proof : Similar to the proof of (**E-uniform computation**) Now there are two cases in which a hole is touched in the decomposition of ${}^{\circ\triangleright}e$, namely cases (2) and (3) of the decomposition lemma. $\square$

7.3.3. LE-Main Theorem

Theorem (eq-reduct): Let $e_0, e_1, \varphi_{0,j}, \varphi_{1,j}$ for $j < J$ be such that for each ${}^{\circ\triangleright}\sigma$ we can find $j \in J$ such that $e_i[{}^{\circ\triangleright}\sigma]$ reduces uniformly via $\xrightarrow{\lambda}$ steps to $\varphi_{i,j}[{}^{\circ\triangleright}\sigma]$ for $i < 2$, and that for each ${}^{\circ\triangleright}\sigma, {}^{\circ\triangleright}v$, and $j \in J$ we can find ${}^{\circ\triangleright}e_c$ such that $\mathbf{app}(\varphi_{i,j}[{}^{\circ\triangleright}\sigma], {}^{\circ\triangleright}v)$ reduces uniformly via $\xrightarrow{\lambda}$ steps to ${}^{\circ\triangleright}e_c$ for $i < 2$. Then $e_0 \cong e_1$.

Corollary (eq-reduct): (**if.lam**) is an example. Here we take

$$\begin{aligned} e_0 &= (\lambda x. \mathbf{if}(v, e_a, e_b)) \\ e_1 &= \mathbf{if}(v, \lambda x. e_a, \lambda x. e_b) \quad \text{where } x \notin \text{FV}(v) \\ J &= \{a, b\} \\ \varphi_{0,j} &= \lambda x. \mathbf{if}(v, e_a, e_b) \\ \varphi_{1,j} &= \lambda x. e_j \text{ for } j \in J \end{aligned}$$

Proof : Let ${}^{\circ\triangleright}\kappa = \langle\langle {}^{\circ\triangleright}\alpha \mid {}^{\circ\triangleright}\mu \rangle\rangle$ be a closing LE-configuration for $e_0, e_1, \varphi_{0,j}, \varphi_{1,j} \mid j \in J$.

Assume $\pi_0 \in \mathcal{F}({}^{\circ\triangleright}\kappa[\circ := e_0][\triangleright_j := \varphi_{0,j}]) = [\kappa_i \xrightarrow{L_i} \kappa_{i+1} \mid i \in \mathbb{N}]$. We want to find ${}^{\circ\triangleright}\kappa_i$, and L_i , such that $\kappa_i = {}^{\circ\triangleright}\kappa_i[\circ := e_0]$ and, letting $\pi_1 = [{}^{\circ\triangleright}\kappa_i[\circ := e_1][\triangleright_j := \varphi_{1,j}] \xrightarrow{L_i} {}^{\circ\triangleright}\kappa_{i+1}[\circ := e_1][\triangleright_j := \varphi_{1,j}] \mid i \in \mathbb{N}]$, we have $\pi_1 \in \mathcal{F}({}^{\circ\triangleright}\kappa[\circ := e_1])$ and $\text{obs}(\pi_0) = \text{obs}(\pi_1)$. At each stage i we first consider dangling steps. Suppose ${}^{\circ\triangleright}\alpha_i(a) = [{}^{\circ\triangleright}R[\square := \circ[{}^{\circ\triangleright}\sigma]]]$ and the reduction of e_0 is trivial, i.e. $e_0[{}^{\circ\triangleright}\sigma] = \varphi_{0,j}[{}^{\circ\triangleright}\sigma]$ for some j . Then we prefix L_i with the transitions for $e_1[{}^{\circ\triangleright}\sigma] \xrightarrow{\lambda} \varphi_{1,j}[{}^{\circ\triangleright}\sigma]$ and convert the E-hole to $\triangleright_j$. This is done for each $a \in \text{Dom}({}^{\circ\triangleright}\alpha_i)$ meeting the condition.

Now we consider the decomposition of the configuration at stage $i+1$ in the case L_i touches a hole. Suppose L_i is an execution by a with ${}^{\circ\triangleright}\alpha_i(a) = [{}^{\circ\triangleright}R[\square := \circ[{}^{\circ\triangleright}\sigma]]]$. By the elimination of ‘dangling steps’ we may assume that e_0 is not a value expression and hence the execution occurs at the hole. Suppose also that $e_i[{}^{\circ\triangleright}\sigma] \xrightarrow{\lambda} \varphi_{i,j}[{}^{\circ\triangleright}\sigma]$. Define ${}^{\circ\triangleright}\alpha_{i+1} = {}^{\circ\triangleright}\alpha_i\{a := [{}^{\circ\triangleright}R[\square := \triangleright_j[{}^{\circ\triangleright}\sigma]]]\}$.

Suppose L_i is an execution by a with ${}^{\circ\triangleright}\alpha_i(a) = [{}^{\circ\triangleright}R[\square := \mathbf{app}(\triangleright_j[{}^{\circ\triangleright}\sigma], {}^{\circ\triangleright}v)]]$. Suppose also that $\varphi_{i,j}[{}^{\circ\triangleright}\sigma]$ reduces to $e_c[{}^{\circ\triangleright}\sigma]$ by $\xrightarrow{\lambda}$ steps. Define ${}^{\circ\triangleright}\alpha_{i+1} = {}^{\circ\triangleright}\alpha_i\{a := [{}^{\circ\triangleright}R[\square := e_c[{}^{\circ\triangleright}\sigma]]]\}$.

It is easy to check (as in the proof of (**fun-red-eq**)) that fairness is preserved. $\square$

7.4. Equivalence of Reduction Contexts

To establish the equivalence of reduction contexts, we define templates for syntactic entities — expressions, reduction context, redexes, configurations — with holes to be filled by a reduction context. We then proceed as before, to show how configurations can be suitably decomposed in order to define the desired path correspondences.

7.4.1. R-Syntax

We use $\diamond$ for reduction context holes and signify the corresponding syntactic entities with a mark $\diamond$. We prefix names of templates for syntactic classes by R-, thus expression templates are called R-expressions, etc.

Definition ($\diamond\mathbb{E} \quad \diamond\mathbb{V}$):

$$\diamond\mathbb{V} = \text{At} \cup \mathbb{X} \cup \lambda\mathbb{X}. \diamond\mathbb{E} \cup \mathbf{pr}(\diamond\mathbb{V}, \diamond\mathbb{V})$$

$$\diamond\mathbb{E} = \diamond\mathbb{V} \cup \Theta_n^e(\diamond\mathbb{E}^n) \cup \diamond^{\diamond\mathbb{S}}[\square := \diamond\mathbb{E}]$$

$$\diamond\mathbb{S} = \mathbb{X} \xrightarrow{f} \diamond\mathbb{V}$$

$\diamond e[\diamond := R]$ is the result of filling R-holes in $\diamond e$ with R . We give only the clause for the hole case in the recursive definition of filling.

Definition ($\diamond e[\diamond := R]$):

$$(\diamond^{\diamond\sigma}[\square := \diamond e])[\diamond := R] = R[\sigma][\square := \diamond e[\diamond := R]]$$

$$\text{where } \sigma = \diamond\sigma[\diamond := R] = \lambda x \in \text{Dom}(\diamond\sigma). \diamond\sigma(x)[\diamond := R]$$

Definition ($\diamond\mathbb{R} \quad \diamond\mathbb{E}_{\text{rdx}}$):

$$\diamond\mathbb{R} = \{\square\} \cup \Theta_{m+n+1}(\diamond\mathbb{V}^m, \diamond\mathbb{R}, \diamond\mathbb{E}^n) \cup \diamond^{\diamond\mathbb{S}}[\square := \diamond\mathbb{R}]$$

$$\diamond\mathbb{E}_{\text{rdx}} = \Theta_n^e(\diamond\mathbb{V}^n)$$

The clauses directly involving holes in the definitions of hole filling for R-reduction contexts are:

$$\square[\diamond := R] = \square$$

$$\square[\square := \diamond e] = \diamond e$$

$$(\diamond^{\diamond\sigma}[\square := \diamond R])[\diamond := R] = R^{\sigma}[\square := \diamond R[\diamond := R]]$$

$$\text{where } \sigma = \diamond\sigma[\diamond := R] = \lambda x \in \text{Dom}(\diamond\sigma). \diamond\sigma(x)[\diamond := R]$$

$$(\diamond^{\diamond\sigma}[\square := \diamond R])[\square := \diamond e] = \diamond^{\diamond\sigma}[\square := \diamond R[\square := \diamond e]]$$

Note that filling R-holes in R-expressions, R-reduction contexts, or R-redexes with a reduction context yields an expression, a reduction context, or redex, respectively.

An R-expression $\diamond e$ is either an R-value (element of $\diamond\mathbb{V}$) or it can be decomposed uniquely into an R-reduction context filled with either an R-redex or an R-hole.

Lemma (R-expression decomposition):

$$(0) \quad \diamond e \in \diamond\mathbb{V}, \quad \text{or}$$

$$(1) \quad (\exists! \diamond R, \diamond r)(\diamond e = \diamond R[\square := \diamond r]), \quad \text{or}$$

$$(2) \quad (\exists! \diamond R, \diamond\sigma, \diamond v)(\diamond e = \diamond R[\square := \diamond^{\diamond\sigma}[\square := \diamond v]])$$

Proof : An easy induction on the structure of $\diamond e$. $\square$

7.4.2. R-Configurations

Definition ($\diamond\mathbb{K}$): An R-configuration for reduction contexts $\diamond\kappa$ is formed like a configuration, but using R-expressions instead of simple expressions.

$$\begin{aligned}\diamond\mathbb{K} &= \left\langle\left\langle \diamond\mathbb{A}c \mid \diamond\mathbb{M} \right\rangle\right\rangle_{\chi}^{\rho} \\ \diamond\mathbb{A}c &= \mathbb{A}d \xrightarrow{f} \diamond\mathbb{A}s \\ \diamond\mathbb{A}s &= (\lambda x. \diamond\mathbb{E}) \cup [\diamond\mathbb{E}] \cup \{(\text{?}_{\mathbb{A}d})\} \\ \diamond\mathbb{M} &= \langle \diamond\mathbb{V} \Leftarrow \diamond\mathbb{V} \rangle\end{aligned}$$

We let $\diamond\kappa$ range over $\diamond\mathbb{K}$, and $\diamond\alpha$ range over $\diamond\mathbb{A}c$. Filling holes of an R-configuration is analogous to filling holes of an E-configuration. An R-configuration $\diamond\kappa$ is closing for R if $\diamond\kappa[\diamond := R]$ is a closed configuration.

7.4.3. R-Reduction

Definition (touching R-holes): A transition l from $\diamond\kappa[\diamond := R]$ touches an R-hole at a if l is an execution transition with focus a and execution state of $\diamond\kappa$ at a decomposes according to case (2) of the decomposition lemma.

7.4.4. R-Uniform Computation

Lemma (Uniform Computation): An R-redex reduces or hangs uniformly (for a given enabling occurrence in a configuration). Hence transitions not touching an R-hole are uniform. More precisely, if $\diamond\kappa_i[\diamond := R] \xrightarrow{l_i} \kappa_{i+1}$, with focus a , that does not touch an R-hole at a , then l_i is either a receive or an execution transition in which the execution state of $\diamond\kappa_i$ at a decomposes according to case (1) of the decomposition lemma. Let $\diamond\kappa_i = \left\langle\left\langle \diamond\alpha_i \mid \diamond\mu_i \right\rangle\right\rangle$.

Then the decomposition of $\kappa_{i+1} = \left\langle\left\langle \diamond\alpha_{i+1} \mid \diamond\mu_{i+1} \right\rangle\right\rangle$ is defined as follows.

Receive: In the receive case we must have that $\diamond\alpha_i(a) = (\lambda x. \diamond e)$. Thus $\diamond\alpha_{i+1} = \diamond\alpha_i\{a := [\text{app}(\lambda x. \diamond e, cv)]\}$, and $\diamond\mu_i = \diamond\mu_{i+1} + \langle a \Leftarrow cv \rangle$.

Uniform execution: In the uniform execution case $\diamond\alpha_i(a) = [\diamond e]$, where $\diamond e$ has the form $\diamond R[\square := \diamond r]$. In this case the step is independent of what fills the holes. Thus we can find $\diamond\kappa_{i+1}$ such that $\diamond\kappa_i \xrightarrow{l_i} \diamond\kappa_{i+1}$ uniformly.

Now we show how to establish equivalence of expressions of the form $R_j[\square := e]$ for $j < 2$ where the $R_j[\square := v]$ have a common reduct for any value expression.

Theorem (eq-r): If for z fresh, there is some e such that $R_j[\square := z]$ reduces uniformly via 0 or more $\xrightarrow{\lambda}$ steps to e for $j < 2$, then $R_0[\square := e] \cong R_0[\square := e]$ for any e .

Corollary (eq-r): (**app**), (**cmps**), (**id**), (**letx**), (**let.dist**), (**if.dist**) are instances of (**eq-r**).

In fact we prove a slightly more general result, since we use a weaker assumption on the reduction points: for each $\diamond\sigma$, $\diamond v$, we can find $\diamond e_2$ such that $R_j^{\diamond\sigma}[\square := \diamond v]$ reduces uniformly via 0 or more $\xrightarrow{\lambda}$ steps to $\diamond e_2$ for $j < 2$.

Proof (eq-r): Suppose R_0, R_1 are reduction points that we wish to establish the observational equivalence of. I.e. we want to show $R_0[\Box := e] \cong R_1[\Box := e]$ for all expressions e . Let $\diamond\kappa = \langle\langle \diamond\alpha \mid \diamond\mu \rangle\rangle$ be a closing R-configuration for R_0, R_1 . Assume $\pi_0 \in \mathcal{F}(\diamond\kappa[\Box := R_0]) = [\kappa_i \xrightarrow{L_i} \kappa_{i+1} \mid i \in \mathbb{N}]$. We want to find $\diamond\kappa_i$, and L_i , such that $\kappa_i = \diamond\kappa_i[\Box := R_0]$ and, letting $\pi_1 = [\diamond\kappa_i[\Box := R_1] \xrightarrow{L_i} \diamond\kappa_{i+1}[\Box := R_1] \mid i \in \mathbb{N}]$, we have $\pi_1 \in \mathcal{F}(\diamond\kappa[\Box := R_1])$ and $obs(\pi_0) = obs(\pi_1)$. As in the expression context case, we can focus our attention on the construction of the actor configuration part, since here also deliverable messages can not have holes. For the base case we have $\diamond\alpha_0 = \diamond\alpha$. Assume we have $\diamond\alpha_i$ and consider cases on the transition label L_i . We have three cases to consider. For the base case we have $\diamond\alpha_0 = \diamond\alpha$. At stage i suppose $\diamond\alpha_i(a') = \diamond R[\Box := \diamond^\sigma[\Box := \diamond v]]$. If $R_0^\sigma[\Box := \diamond v]$ is the common form (i.e. the reduction is trivial), then we prefix L_i with steps for reduction of $R_1^\sigma[\Box := \diamond v]$ to common form and remove this hole. This is carried out for each a' in the domain of $\diamond\alpha_i$.

Now, suppose L_i is an execution with focus a that touches a hole. Thus $\diamond\alpha_i(a) = [\diamond e]$ where $\diamond e$ has the form $\diamond R[\Box := \diamond^\sigma[\Box := \diamond v]]$. Suppose also that $R_j^\sigma[\Box := \diamond v] \xrightarrow{\lambda} \diamond e_c$ for $j \in 2$. Then we define $\diamond\alpha_{i+1} = \diamond\alpha_i\{a := [\diamond R[\Box := \diamond e_2]]\}$.

Let $\pi_1 = [\diamond\kappa_i[\Box := e_1] \xrightarrow{L_i} \diamond\kappa_{i+1}[\Box := e_1] \mid i < \mathbb{N}]$ be the constructed computation path. Note that the transitions are the same except for the points where holes are touched, but these differences are not observable. Clearly, under the assumptions on R_j , π_1 is a computation path. It remains to show that the construction preserves fairness.

Suppose some transition l is enabled at stage i in π_1 . If l is a receive or uniform execution, then l is also enabled in π_0 at stage i and will eventually occur, uniformly. Suppose l is an execution step by a with $\diamond\alpha_i(a) = [\diamond R[\Box := \diamond^\sigma[\Box := \diamond v]]]$. Either $R_0^\sigma[\Box := \diamond v]$ is in common form and the transition occurs as soon as it is enabled in π_1 , or a transition is enabled at this hole in π_0 . This transition will eventually occur. If $R_1^\sigma[\Box := \diamond v]$ is not in common form, l will happen in π_1 at the same stage. If $R_1^\sigma[\Box := \diamond v]$ is in common form, then we must consider whether l occurs at the hole – i.e. whether or not the common form is an R-value expression. If l occurs at the hole, then the same transition is now enabled in π_0 and will eventually occur in both paths. Otherwise consider the decomposition after the transition in π_0 . As shown before, this reduces to considering a proper subexpression of $\diamond R[\Box := \diamond^\sigma[\Box := \diamond v]]$ and the process will eventually terminate with a uniform execution.

□_{eq-r}

8. Discussion

In this paper we have presented a mathematical theory of actor computation.

8.1. Highlights

The highlights of this theory include:

- (1) a language for actor computation which is an extension of the call-by-value lambda calculus that includes primitives for creating and manipulating actors;

- (2) the concept of an *actor configuration* that makes explicit the notion of open system component;
- (3) an operational semantics defined by a labelled transition relation on configurations;
- (4) a notion of computation that incorporates fairness in an essential way;
- (5) a notion of observational equivalence of expressions based on traditional operational and testing equivalence;
- (6) a proof that in the presence of fairness the classic three testing equivalences collapse to two;
- (7) a plethora of laws of expression equivalence that incorporates the equational theory of the embedded functional language;
- (8) a composition operation on actor configurations that is associative, commutative, has a unit, and is thus a first step towards an algebra of configurations;
- (9) a notion of interaction equivalence of actor configurations that generalizes traditional trace equivalence;
- (10) a notion of observational equivalence of actor configurations that generalizes the same notion on expressions;
- (11) a collection of results that establish that interaction equivalence implies observational equivalence, and is a congruence with respect to composition;
- (12) examples of interaction equivalent configurations, and methods for establishing their equivalence;
- (13) methods for establishing equivalence of expressions.

8.2. Future Directions

The theory presented is perhaps best viewed as a starting point for further research rather than a final product. In particular there are at least 3 directions for further research:

- (a) developing an algebra of operations on configurations;
- (b) a systematic study of notions of equivalence within the actor theory;
- (c) developing a logic for specifying components as actor configurations, methods for verifying that programs implementing components meet their specifications, and methods for refining specifications into implementations.

and 3 open problems:

- (a) does observational equivalence imply interaction equivalence?
- (b) is the conjecture (**io-exp**) true?
- (c) is there a context lemma (similar to (**ciu**)) for observational equivalence of expressions?

Furthermore, in hindsight, the theory as presented here might benefit from:

- (a) replacing the **newadr**, **initbeh** choice of language primitives by a **letactor** primitive;
- (b) making the operational semantics truly concurrent;

- (c) mechanical checking of the more tedious parts of theory, for example the (**independence**) lemma.

Acknowledgements

The authors would like to thank Carl Hewitt and Richard Weyhrauch for many discussions about actor computation that served as a foundation for this work.

This research was partially supported by DARPA contract NAG2-703, NSF grants CCR-8917606, CCR-8915663, CCR-9109070 and INT-89-20626, and DARPA and NSF joint contract CCR 90-07195, ONR contract N00014-90-J-1899, and by the Digital Equipment Corporation.

9. References

- [1] G. Agha. *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press, Cambridge, Mass., 1986.
- [2] Gul Agha. Concurrent object-oriented programming. *Communications of the ACM*, 33(9):125–141, September 1990.
- [3] G. Attardi and C. Hewitt. Proving properties of concurrent programs expressed as behavioral specifications. In *Summer School on Foundations of Artificial Intelligence and Computer Science (Pisa 1978)*, 1978.
- [4] G. Attardi and C. Hewitt. Specifying and proving properties of guardians for distributed systems, 1978.
- [5] Henry G. Baker and Carl Hewitt. Laws for communicating parallel processes. In *IFIP Congress*, pages 987–992. IFIP, August 1977.
- [6] D. Berry, R. Milner, and D.N. Turner. A semantics for ML concurrency primitives. In *Conference record of the 19th annual ACM symposium on principles of programming languages*, pages 105–129, 1992.
- [7] S.D. Brookes, C.A.R. Hoare, and A.W. Roscoe. A Theory of Communicating Sequential Processes. *Journal of the ACM*, 31(3):560–599, July 1984.
- [8] R. de Nicola and M. C. B. Hennessy. Testing equivalences for processes. *Theoretical Computer Science*, 34:83–133, 1984.
- [9] D. L. Dill. *Trace Theory for Automatic Hierarchical Verification of Speed-Independent Circuits*. ACM Distinguished Dissertations. MIT Press, 1989.
- [10] M. Felleisen and R. Hieb. The revised report on the syntactic theories of sequential control and state. *Theoretical Computer Science*, 103:235–271, 1992.
- [11] M. Felleisen and A. K. Wright. A syntactic approach to type soundness. Technical Report Rice COMP TR91-160, Rice University Computer Science Department, 1991.
- [12] C. Hewitt. *Description and Theoretical Analysis (Using Schemata) of PLANNER: A Language for Proving Theorems and Manipulating Models in a Robot*. PhD thesis, MIT, 1971.

- [13] C. Hewitt. Viewing control structures as patterns of passing messages. *Journal of Artificial Intelligence*, 8(3):323–364, 1977.
- [14] C. Hewitt and R. Atkinson. Synchronization in actor systems. In *Conference record of ACM symposium on principles of programming languages*, pages 267–280, 1977.
- [15] C. A. R. Hoare. *Communicating Sequential Processes*. Prentice-Hall, 1985.
- [16] H. Lieberman. A preview of act i. AI-TR- 625, MIT Artificial Intelligence Laboratory, 1981.
- [17] H. Lieberman. Thinking about lots of things at once without getting confused: Parallelism in act i. AI-TR- 626, MIT Artificial Intelligence Laboratory, 1981.
- [18] I. A. Mason and C. L. Talcott. Equivalence in functional languages with effects. *Journal of Functional Programming*, 1:287–327, 1991.
- [19] J. Meseguer. Rewriting as a unified model of concurrency. In *Proceedings of Concur'90*, volume 458 of *Lecture Notes in Computer Science*, pages 384–400. Springer-Verlag, 1990.
- [20] R. Milner, J. G. Parrow, and D. J. Walker. A calculus of mobile processes, parts i and ii. Technical Report ECS-LFCS-89-85, -86, Edinburgh University, 1989.
- [21] E. Moggi. Computational lambda-calculus and monads. Technical Report ECS-LFCS-88-86, University of Edinburgh, 1988.
- [22] J. H. Morris. *Lambda calculus models of programming languages*. PhD thesis, Massachusetts Institute of Technology, 1968.
- [23] G. Plotkin. Call-by-name, call-by-value and the lambda-v-calculus. *Theoretical Computer Science*, 1:125–159, 1975.
- [24] J. H. Reppy. An operational semantics of first-class synchronous operations. Technical Report TR 91-1232, Cornell University, 1991.
- [25] J. H. Reppy. *Higher-Order Concurrency*. PhD thesis, Cornell University, 1992. available as Technical Report 92-1285.
- [26] V. Sassone, M. Nielsen, and G. Winskel. A hierarchy of models for concurrency. In *The Fourth International Conference on Concurrency Theory (CONCUR '93)*, Lecture Notes in Computer Science. Springer Verlag, 1993.
- [27] C. L. Talcott. *The essence of Rum: A theory of the intensional and extensional aspects of Lisp-type computation*. PhD thesis, Stanford University, 1985.
- [28] C. L. Talcott. Programming and proving with function and control abstractions. Technical Report STAN-CS-89-1288, Stanford University Computer Science Department, 1989.
- [29] C. L. Talcott. A theory for program and data specification. In *Design and Implementation of Symbolic Computation Systems, DISCO'90*, volume 429 of *Lecture Notes in Computer Science*, pages 91–100. Springer-Verlag, 1990. Full version to appear in TCS special issue.

- [30] C. L. Talcott. Binding structures. In Vladimir Lifschitz, editor, *Artificial Intelligence and Mathematical Theory of Computation*. Academic Press, 1991.
- [31] C. L. Talcott. A theory for program and data specification. *Theoretical Computer Science*, 104:129–159, 1993.
- [32] C.L. Talcott. A theory of binding structures and its applications to rewriting. *Theoretical Computer Science*, to appear, 1991.
- [33] A. Yonezawa. *ABCL: An Object-Oriented Concurrent System*. MIT Press, Cambridge Mass., 1990.

10. Index of Notations

Symbol	Description	§
$\mathbb{N}$	The natural numbers, $i, j, \dots, n \in \mathbb{N}$	1.3
Y^n	Sequences from Y of length n , $\bar{y} = [y_1, \dots, y_n] \in Y^n$	1.3
Y^*	Finite sequences from Y	1.3
$[]$	The empty sequence	1.3
$\text{Len}(\bar{y})$	The length of the sequence $\bar{y}$	1.3
$u * v$	The concatenation of the sequences u and v	1.3
$\text{Last}(u)$	The last element of the sequence u	1.3
$\mathbf{P}_\omega[Y]$	Finite subsets of Y	1.3
$\mathbf{M}_\omega[Y]$	Finite multi-sets with elements in Y	1.3
$Y_0 \xrightarrow{f} Y_1$	Finite maps from Y_0 to Y_1	1.3
$Y_0 \rightarrow Y_1$	Total functions from Y_0 to Y_1	1.3
$\text{Dom}(f)$	The domain of the function f	1.3
$\text{Rng}(f)$	The range of the function f	1.3
$f\{y := y'\}$	An extension to, or alteration of, the function f	1.3
$f _Y$	The restriction of f to the set Y	1.3
$\mathbb{X}$	A countably infinite set of variables, $x, y, z \in \mathbb{X}$	2.1
At	Atoms	2.1
$\mathbf{t}, \mathbf{nil}$	Atoms playing the role of booleans	2.1
$\mathbb{F}$	Operations, $\delta \in \mathbb{F}$	2.1
$\mathbb{F}_n$	n -ary operation symbols	2.1
$\mathbb{F}_0$	Zero-ary operation symbols $\supseteq \{\mathbf{newadr}\}$	2.1
$\mathbb{F}_1$	Unary operation symbols $\supseteq \{\mathbf{isatom}, \mathbf{isnat}, \mathbf{ispr}, \mathbf{1^{st}}, \mathbf{2^{nd}}, \mathbf{become}\}$	2.1
$\mathbb{F}_2$	Binary operation symbols $\supseteq \{\mathbf{pr}, \mathbf{initbeh}, \mathbf{send}\}$	2.1
$\mathbb{F}_3$	Ternary operation symbols $\supseteq \{\mathbf{br}\}$	2.1
$\mathbb{L}$	λ -abstractions, $\lambda x.e \in \mathbb{L}$	2.1
$\mathbb{V}$	Value expressions, $v \in \mathbb{V}$	2.1
$\mathbb{E}$	Expressions, $e \in \mathbb{E}$	2.1
$\lambda x.e$	Abstractions	2.1
$\mathbf{app}(e_0, e_1)$	Application	2.1
$\delta(\bar{e})$	Application of operations	2.1
$\mathbf{if}(e_0, e_1, e_2)$	Conditional branching	2.1
$\mathbf{let}\{x := e_0\}e_1$	Lexical variable binding	2.1
$\mathbf{seq}(e_1, \dots, e_n)$	Sequencing construct	2.1
$\text{FV}(e)$	The free variables of the expression e	2.1
$e\bar{x}[e']$	The result of substituting e' for x in e	2.1
$\mathbb{C}$	Contexts, $C \in \mathbb{C}$	2.1
$\bullet$	The hole in contexts	2.1
$C[e]$	The result of filling the context with e	2.1

Symbol	Description	§
$\mathbb{A}d$	Actor addresses, identified with $\mathbb{X}$	2.2
$\mathbb{A}s$	Actor states, $(?_a), (b), [e] \in \mathbb{A}s$	2.2
$(?_a)$	An uninitialized actor created by a	2.2
(b)	An actor with behavior b ready to accept a message	2.2.1
$[e]$	A processing actor with current computation e	2.2.1
$\mathbb{M}$	Messages, $\langle \mathbb{V} \Leftarrow \mathbb{V} \rangle \in \mathbb{M}$	2.2.1
$c\mathbb{V}$	Communicable values, $cv \in c\mathbb{V}$	2.2.1
$\langle\langle \alpha \mid \mu \rangle\rangle_\chi^\rho$	An actor configuration with: α – an actor map μ – a multi-set of messages ρ – the receptionists χ – the external actors	2.2.1
$\mathbb{K}$	Actor configurations, $\kappa \in \mathbb{K}$	2.2.2
$\mathbb{E}_{rdx}$	The set of redexes, $r \in \mathbb{E}_{rdx}$	2.2.2
$\mathbb{R}$	The set of reduction contexts, $R \in \mathbb{R}$	2.2.2
$\square$	The reduction context hole	2.2.2
$\xrightarrow{\lambda}$	The reduction relation for functional redexes, $e_0 \xrightarrow{\lambda} e_1$	2.2.2
$\mapsto$	The reduction relation for configurations, $\kappa_0 \mapsto \kappa_1$	2.2.2
$\kappa_0 \xrightarrow{l} \kappa_1$	$\kappa_0 \mapsto \kappa_1$ via the rule labelled by l	2.2.2
Labels :	Transition labels, $l \in \text{Labels}$	2.2.2
$\langle \text{fun} : a \rangle$	A functional transition	2.2.2
$\langle \text{new} : a, a' \rangle$	newadr redex transition with focus a	2.2.2
$\langle \text{init} : a, a' \rangle$	initbeh redex transition with focus a	2.2.2
$\langle \text{bec} : a, a' \rangle$	become redex transition with focus a	2.2.2
$\langle \text{send} : a, m \rangle$	send redex transition with focus a	2.2.2
$\langle \text{rcv} : a, cv \rangle$	The receipt of a message with focus a	2.2.2
$\langle \text{out} : m \rangle$	A message exiting the configuration	2.2.2
$\langle \text{in} : m \rangle$	A message entering the configuration	2.2.2
$\mathcal{T}(\kappa)$	All finite sequences of labeled transitions from κ , $\nu \in \mathcal{T}(\kappa)$	2.2.3
$\mathcal{T}^\infty(\kappa)$	the set of all computation paths in $\mathcal{T}(\kappa)$, $\pi \in \mathcal{T}^\infty(\mathbb{K})$	2.2.3
$\bowtie \in \mathbb{N} \cup \{\omega\}$	The length of a finite or infinite sequence	2.2.3
$\text{Cfig}(\kappa, L, i)$	The i th configuration of the computation from κ via L	2.2.3
$\kappa \xrightarrow{L} \kappa'$	A multi-step transition	2.2.3
$\mathcal{F}(\kappa)$	the fair subset of $\mathcal{T}^\infty(\kappa)$	2.2.4
event	A zero-ary primitive/observation	3.1
$\langle \mathbf{e} : a \rangle$	An observation transition	3.1
$\langle\langle \alpha, [C]_a \mid \mu \rangle\rangle$	An observing configuration	3.1
$\mathbb{O}$	The set of observing configurations, $O \in \mathbb{O}$	3.1

Symbol	Description	§
s	Signifies that an event transition occurs	3.1
f	Signifies that an event transition does not occur	3.1
$obs(\pi)$	The s/f classification of the path π	3.1
$Obs(\kappa)$	The s/f classification of the configuration, $\kappa, \in \{\mathbf{s}, \mathbf{f}, \mathbf{sf}\}$	3.1
$e_0 \cong_1 e_1$	Testing or Convex or Plotkin or Egli-Milner equivalence	3.2
$e_0 \cong_2 e_1$	Must or Upper or Smyth equivalence	3.2
$e_0 \cong_3 e_1$	May or Lower or Hoare equivalence	3.2
$e_0 \cong e_1$	Operational equivalence (either $\cong_1$ or equivalently $\cong_2$)	3.2
<i>Hang</i>	The set of all stuck expressions, $\mathbf{stuck} \in \text{Hang}$	4.1
<i>Infin</i>	The set of all diverging expressions, $\mathbf{bot} \in \text{Infin}$	4.1
$\kappa_0 \parallel \kappa_1$	The composition of the two configurations	5.1
i/o	The interactions (Input/Output) of a path or configuration	5.1.1
<i>sMerged</i>	The merge relation on finite computations	5.1.1
$\langle \mathbf{io} : a, cv \rangle$	A pseudo transition	5.1.1
$\mathcal{T}(\kappa_0) \parallel \mathcal{T}(\kappa_1)$	The merge operation on trees	5.1.2
$\kappa_0 \cong_i \kappa_1$	Interaction equivalence between configurations	5.2.1
$\kappa_0 \cong \kappa_1$	Observational equivalence between configurations	5.2.2
$\langle \mathbf{exec} : a \rangle$	Any transition with <i>focus</i> a , other than rcv	5.3
$l_0 - l_1$	l_0 and l_1 are independent transitions	5.3
o^σ	An expression hole with substitution $^\sigma$	7.1
Θ_n	Syntactic constructs of arity n	7.1
Θ_n^e	Syntactic constructs of arity n (including event)	7.1
$^\circ \mathbb{E}$	Expressions with expression holes, $^\circ e \in ^\circ \mathbb{E}$	7.2.1
$^\circ \mathbb{V}$	Value expressions with expression holes, $^\circ v \in ^\circ \mathbb{V}$	7.2.1
$^\circ \mathbb{S}$	Substitutions with expression holes, $^\circ \sigma \in ^\circ \mathbb{S}$	7.2.1
$^\circ \mathbb{R}$	Reduction contexts with expression holes, $^\circ R \in ^\circ \mathbb{R}$	7.2.1
$^\circ \mathbb{E}_{\text{rdx}}$	Redexes with expression holes, $^\circ r \in ^\circ \mathbb{E}_{\text{rdx}}$	7.2.1
$^\circ e[o := e]$	Expression hole filling	7.2.1
$^\circ \mathbb{K}$	Configurations with expression holes $^\circ \kappa \in ^\circ \mathbb{K}$	7.2.3
$^\circ \mathbb{A}c$	An actor map with expression holes $^\circ \alpha \in ^\circ \mathbb{A}c$	7.2.3
$^\circ \mathbb{M}$	Messages with expression holes	7.1

Symbol	Description	§
$\triangleright_j$	A family of abstraction holes	7.3
$X^{\circ\triangleright}$	The syntactic class X with expression and abstraction holes	7.3
${}^{\circ\triangleright}\mathbb{V}$	Value expressions with expression and abstraction holes, ${}^{\circ\triangleright}v \in {}^{\circ\triangleright}\mathbb{V}$	7.3
${}^{\circ\triangleright}\mathbb{E}$	Expressions with expression and abstraction holes, ${}^{\circ\triangleright}e \in {}^{\circ\triangleright}\mathbb{E}$	7.3
$\triangleright_N^{\circ\triangleright\mathbb{S}}$	Abstraction holes annotated with substitutions	7.3
${}^{\circ\triangleright}e[\triangleright_j := \varphi_j]$	Abstraction hole filling	7.3
$\diamond^{\circ\sigma}$	A reduction context hole	7.4
${}^{\circ}\mathbb{E}$	Expressions with reduction context holes, ${}^{\circ}e \in {}^{\circ}\mathbb{E}$	7.4
${}^{\circ}\mathbb{V}$	Value expressions with reduction context holes, ${}^{\circ}v \in {}^{\circ}\mathbb{V}$	7.4
${}^{\circ}\mathbb{S}$	Substitutions with reduction context holes ${}^{\circ}\sigma \in {}^{\circ}\mathbb{S}$	7.3
${}^{\circ}e[\diamond := R]$	Reduction context hole filling	7.4
${}^{\circ}\mathbb{R}$	Reduction contexts with reduction context holes, ${}^{\circ}R \in {}^{\circ}\mathbb{R}$	7.4
${}^{\circ}\mathbb{E}_{\text{rdx}}$	Redexes with reduction context holes, ${}^{\circ}r \in {}^{\circ}\mathbb{E}_{\text{rdx}}$	7.4
${}^{\circ}\mathbb{K}$	Configurations with reduction context holes, ${}^{\circ}\kappa \in {}^{\circ}\mathbb{K}$	7.4
${}^{\circ}\mathbb{Ac}$	Actor maps with reduction context holes, ${}^{\circ}\alpha \in {}^{\circ}\mathbb{Ac}$	7.4
${}^{\circ}\mathbb{M}$	Messages with reduction context holes	7.4
${}^{\circ}\mathbb{As}$	Actor states with reduction context holes	7.4

Contents

1. Introduction	1
1.1. Overview	1
1.2. The Actor Model of Computation	2
1.2.1. The Actor Philosophy	2
1.2.2. Related Models of Concurrency	3
1.2.3. Actor Primitives	4
1.2.4. Trivial Examples	4
1.2.5. Actor Cells	5
1.2.6. Join Continuations	5
1.3. Notation.	7
2. A Simple Lambda Based Actor Language	7
2.1. Syntax	7
2.2. Reduction Semantics for Actor Configurations	8
2.2.1. Actor Configurations	9
2.2.2. Decomposition and Reduction	10
2.2.3. Computation Sequences and Paths	12
2.2.4. Fairness	13
3. Equivalence of Expressions	13
3.1. Events	14
3.2. Three Equivalences	14
3.3. Partial Collapse	15
4. Laws of Expression Equivalence	17
4.1. Functional Laws	17
4.2. Basic Laws for Actor Primitives	19
4.2.1. Commuting Operations	19
5. Manipulation of Actor Configurations	22
5.1. Composition of Actor Configurations	23
5.1.1. Projection and Merging of Sequences	23
5.1.2. The Merging of Paths	27
5.2. Equivalence of Actor Configurations	30
5.2.1. Interaction Equivalence	30
5.2.2. Observational Equivalence	32
5.3. Transforming Paths	33
6. Examples of Configuration Equivalence	36
6.1. Indirection Elimination	36

6.2. Varieties of Recursion as Patterns of Message Passing	40
7. Proving Expression Equivalence	46
7.1. The General Method	46
7.2. Common Reduct Case	48
7.2.1. E-Syntax	48
7.2.2. E-Expression Decomposition	50
7.2.3. E-Configurations	50
7.2.4. E-Reduction	52
7.2.5. E-Uniform Computation	53
7.2.6. The (fun-red-eq) Theorem	54
7.2.7. The Proof of the (hang-infin) Theorem	57
7.2.8. The Proofs of the Actor Primitive Laws	58
7.3. Equivalence by Two Stage Reduction	60
7.3.1. LE-Syntax	60
7.3.2. LE-computation	61
7.3.3. LE-Main Theorem	62
7.4. Equivalence of Reduction Contexts	62
7.4.1. R-Syntax	63
7.4.2. R-Configurations	64
7.4.3. R-Reduction	64
7.4.4. R-Uniform Computation	64
8. Discussion	65
8.1. Highlights	65
8.2. Future Directions	66
9. References	67
10. Index of Notations	69